



Indira Gandhi National Open University
School of Computer and Information
Sciences (SOCIS)



MCS-068PREDICTIVE DATA ANALYSIS

BLOCK 1: DATA ANALYSIS – AN INTRODUCTION

BLOCK 2: PREDICTIVE DATA ANALYSIS – I

BLOCK 3: PREDICTIVE DATA ANALYSIS – II

BLOCK 4: R - PROGRAMMING FOR DATA ANALYSIS

BLOCK 3: PREDICTIVE DATA ANALYSIS - II

UNIT 8: REGRESSION MODELS

UNIT 9: TIME SERIES MODELS

UNIT 10: SUPERVISED LEARNING MODELS

UNIT 11: UNSUPERVISED LEARNING MODELS

UNIT 12: PERFORMANCE EVALUATION MEASURES

PROGRAMME DESIGN COMMITTEE

Prof. D. K. Lobiyal, JNU, New Delhi	Prof. Sandeep Singh Rawat, SOCIS, IGNOU
Prof. Dinesh Kumar Vishwakarma, DTU, New Delhi	Prof. P.V. Suresh, SOCIS, IGNOU
	Prof. V.V. Subrahmanyam, SOCIS, IGNOU
	Prof. Divakar Yadav, SOCIS, IGNOU
	Dr. Akshay Kumar, Associate Professor, SOCIS IGNOU
	Dr. M.P. Mishra, Associate Professor, SOCIS IGNOU
	Dr. Sudhansh Sharma, Assistant Professor, SOCIS IGNOU

COURSE DESIGN COMMITTEE

Prof. D. K. Lobiyal, JNU, New Delhi	Prof. Sandeep Singh Rawat, SOCIS, IGNOU
Prof. Dinesh Kumar Vishwakarma, DTU, New Delhi	Prof. P.V. Suresh, SOCIS, IGNOU
Prof. T. V. Vijay Kumar, JNU, New Delhi	Prof. V.V. Subrahmanyam, SOCIS, IGNOU
Prof. D. P. Vidyarthi, JNU, New Delhi	Prof. Divakar Yadav, SOCIS, IGNOU
Prof. Vivek Kumar Singh, University of Delhi, Delhi	Dr. Akshay Kumar, Associate Professor, SOCIS IGNOU
Prof. A. K. Mohapatra, IGDTUW, New Delhi	Dr. M.P. Mishra, Associate Professor, SOCIS IGNOU
Mr. Kapil Pandey, HCL Technologies Limited, Delhi – NCR	Dr. Sudhansh Sharma, Assistant Professor, SOCIS IGNOU
Prof. Manish Trivedi, SOS (Statistics), IGNOU	
Dr. Rajesh Kaliraman, Assistant Professor, SOS (Statistics), IGNOU	
Dr. Prabhat Kumar Sangal, Assistant Professor, SOS (Statistics), IGNOU	

SOCIS FACULTY

Prof. Sandeep Singh Rawat, Director, SOCIS, IGNOU	Prof. P.V. Suresh, IGNOU, SOCIS, IGNOU
Prof. V.V. Subrahmanyam, SOCIS, IGNOU	Prof. Divakar Yadav, SOCIS, IGNOU
Prof. Akshay Kumar, SOCIS, IGNOU	Prof.M.P. Mishra, SOCIS, IGNOU
Dr. Sudhansh Sharma, Associate Professor, SOCIS, IGNOU	

COURSE PREPARATION TEAM

Content Writer (Unit-8,9,10,11)

Prof. Durgansh Sharma
School of Business and Management
Christ University-Delhi (NCR Campus)

Content Writer (Unit-12)

Dr. Jaidev Sharma, Assistant Professor
School of Engineering and Sciences
Center for Artificial Intelligence in medicine,
imaging & forensics,
Sharda University, Greater Noida, U.P.

Course Coordinator

Dr. Sudhansh Sharma, Associate Professor,
School of Computer and Information Sciences, IGNOU

PRINT PRODUCTION

Sh. Sanjay Aggarwal
Assistant Registrar, MPDD, IGNOU, New Delhi

Content Editor (Unit-8,9,10,11,12)

Prof. Manish Trivedi
Director, School of Sciences, IGNOU

Language Editor

Prof. Parmod Kumar
School of Humanities, IGNOU

,2026

©Indira Gandhi National Open University, 2026

ISBN – xx-xxxx-xxx-x

All rights reserved. No part of this work may be reproduced in any form, by mimeography or any other means, without permission in writing from the Indira Gandhi National Open University.

Further information on the Indira Gandhi National Open University courses may be obtained from the University's office at Maidan Garhi, New Delhi 110068.

Printed and published on behalf of the Indira Gandhi National Open University, New Delhi by MPDD, IGNOU.

BLOCK INTRODUCTION (BLOCK 3)

Block-3, titled *Predictive Data Analysis-II*, forms the core analytical component of the course *Predictive Data Analysis*. It focuses on advanced predictive techniques and machine learning models used for data-driven decision-making. The block consists of five units (Unit 8 to Unit 12) covering predictive modeling, forecasting, classification, clustering, and model evaluation.

Unit-8: Regression Models introduces techniques such as linear, multiple, polynomial, ridge, lasso, and support vector regression for predicting continuous outcomes.

Unit-9: Time Series Models covers forecasting methods including ARIMA, SARIMA, LSTM, and Bi-LSTM, along with trend and seasonal analysis techniques.

Unit-10: Supervised Learning Models explains classification algorithms such as KNN, Naïve Bayes, Logistic Regression, Decision Trees, Random Forest, and SVM.

Unit-11: Unsupervised Learning Models focuses on clustering and association rule techniques including K-Means, Hierarchical Clustering, and Agglomerative Clustering.

Unit-12: Performance Evaluation Measures introduces metrics such as accuracy, precision, recall, F1-score, ROC curve, and PR curve for evaluating predictive models.

This block is highly important in both industry and academia. Industries such as finance, healthcare, marketing, and e-commerce use these models for forecasting, classification, and decision-making, while academia uses them for advanced research in machine learning and data analytics.

Overall, Block-3 equips learners with essential predictive modeling and evaluation skills required for solving real-world analytical problems and conducting advanced data-driven research.

UNIT 8 – REGRESSION MODELS

8.0 Introduction

8.1 Expected Learning Outcomes

8.2 Introduction to Regression

8.3 Regression Models

8.3.1 Linear Regression

8.3.2 Multiple Regression

8.3.3 Polynomial Regression

8.3.4 Ridge and Lasso Regression

8.3.5 Support Vector Regression

8.4 Summary

8.5 Answers

8.6 References/Further Readings

8.0 INTRODUCTION

Regression analysis is a key method in statistics that allows us to make predictions and understand how different factors and variables are connected. Currently, where decisions are often based on analyzing large amounts of data—learning about regression models is a crucial skill for anyone interested in data science or analytics for better decision-making.

Regression analysis models help the learner to figure out how one thing changes based on others. For example, a company might want to predict next month's sales based on how much they spend on advertising and the number of sales calls made. These influencing factors, like advertising spending and sales calls, are called predictor variables. The thing you are trying to predict, such as sales, is known as the dependent variable. Similarly, a student could use regression to see how their study hours and attendance affect their test scores, or a homebuyer might want to estimate house prices based on location and size.

Regression models are used everywhere—from businesses forecasting revenue, to doctors estimating disease risks, and to engineers predicting material performance. By using regression, you can make informed guesses about future outcomes and learn which factors matter most.

There are several types of regression models. The simplest, linear regression, looks for a straight-line relationship between variables. Sometimes, relationships are more complex; that's where models like polynomial regression or support vector regression come in, as they can capture curved or intricate patterns in the data.

As you build more advanced models, you might encounter problems like overfitting. Overfitting happens when a model matches the training data too closely and fails to predict new data accurately. This is like memorizing answers for a test rather than understanding the material—great for the practice test, but not for new questions. To help prevent overfitting, special techniques such as Ridge and Lasso regression are used. These techniques add

penalties to the model to keep it from becoming too complex, encouraging simpler, more reliable predictions.

Learning about regression isn't just about making predictions, it's also about measuring how strong and meaningful the relationships are between different things and understanding what really drives outcomes. Throughout this section, you'll discover practical ways to build, apply, and improve regression models, making them powerful tools for solving real-world problems with data, whether you're curious about exam results, sales forecasts, or the price of your dream home.

8.1 EXPECTED LEARNING OUTCOMES

By the end of this unit, you will be able to:

- Understand the fundamental concepts of regression analysis and its role in predictive data analysis
- Distinguish between different types of regression models and their appropriate applications
- Evaluate regression model performance using appropriate metrics and diagnostic tools
- Select appropriate regression techniques based on data characteristics and problem requirements
- Interpret regression results in the context of business and scientific applications

8.2 INTRODUCTION TO REGRESSION

In Unit 7 of this course, we had a brief introduction of Regression and its types, now in this unit we are going to discuss it in detail. Actually, Regression is a statistical method that examines the relationship between a dependent variable (also called response variable, target variable, or outcome variable) and one or more independent variables (also called predictor variables, explanatory variables, or features). The primary goal of regression analysis is to model this relationship in a way that allows us to predict the value of the dependent variable based on known values of the independent variables.

The concept of regression was first introduced by Sir Francis Galton in the 1880s while studying the relationship between the heights of parents and their children. Galton observed that extremely tall parents tend to have children who are tall but closer to the average height - a phenomenon he called "regression toward the mean". This foundational observation led to the development of what we now know as regression analysis.

The mathematical framework for regression was further developed by Karl Pearson and later refined by Ronald Fisher, who introduced the method of maximum likelihood estimation. The evolution continued with the advent of computer technology, enabling the analysis of complex datasets and the development of sophisticated regression techniques.

We learned that Regression is one of the most important techniques in data analysis and predictive modelling because it helps us understand how one variable is influenced by one or more other variables. In simple terms, regression is used when we want to predict or explain the value of an outcome on the basis of certain input factors. It is widely used in research, business, economics, healthcare, engineering, and many other fields because it not only helps in prediction but also explains the nature of relationships among variables

Core Concepts in Regression involves Dependent and Independent Variables, let's understand them.

- The **Dependent variable (Y)** is the variable that we want to predict, estimate, or explain. It is called dependent because its value is assumed to depend on other variables included in the model.
- On the other hand, the **independent variables (X)** are the predictor or explanatory variables i.e. The variables used to predict the dependent variable. These are the input features that influence the outcome

These are the input features that influence or help explain changes in the dependent variable. For example, if we want to predict house prices, then the house price becomes the dependent variable, while factors such as area, number of rooms, and location become the independent variables.

The connection between the Dependent and Independent variable is established by the Regression Equation, the general form of a regression equation can be expressed as:

$$Y = f(X_1, X_2, \dots, X_n) + \epsilon$$

Where:

- Y is the dependent variable
- $X_1, X_2, \dots, X_n$ are independent variables
- $f()$ is the function that describes the relationship
- ϵ (epsilon) is the error term representing unexplained variation

Based on the analysis of the Regression equation, the Regression analysis can capture different kinds of relationships between dependent and independent variables, they are as follows:

- In a **linear relationship**, the association between variables can be represented by a straight line, meaning that a change in the predictor leads to a proportional change in the outcome.
- In a **non-linear relationship**, the association follows a curved or more complex pattern, which means the effect of a predictor may change at different levels. Regression can also identify the **direction** of relationships.
- In a **positive relationship**, as one variable increases, the other also tends to increase.
- In a **negative relationship**, as one variable increases, the other tends to decrease.

Understanding the type of relationship is important because it guides the choice of the correct regression model.

This tool of Regression analysis has applications across different domains be it finance, or healthcare or marketing, engineering etc. A few of them are listed below:

- In **finance**, it is used to predict stock prices, assess credit risk, and support portfolio optimization.
- In **healthcare**, regression helps model disease progression, predict treatment outcomes, and analyze factors affecting patient recovery.
- In **marketing**, it is commonly applied to forecast sales on the basis of advertising expenditure, segment customers, and optimize pricing strategies.
- In **engineering**, regression is useful for quality control, performance improvement, and reliability analysis.
- In **economics**, it supports the analysis of factors affecting GDP, inflation, and consumer demand. These applications show that regression is not only a statistical tool but also a practical method for solving real-world problems.

The predictive analysis is one of the key analyses practiced in almost every field of work, and regression plays a vital role in performing predictive analysis. So, let's discuss the role of Regression in Predictive Analytics. Within predictive analytics, regression plays a foundational role due to following reasons:

- It transforms historical data into meaningful predictions about future outcomes. It helps organizations make **data-driven predictions** by using past observations to estimate what may happen next.
- It helps in **identifying key drivers**, meaning it reveals which variables have the strongest influence on the target variable.
- Also, it **quantifies relationships**, as it measures both the strength and the direction of influence between variables. This makes it extremely useful for **decision making**, since managers and researchers can rely on quantitative evidence rather than intuition alone.
- It also contributes to **risk assessment** by helping evaluate possible outcomes and their likelihood, which is valuable in uncertain environments.

In this course we learned that there are a variety of analytical techniques be it Descriptive, or Diagnostic or Prescriptive or Predictive analysis where Regression is one of the most important techniques for data analysis. So, we need to compare it with the other analytical techniques i.e. Regression should also be clearly distinguished from other analytical methods like classification, correlation, time series analysis etc.; the key observations are as follows:

- In **regression vs. classification**, the key difference is that regression predicts **continuous numerical values**, such as income, sales, or temperature, whereas

classification predicts **discrete categories**, such as yes/no, pass/fail, or disease/no disease.

- **In regression vs. correlation**, correlation only measures the strength and direction of association between variables, while regression goes a step further by modelling the predictive relationship and estimating how one variable changes with another.
- **In regression vs. time series analysis**, both methods may be used for prediction, but time series analysis is specifically focused on data observed over time and gives special importance to temporal patterns, trends, and seasonality.

Although regression is a powerful method, it also comes with certain **challenges and limitations**, they are discussed below:

- One common issue is assumption violations, because regression models rely on assumptions such as linearity, independence, homoscedasticity, and normality, which may not always hold in real data.
- Another challenge is overfitting, where a model performs very well on training data but fails to generalize to new or unseen data.
- Multicollinearity is another important problem, occurring when predictor variables are highly correlated with each other, which can make model estimates unstable.
- Outliers can also create difficulties, as extreme values may disproportionately influence the regression line and distort the results.
- Finally, missing data is a common practical issue, since incomplete datasets need careful handling to avoid biased or invalid conclusions. Therefore, while regression is highly useful, it must be applied thoughtfully and with proper diagnostic checks.

☛ Check Your Progress 1

Q1. What is Regression Analysis? Explain its purpose in predictive data analysis.

.....

Q2. Explain the difference between dependent and independent variables with examples.

.....

Q3. Write the general form of a regression equation and explain its components.

.....

Q4. Discuss the advantages and limitations of regression analysis.

.....

Q5. What is the role of regression in predictive analytics?

.....

8.3 REGRESSION MODELS

Regression models form one of the most important components of predictive data analysis because they help in understanding, explaining, and predicting the relationship between a dependent variable and one or more independent variables. In predictive analytics, the

purpose of regression is not limited to estimating future values; it is also used to identify influential factors, measure the strength of relationships, and support evidence-based decision making. Different regression models are used depending on the nature of the data, the type of dependent variable, the relationship among variables, and the assumptions that can reasonably be satisfied. Therefore, a proper understanding of regression models is essential for selecting the right analytical method in real-world applications.

We learned that a regression model is a mathematical representation that expresses how the dependent variable changes with changes in the independent variables. It provides a systematic way to estimate outcomes and interpret how predictor variables contribute to those outcomes. In predictive data analysis, regression models are especially useful because many practical problems involve forecasting a numerical quantity, estimating probability, or modeling patterns in data. For example, an organization may want to predict sales revenue, a bank may want to estimate credit risk, a hospital may want to analyze the impact of patient characteristics on recovery time, or an economist may want to study how inflation affects consumer demand. In all such cases, regression models provide the analytical structure required to move from raw data to meaningful prediction and interpretation.

Regression models are important for several reasons. They help transform historical data into future predictions, reveal hidden relationships among variables, and quantify the magnitude and direction of influence. They also support decision-making by allowing analysts to test hypotheses and compare the effect of different factors. Because of this dual role of explanation and prediction, regression models are widely used in business analytics, economics, healthcare, engineering, finance, social sciences, and machine learning.

The general form of a regression model can be written as:

$$Y = f(X_1, X_2, X_3, \dots, X_n) + \epsilon$$

where Y is the dependent variable, $X_1, X_2, X_3, \dots, X_n$ are the independent variables, $f(\cdot)$ represents the functional relationship between predictors and outcome, and ϵ is the error term. The error term captures the part of variation in the dependent variable that is not explained by the predictors included in the model. In some models, the function f is linear, while in others it may be nonlinear or transformed to fit complex relationships. The choice of the model depends on the underlying structure of the problem being studied.

The most commonly used regression techniques in predictive data analysis include simple linear regression, multiple linear regression, polynomial regression, logistic regression, ridge regression, lasso regression, and elastic net regression. Each of these models has a different purpose and is suitable for different types of problems.

8.3.1 SIMPLE LINEAR REGRESSION

Simple linear regression is the most basic form of regression analysis. It is used when there is one dependent variable and one independent variable, and the relationship between them is assumed to be linear. The equation of simple linear regression is:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

where Y is the dependent variable, X is the independent variable, β_0 is the intercept, β_1 is the slope coefficient, and ϵ is the error term. The intercept represents the value of Y when $X = 0$, while the slope indicates how much Y changes for a one-unit change in X .

So far as the mathematical foundation of Simple Linear Regression is concerned, the relationship between the dependent variable and the independent variable is expressed through the equation:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

In this equation, β_0 is the intercept, β_1 is the slope of the regression line, and ϵ is the error term. The values of β_0 and β_1 are estimated using the Ordinary Least Squares (OLS) method. The main objective of OLS is to find the line that best fits the observed data by minimizing the sum of squared residuals, where a residual is the difference between the actual value and the predicted value.

Mathematically, this is written as:

$$\text{Minimize } \sum (Y_i - \hat{Y}_i)^2$$

Here, Y_i represents the actual observed value, while $\hat{Y}_i$ represents the predicted value obtained from the regression line.

By applying this minimization principle, the estimates of the slope and intercept are obtained as follows:

The formulas used are:

$$\beta_1 = \frac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{\sum (X_i - \bar{X})^2}$$

$$\beta_0 = \bar{Y} - \beta_1 \bar{X}$$

where $\bar{X}$ is the mean of the independent variable values and $\bar{Y}$ is the mean of the dependent variable values. In simple terms, β_1 shows how much the dependent variable changes for a one-unit change in the independent variable, while β_0 gives the expected value of Y when $X = 0$. These formulas help determine the best-fitting straight line for the given dataset.

This model is useful when the relationship between two variables can be represented by a straight line. For example, a researcher may study the relationship between hours of study and exam marks. If more hours of study generally lead to higher marks, then simple linear regression can estimate that relationship and predict marks for a given study time. Its main strength lies in its simplicity and interpretability, but it becomes insufficient when more than one predictor influences the outcome.

Let's understand the above learned concepts with the help of an Example

Example 1: Using Regression analysis, a teacher wants to study the relationship between **hours studied** and **marks obtained** by students. Here Hours Studied is the **independent variable (X)** and Marks Obtained is the **Dependent variable (Y)**. The data is:

Student	Hours Studied (X)	Marks Obtained (Y)
1	1	52
2	2	55
3	3	61
4	4	66
5	5	70

Solution:

Step 1: Find the Mean of X and Y

Mean of X: $\bar{X} = \frac{1+2+3+4+5}{5} = \frac{15}{5} = 3$ and **Mean of Y:** $\bar{Y} = \frac{52+55+61+66+70}{5} = \frac{304}{5} = 60.8$

So, $\bar{X} = 3, \bar{Y} = 60.8$

Step 2: Calculate $(X_i - \bar{X})$, $(Y_i - \bar{Y})$, $(X_i - \bar{X})(Y_i - \bar{Y})$, and $(X_i - \bar{X})^2$.

X	Y	$X_i - \bar{X}$	$Y_i - \bar{Y}$	$(X_i - \bar{X})(Y_i - \bar{Y})$	$(X_i - \bar{X})^2$
1	52	-2	-8.8	17.6	4
2	55	-1	-5.8	5.8	1
3	61	0	0.2	0.0	0
4	66	1	5.2	5.2	1
5	70	2	9.2	18.4	4

Now find the totals:

$$\sum(X_i - \bar{X})(Y_i - \bar{Y}) = 17.6 + 5.8 + 0 + 5.2 + 18.4 = 47$$

$$\sum(X_i - \bar{X})^2 = 4 + 1 + 0 + 1 + 4 = 10$$

Step 3: Calculate the Slope β_1 : Using the formula: $\beta_1 = \frac{\sum(X_i - \bar{X})(Y_i - \bar{Y})}{\sum(X_i - \bar{X})^2}$

Substituting the values we get $\beta_1 = \frac{47}{10} = 4.7$ i.e. the slope is: $\beta_1 = 4.7$

Step 4: Calculate the Intercept β_0 : Using the formula: $\beta_0 = \bar{Y} - \beta_1 \bar{X}$

Substituting the values we get $\beta_0 = 60.8 - (4.7 \times 3) = 60.8 - 14.1 = 46.7$

So, the intercept is: $\beta_0 = 46.7$

Step 5: Write the Regression Equation

Now substitute β_0 and β_1 into the regression equation

$Y = \beta_0 + \beta_1 X$, we get $Y = 46.7 + 4.7X$

This is the **simple linear regression equation** for the given data.

Step 6: Compute Predicted Values Using the equation: $\hat{Y} = 46.7 + 4.7X$

Let us calculate the predicted marks for each value of X .

<u>X</u>	<u>Actual Y</u>	<u>Predicted $\hat{Y} = 46.7 + 4.7X$</u>	<u>Residual = $Y - \hat{Y}$</u>
1	52	51.4	0.6
2	55	56.1	-1.1
3	61	60.8	0.2
4	66	65.5	0.5
5	70	70.2	-0.2

Step 7: Check the Residuals and Ordinary Least Square (OLS) method

From the above table we identified the Residual = $Y_i - \hat{Y}_i$

These residuals show the difference between actual marks and predicted marks.

Ordinary Least Square (OLS) method chooses the regression line in such a way that the **sum of squared residuals** becomes minimum. To perform this, tabulate the square of residuals and determine their sum.

X	Residual $Y - \hat{Y}$	Residual Squared $(Y - \hat{Y})^2$
1	0.6	0.36
2	-1.1	1.21

X	Residual $Y - \hat{Y}$	Residual Squared $(Y - \hat{Y})^2$
3	0.2	0.04
4	0.5	0.25
5	-0.2	0.04

$$\sum(Y_i - \hat{Y}_i)^2 = 0.36 + 1.21 + 0.04 + 0.25 + 0.04 = 1.90$$

Since the residuals are small and the sum of squared residuals is low, that is, (1.90), which indicates that the predicted values are close to the observed values, suggesting that the fitted regression line provides a good representation of the data.

This small value indicates that the fitted regression line is close to the actual data points. Because This quantity i.e. $\sum(Y_i - \hat{Y}_i)^2 = 1.90$ measures the total prediction error of the regression line. Each residual is the gap between the actual value and the predicted value. When we square and add these gaps:

- small total $\rightarrow$ actual and predicted values are close
- large total $\rightarrow$ actual and predicted values are far apart

In your example, the residuals were: 0.6, -1.1, 0.2, 0.5, -0.2; and these are all quite small in magnitude, and their squared sum is only 1.90, which suggests the regression line does not deviate much from the observed points.

We observed that the prediction errors are all around less than 1.1 marks, which is very small compared with the scale of marks ranging from 52 to 70. That is why we say the line is close to the actual data points.

Step 8: Interpretation of the Regression Coefficients

Interpretation of Slope $\beta_1 = 4.7$: The slope value of 4.7 means that for every additional 1 hour of study, the marks are expected to increase by 4.7 marks, on average.

So, there is a **positive linear relationship** between study hours and marks.

Interpretation of Intercept $\beta_0 = 46.7$: The intercept value of 46.7 means that if a student studies for 0 hours, the predicted marks would be 46.7.

In practical situations, the intercept may or may not always have direct real-world meaning, but mathematically it helps define the regression line.

Step 9: Prediction Using the Model: Suppose we want to predict the marks of a student who studies for 6 hours. Using the regression equation:

$$\begin{aligned}\hat{Y} &= 46.7 + 4.7(6) \\ \hat{Y} &= 46.7 + 28.2 = 74.9\end{aligned}$$

So, if a student studies for **6 hours**, the predicted marks are: $\hat{Y} = 74.9$.

Thus, the model predicts that the student may score approximately **75 marks**.

Overall Interpretation of Results: The regression equation obtained is $Y = 46.7 + 4.7X$. This equation shows that there is a **clear positive relationship** between hours studied and marks obtained. As the number of study hours increases, the marks also increase. The slope indicates that the increase is about **4.7 marks per extra hour studied**. The actual and predicted values are quite close, and the residuals are small, which suggests that the regression line fits the data well.

From the predictive data analysis point of view, this example shows how simple linear regression can be used both to **understand the relationship** between two variables and to **make future predictions**. It is especially useful when the relationship is approximately linear and only one predictor variable is involved.

This numerical example clearly demonstrates the working of **simple linear regression**. First, the means of the variables were calculated. Then, the slope and intercept were estimated using the **OLS formulas**. After that, the regression equation was formed, predicted values were computed, and the results were interpreted.

Now we extend the understanding of the linear Regression by calculating few more important parameters like Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Coefficient of Determination (R^2). By Using the same Example 1, given above, where:

- Actual Y: 52, 55, 61, 66, 70
- Predicted $\hat{Y}$: 51.4, 56.1, 60.8, 65.5, 70.2
- Residuals ($Y - \hat{Y}$): 0.6, -1.1, 0.2, 0.5, -0.2
- Sum of Squared Errors (SSE) = $\sum (Y - \hat{Y})^2 = 1.90$

Calculating Mean Squared Error (MSE): The formula for MSE is $MSE = \frac{\sum (Y_i - \hat{Y}_i)^2}{n}$

Here, $n = 5$. So, $MSE = \frac{1.90}{5} = 0.38$

The interpretation of the resulting **MSE = 0.38** means that the average squared prediction error of the regression model is **0.38**. Since this value is quite low, it indicates that the model's predictions are very close to the actual values. However, because the errors are squared, MSE is expressed in squared units, so it is less directly interpretable than RMSE.

Calculating Root Mean Squared Error (RMSE): The formula for RMSE is

$$RMSE = \sqrt{MSE}$$

$$RMSE = \sqrt{0.38} \approx 0.616$$

The interpretation of the resulting **RMSE ≈ 0.616** means that, on average, the predicted marks differ from the actual marks by about **0.62 marks**. Since the marks in the dataset range roughly from **52 to 70**, an average prediction error of less than 1 mark is very small. This shows that the regression model fits the data very well.

Calculating Coefficient of Determination (R^2): The formula for R^2 is

$$R^2 = 1 - \frac{SSE}{SST}$$

where $SSE = \sum (Y_i - \hat{Y}_i)^2$ and $SST = \sum (Y_i - \bar{Y})^2$,

We already know $\bar{Y} = 60.8$;

Now calculate SST:

Y_i	$Y_i - \bar{Y}$	$(Y_i - \bar{Y})^2$
52	-8.8	77.44
55	-5.8	33.64
61	0.2	0.04
66	5.2	27.04
70	9.2	84.64

Now, $SST = 77.44 + 33.64 + 0.04 + 27.04 + 84.64 = 222.80$

Already we calculated the Sum of Squared Errors ($SSE = \sum (Y - \hat{Y})^2 = 1.90$)

Now apply the formula: $R^2 = 1 - \frac{SSE}{SST}$

$$R^2 = 1 - \frac{1.90}{222.80}$$

$$R^2 = 1 - 0.00853 \approx 0.9915$$

$$R^2 \approx 0.9915 \text{ i.e. about } \mathbf{99.15\%}$$

The value of $R^2 \approx 0.992$ means that about **99.15% of the variation in marks** is explained by the number of hours studied. This indicates an **excellent fit** of the regression model. In other words, study hours explain almost all the variation in students' marks in this small dataset.

Final Results Summary for the given numerical problem is as follows:

Metric	Value	Interpretation
SSE	1.90	Total squared prediction error is very low
MSE	0.38	Average squared error is low
RMSE	0.616	Average prediction error is about 0.62 marks
R^2	0.9915	About 99.15% of variation in marks is explained by study hours

Overall Interpretation: These values together show that the simple linear regression model provides a very strong fit for the given data.

- The low MSE shows that prediction errors are very small.

- The low RMSE shows that the model's predictions differ from actual marks by less than 1 mark on average.
- The very high R^2 shows that the independent variable, hours studied, explains almost all the variation in the dependent variable, marks obtained.

So, for this dataset, the regression line is highly effective for both understanding the relationship and making predictions.

With the above numerical example, we learned that the Simple linear regression is easy to understand, mathematically elegant, and highly useful in practical situations such as predicting marks from study hours, sales from advertising expense, crop yield from fertilizer use, or income from years of education. It serves as the foundation for more advanced regression techniques used in predictive analytics.

The Linear regression follows some assumptions and they are as follows:

1. Linearity: The relationship between independent and dependent variables is linear
2. Independence: Observations are independent of each other
3. Homoscedasticity: Constant variance of residuals across all levels of independent variables
4. Normality: Residuals are normally distributed
5. No Multicollinearity: Independent variables are not highly correlated (for multiple regression)

Apart from this the Simple Linear Regression has some advantages and Limitations both, they are as follows:

Advantages: Linear regression offers several important advantages, they are:

- It is simple to understand and easy to implement, which makes it one of the most widely used regression techniques.
- It requires relatively low computational effort, so results can be obtained quickly even for moderately sized datasets.
- Another major advantage is that it provides interpretable coefficients, allowing analysts to clearly understand how each independent variable influences the dependent variable.
- It also serves as a strong baseline model for comparing the performance of more advanced predictive techniques.

When the underlying assumptions are reasonably satisfied, linear regression can produce reliable and effective results.

Limitations: Despite its usefulness, linear regression has certain limitations, they are:

- It is based on the assumption that the relationship between variables is linear, which may not always reflect real-world situations.
- The model is also sensitive to outliers, meaning that extreme values can significantly affect the regression line and distort the results.

- In addition, its performance depends on the extent to which key assumptions, such as linearity, homoscedasticity, independence, and normality of errors, are satisfied.
- Another limitation is that linear regression is not well suited for capturing complex or highly non-linear relationships unless additional transformations or advanced extensions are applied.

It is to be noted that the Linear regression is widely applied across many practical domains, a few are listed below:

- In business, it is used for sales forecasting based on advertising expenditure and for revenue prediction based on customer volume.
- In economics, it helps in analysing the relationship between education and income as well as in studying inflation-related trends.
- In the field of sports, linear regression can be used to predict player performance using training and fitness metrics.
- In agriculture, it supports crop yield prediction based on factors such as fertilizer usage and weather conditions.

These applications demonstrate the broad relevance of linear regression as a foundational tool in predictive data analysis.

📌 Check Your Progress 2

Q1. What is a Regression Model? Explain its significance in predictive analytics.

.....

Q2. Define Simple Linear Regression and write its mathematical equation.

.....

Q3. Explain the Ordinary Least Squares (OLS) method used in linear regression.

.....

Q4. Interpret the slope and intercept in a linear regression model..

.....

Q5. List the assumptions and limitations of Simple Linear Regression

.....

.....

8.3.2 MULTIPLE LINEAR REGRESSION

Multiple regression is an extension of simple linear regression in which the dependent variable is explained or predicted using more than one independent variable. This makes it a highly useful technique in predictive data analysis because many real-world outcomes are influenced by several factors simultaneously. Unlike simple linear regression, which studies the effect of only one predictor, multiple regression allows the analyst to model more complex relationships in a more realistic manner. It is especially valuable when the objective is not only to predict the dependent variable accurately but also to understand the separate contribution of each explanatory variable.

The general form of the multiple regression model is expressed as:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \epsilon$$

In this equation, Y represents the dependent variable, while $X_1, X_2, \dots, X_p$ represent the independent variables. The terms $\beta_0, \beta_1, \beta_2, \dots, \beta_p$ are the regression coefficients, and ϵ is the error term. The intercept β_0 gives the expected value of Y when all independent variables are equal to zero. Each slope coefficient shows the effect of its corresponding independent variable on the dependent variable, subject to the presence of the other predictors in the model.

For computational convenience and mathematical clarity, multiple regression is often expressed in matrix form as:

$$Y = X\beta + \epsilon$$

Here, Y is an $n \times 1$ vector of dependent variable values, X is an $n \times (p + 1)$ matrix of independent variables including a column of ones for the intercept, β is a $(p + 1) \times 1$ vector of regression coefficients, and ϵ is an $n \times 1$ vector of error terms. Using the **Ordinary Least Squares method**, the coefficient estimates are obtained from the expression:

$$\beta = (X'X)^{-1}X'Y$$

This matrix formulation provides an efficient way of estimating the regression coefficients, particularly when the model involves many predictors.

Interpretation of Coefficients: In multiple regression, the interpretation of coefficients is based on the principle of **ceteris paribus**, which means “all other things remaining constant.” This means that each coefficient represents the expected change in the dependent variable associated with a one-unit change in the corresponding independent variable, while holding all other independent variables constant. This interpretation is extremely important because, in a multivariable setting, the effect of one predictor is examined after accounting for the influence of the others.

For example, consider the model for predicting house prices:

$$\text{Price} = 20000 + 80 \times \text{Square_Feet} + 15000 \times \text{Bedrooms} + 5000 \times \text{Bathrooms} - 500 \times \text{Age}$$

This equation implies that, holding bedrooms, bathrooms, and age constant, every additional square foot increases the house price by ₹80. Similarly, each additional bedroom increases the price by ₹15,000, each additional bathroom adds ₹5,000, and each additional year of age reduces the price by ₹500. Thus, multiple regression not only predicts house prices but also allows the separate effect of each factor to be clearly understood.

Additional Considerations for Multiple Regression: Although multiple regression is powerful, its application requires careful attention to certain issues, the most important of which is **multicollinearity**. **Multicollinearity occurs when two or more independent variables**

are highly correlated with each other. This can create several problems in the model, such as unstable coefficient estimates, inflated standard errors, and difficulty in interpreting the individual contribution of variables. In other words, when predictors overlap substantially in the information they provide, it becomes harder to determine which variable is actually responsible for changes in the dependent variable.

One common method for detecting multicollinearity is the **Variance Inflation Factor (VIF)**. A VIF value greater than 10 is often considered an indication of problematic multicollinearity, although lower thresholds are also sometimes used in practice. When multicollinearity is detected, several remedies are available. Highly correlated variables may be removed from the model, or they may be combined using techniques such as principal component analysis. Another useful solution is the application of regularization methods such as Ridge regression or Lasso regression, which help stabilize coefficient estimates.

Example 2: To understand **multiple regression** in a practical way, let us consider a simple example in which a house price is predicted using two independent variables: **house size** and **number of bedrooms**. Here, the objective is to estimate the effect of both predictors on the dependent variable and also examine whether multicollinearity exists between the independent variables.

Let:

- Y = House Price (in lakh rupees)
- X_1 = House Size (in hundred square feet)
- X_2 = Number of Bedrooms

The observed data are:

Observation	X_1	X_2	Y
1	1	2	16
2	2	3	21
3	3	5	28
4	4	4	31
5	5	6	38

The general form of the multiple regression model is: $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \epsilon$

For this example, the fitted model will take the form:

$$Y = \beta_0 + \beta_1(\text{House Size}) + \beta_2(\text{Bedrooms}) + \epsilon$$

where:

- β_0 = intercept
- β_1 = coefficient of house size
- β_2 = coefficient of bedrooms
- ϵ = error term

Step 1: Matrix Representation of the Model: In matrix form, multiple regression is written as: $Y = X\beta + \epsilon$

$$\text{where, } Y = \begin{bmatrix} 16 \\ 21 \\ 28 \\ 31 \\ 38 \end{bmatrix}, X = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 3 \\ 1 & 3 & 5 \\ 1 & 4 & 4 \\ 1 & 5 & 6 \end{bmatrix}, \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}$$

The first column of ones in X is included for the intercept term. The Ordinary Least Squares solution is: $\hat{\beta} = (X'X)^{-1}X'Y$

Step 2: Compute $X'X$ and $X'Y$

First, compute the transpose product $X'X = \begin{bmatrix} 5 & 15 & 20 \\ 15 & 55 & 69 \\ 20 & 69 & 90 \end{bmatrix}$

Next, compute $X'Y = \begin{bmatrix} 134 \\ 456 \\ 587 \end{bmatrix}$

Step 3: Compute $(X'X)^{-1}$:

The inverse of $X'X$ is: $(X'X)^{-1} = \begin{bmatrix} 1.9895 & 0.3158 & -0.6842 \\ 0.3158 & 0.5263 & -0.4737 \\ -0.6842 & -0.4737 & 0.5263 \end{bmatrix}$

Step 4: Estimate the Coefficients: Now apply the formula: $\hat{\beta} = (X'X)^{-1}X'Y$

$$\hat{\beta} = \begin{bmatrix} 1.9895 & 0.3158 & -0.6842 \\ 0.3158 & 0.5263 & -0.4737 \\ -0.6842 & -0.4737 & 0.5263 \end{bmatrix} \begin{bmatrix} 134 \\ 456 \\ 587 \end{bmatrix} = \begin{bmatrix} 8.9579 \\ 4.2632 \\ 1.2632 \end{bmatrix}$$

Thus, the estimated coefficients are: $\beta_0 = 8.9579, \beta_1 = 4.2632, \beta_2 = 1.2632$

Step 5: Write the Estimated Regression Equation:

The fitted multiple regression equation is: $\hat{Y} = 8.9579 + 4.2632X_1 + 1.2632X_2$

This is the required regression model.

Step 6: Compute the Predicted Values: Using the estimated equation:

$$\hat{Y} = 8.9579 + 4.2632X_1 + 1.2632X_2$$

the predicted values are:

Obs.	X_1	X_2	Actual Y	Predicted $\hat{Y}$
1	1	2	16	$8.9579 + 4.2632(1) + 1.2632(2) = 15.75$

Obs.	X_1	X_2	Actual Y	Predicted $\hat{Y}$
2	2	3	21	$8.9579 + 4.2632(2) + 1.2632(3) = 21.26$
3	3	5	28	$8.9579 + 4.2632(3) + 1.2632(5) = 28.05$
4	4	4	31	$8.9579 + 4.2632(4) + 1.2632(4) = 31.06$
5	5	6	38	$8.9579 + 4.2632(5) + 1.2632(6) = 37.85$

So, the predicted values are approximately: 15.75, 21.26, 28.05, 31.06, 37.85

Step 7: Interpretation of the Coefficients:

The estimated model is: $\hat{Y} = 8.9579 + 4.2632X_1 + 1.2632X_2$

The interpretation of the coefficients is as follows:

- **Intercept ($\beta_0 = 8.9579$):** This is the predicted house price when both house size and number of bedrooms are zero. In practice, this may not always have direct physical meaning, but mathematically it is necessary for defining the regression plane.
- **Coefficient of X_1 ($\beta_1 = 4.2632$):** Holding the number of bedrooms constant, a one-unit increase in house size increases the expected house price by **4.2632 lakh rupees**.
- **Coefficient of X_2 ($\beta_2 = 1.2632$):** Holding house size constant, an additional bedroom increases the expected house price by **1.2632 lakh rupees**.

This is the **ceteris paribus** interpretation of multiple regression.

Step 8: Detecting Multicollinearity: One of the major concerns in multiple regression is **multicollinearity**, which arises when the independent variables are highly correlated with each other.

(a) Correlation Between X_1 and X_2 : Using the given data, the correlation between house size and bedrooms is: $r_{X_1X_2} = 0.90$. This indicates a **strong positive correlation** between the two predictors.

(b) Coefficient of Determination for Auxiliary Regression: Since there are only two independent variables, we can regress X_1 on X_2 (or vice versa). In the two-variable case:

$$R_1^2 = r^2 = (0.90)^2 = 0.81$$

Thus, about **81% of the variation in X_1 is explained by X_2** , which suggests considerable overlap between the predictors.

(c) **Variance Inflation Factor (VIF):** The formula for VIF is: $VIF_j = \frac{1}{1-R_j^2}$

So, $VIF_1 = \frac{1}{1-0.81} = \frac{1}{0.19} = 5.26$; Similarly, $VIF_2 = 5.26$

(d) **Tolerance:** Tolerance is the reciprocal of VIF:

$$\text{Tolerance} = \frac{1}{VIF} = \frac{1}{5.26} \approx 0.19$$

Interpretation of Multicollinearity Results: The correlation value of **0.90** shows that the two independent variables are strongly related. The VIF value of **5.26** is above 5, which suggests **moderate to fairly high multicollinearity**, although it is still below the more severe cutoff of 10 that is often used in practice. The tolerance value of **0.19** is also relatively low, further confirming that the predictors share substantial common information.

This means that although the regression model can still be estimated, the coefficient estimates may become somewhat unstable if the dataset changes. In practical analysis, such a situation may require closer examination, variable selection, or regularization methods such as Ridge regression.

The fitted model shows that both house size and number of bedrooms positively affect house price. House size has the stronger impact, since its coefficient is much larger than that of bedrooms. This suggests that, in this example, increasing the size of the house contributes more to price than merely increasing the number of bedrooms.

At the same time, the multicollinearity analysis shows that house size and bedrooms are themselves strongly related. This is expected in real-world housing data because larger houses often tend to have more bedrooms. Therefore, while the model is useful for prediction, interpretation of the individual coefficients should be done carefully.

This numerical example demonstrates how **multiple regression** is applied using both its **equation form** and **matrix representation**. The coefficients were estimated through the OLS formula:

$$\hat{\beta} = (X'X)^{-1}X'Y$$

leading to the regression equation:

$$\hat{Y} = 8.9579 + 4.2632X_1 + 1.2632X_2$$

The example also showed how to compute and interpret **multicollinearity** using correlation, R^2 , VIF, and tolerance. The results indicate that while the model is meaningful and interpretable, the predictors exhibit moderate multicollinearity, which is an important consideration in multiple regression analysis.

Next, we are going to discuss the Model Selection Techniques: In multiple regression, another important issue is the selection of the most appropriate set of independent variables. Including too many predictors may lead to overfitting, while including too few may omit important explanatory factors. Therefore, model selection techniques are used to identify a

suitable subset of variables. Therefore, model selection helps in building a regression model that is both **efficient** and **interpretable**.

The three common model selection techniques are **forward selection**, **backward elimination**, and **stepwise selection**.

- **Forward selection** begins with no independent variables in the model and then adds variables one by one based on their contribution to improving model performance.
- **Backward elimination** starts with all candidate variables included and removes those that do not contribute significantly to the model.
- **Stepwise selection** combines both forward and backward procedures, adding and removing variables iteratively in order to arrive at a balanced and efficient model.

These techniques are important because they help in developing a regression model that is both interpretable and predictive.

These can be understood clearly with a small numerical example.

Example 3: Suppose a company wants to predict **monthly sales (Y)** using three possible predictors:

- X_1 : Advertising spend
- X_2 : Number of sales calls
- X_3 : Average temperature

The observed data are:

Observation	Advertising X_1	Sales Calls X_2	Temperature X_3	Sales Y
1	2	5	30	40
2	4	6	32	50
3	6	7	31	65
4	8	8	29	78
5	10	9	30	90

Now we need to prepare a table to tabulate the **Candidate Models and Their Results**. That table is usually **prepared after fitting several regression models separately** and then comparing their performance measures. So, Let's understand **How the “Candidate Models and Their Results” table is prepared**

Solution: Suppose we have:

- Y = Sales
- X_1 = Advertising
- X_2 = Sales Calls
- X_3 = Temperature

To prepare the table, we do **not** start with one final model directly. Instead, we create **different possible regression models** using different combinations of predictors, and then compute measures like R^2 , **Adjusted R^2** , and **p-values**, sometimes **AIC**, **BIC**, **MSE**, **RMSE**, etc. also.

Step 1: List all possible candidate models: With 3 predictors (X_1, X_2, X_3), the possible models are:

Model	Predictors Included
M0	None (only intercept)
M1	X_1
M2	X_2
M3	X_3
M4	X_1, X_2
M5	X_1, X_3
M6	X_2, X_3
M7	X_1, X_2, X_3

This is exactly why the candidate model table had 8 rows.

Step 2: Fit each model separately: Now each model is estimated using regression.

For example:

Model M1: $Y = \beta_0 + \beta_1 X_1$

Model M2: $Y = \beta_0 + \beta_2 X_2$

Model M4: $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2$

Model M7: $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3$

Each time, software or manual regression calculations are used to estimate the coefficients.

Step 3: Compute model evaluation measures: After fitting each model, we compute:

1. **R^2 :** It tells us how much variation in Y is explained by that model.

$$R^2 = 1 - \frac{SSE}{SST}$$

where:

- SSE = Sum of Squared Errors
- SST = Total Sum of Squares

2. **Adjusted R^2 :** This adjusts R^2 for the number of predictors, because plain R^2 usually increases when more variables are added.

$$\text{Adjusted } R^2 = 1 - \left(\frac{(1 - R^2)(n - 1)}{n - k - 1} \right)$$

where:

- n = number of observations
- k = number of predictors

This is why Adjusted R^2 is often more useful for model selection.

3. **p-values:** These are computed for each coefficient to check whether a predictor significantly contributes to the model.

Step 4: Put the results into one comparison table

After estimating each candidate model, the results are summarized into one table like this:

Model	Predictors Included	R^2	Adjusted R^2	Remark
-------	---------------------	-------	----------------	--------

Model	Predictors Included	R^2	Adjusted R^2	Remark
M0	None (intercept only)	0.000	0.000	Baseline model
M1	X_1	0.978	0.971	Very strong predictor
M2	X_2	0.930	0.907	Good predictor
M3	X_3	0.050	-0.267	Very weak predictor
M4	X_1, X_2	0.985	0.970	Slight improvement
M5	X_1, X_3	0.979	0.958	No real gain from X_3
M6	X_2, X_3	0.932	0.864	X_3 adds little
M7	X_1, X_2, X_3	0.986	0.944	Model becomes more complex

So that table is basically a **summary table of all separately fitted models**.

Let's Understand How is one row actually prepared? :Take Model M1 as an example.

Suppose the data are:

Obs	X_1	Y
1	2	40
2	4	50
3	6	65
4	8	78
5	10	90

We fit: $Y = \beta_0 + \beta_1 X_1$;

After calculation, suppose we get: $\hat{Y} = 27.6 + 6.3X_1$

Then we calculate:

- predicted values $\hat{Y}$
- residuals $(Y - \hat{Y})$
- $SSE = \sum (Y - \hat{Y})^2$
- $SST = \sum (Y - \bar{Y})^2$
- $R^2 = 1 - \frac{SSE}{SST}$
- Adjusted R^2

Then the row for M1 is filled.

So, the row is not guessed. It is based on actual regression calculations.

Now suppose, after fitting regression models in software, the following results are obtained.

Model	Predictors Included	R^2	Adjusted R^2	Remark
M0	None (intercept only)	0.000	0.000	Baseline model
M1	X_1	0.978	0.971	Very strong predictor
M2	X_2	0.930	0.907	Good predictor
M3	X_3	0.050	-0.267	Very weak predictor
M4	X_1, X_2	0.985	0.970	Slight improvement
M5	X_1, X_3	0.979	0.958	No real gain from X_3
M6	X_2, X_3	0.932	0.864	X_3 adds little
M7	X_1, X_2, X_3	0.986	0.944	Model becomes more complex

Also suppose the p-values in the full model $M7$ are:

- $X_1: p = 0.01$
- $X_2: p = 0.08$
- $X_3: p = 0.72$

These values suggest that X_1 is highly important, X_2 is somewhat useful, and X_3 is not useful.

1. Forward Selection: In forward selection, we begin with **no predictor** in the model and add variables one at a time.

Step 1: Start with no variable: Initial model $Y = \beta_0$; This is only the intercept model.

Step 2: Add the best single predictor: We compare one-variable models:

- X_1 : Adjusted $R^2 = 0.971$
- X_2 : Adjusted $R^2 = 0.907$
- X_3 : Adjusted $R^2 = -0.267$

Since X_1 gives the best performance, it is added first.

Selected model: $Y = \beta_0 + \beta_1 X_1$

Step 3: Test remaining variables: Now we check whether adding X_2 or X_3 improves the model.

- Add X_2 : Adjusted $R^2 = 0.970$
- Add X_3 : Adjusted $R^2 = 0.958$

Neither gives a meaningful improvement over 0.971. In fact, the adjusted R^2 does not improve. So, the **Final Forward Selection Model** is $Y = \beta_0 + \beta_1 X_1$

It can be interpreted that, Forward selection chooses Advertising spend (X_1) as the only predictor because it explains sales extremely well by itself. Adding the other variables does not improve the model enough to justify extra complexity.

2. Backward Elimination: In backward elimination, we start with **all predictors** and remove them one by one.

Step 1: Start with full model: $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3$

From the full model:

- $X_1: p = 0.01$
- $X_2: p = 0.08$
- $X_3: p = 0.72$

Since X_3 has the highest p-value and is clearly not significant, we remove X_3 .

Step 2: Fit reduced model: $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2$

Now suppose in this reduced model:

- $X_1: p = 0.01$
- $X_2: p = 0.06$

X_2 is borderline, but the model remains acceptable and interpretable.

So, the Final Backward Elimination Model is $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2$

It can be interpreted that, Backward elimination removes Temperature (X_3) because it does not contribute to predicting sales. The final model keeps Advertising and Sales Calls, suggesting that both business-related variables are useful, while weather is not relevant here.

3. Stepwise Selection: Stepwise selection is a combination of forward selection and backward elimination. A variable can be added if it helps the model, and later removed if it becomes unnecessary.

Step 1: Start with no predictor: As before, X_1 is added first because it is the strongest individual predictor.

$$Y = \beta_0 + \beta_1 X_1$$

Step 2: Consider adding another variable: Among the remaining variables, X_2 provides some additional improvement, so it is added.

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2$$

Step 3: Recheck all included variables: After adding X_2 , both X_1 and X_2 are checked again. Both remain reasonably useful, so both are kept.

Step 4: Test X_3 : Adding X_3 does not improve the model and its p-value is very high, so it is not included.

Final Stepwise Model: $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2$

It can be interpreted that, Stepwise selection arrives at a balanced model using Advertising and Sales Calls. It excludes Temperature because it does not make a meaningful contribution.

Comparison of Results

<u>Technique</u>	<u>Final Selected Model</u>
Forward Selection	$Y = \beta_0 + \beta_1 X_1$
Backward Elimination	$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2$
Stepwise Selection	$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2$

Overall Interpretation of the Results produced in this example shows that Advertising spend(X_1) is the most important predictor of sales. It alone explains a very large proportion of variation in sales. Sales calls (X_2) add some extra value, though their contribution is smaller. Temperature (X_3) does not significantly help in explaining sales and is therefore excluded in most cases.

The example also shows why model selection techniques are useful:

- Forward selection tends to produce a simpler model.
- Backward elimination starts with everything and removes unimportant variables.
- Stepwise selection balances both addition and removal of predictors.

In practice, these techniques help researchers and analysts choose a model that is not only statistically sound but also easier to interpret and use for prediction.

The discussion on model selection concludes with the understanding that model selection techniques are essential in multiple regression because they help identify the most suitable subset of predictors. In the present example, all three methods confirmed that Temperature was not useful, while Advertising was the strongest predictor. Depending on the technique used, the final model may be simpler or slightly more comprehensive, but the main objective remains the same: to obtain a regression model that is both predictive and interpretable.

Finally, it can be understood from the above discussion that Multiple regression has wide applications in both business and scientific research because many practical problems depend on several explanatory factors at the same time. In marketing, it is used to study the effect of price, promotion, and placement on sales performance. In healthcare, it helps examine how age, treatment type, and comorbidities influence patient outcomes. In finance, multiple regression supports credit scoring models by considering factors such as income, debt-to-income ratio, and credit history. In human resources, it is used for salary determination based on experience, education, and performance indicators. These examples show that multiple

regression is a versatile and powerful method for both explanation and prediction in complex real-world situations.

This model is widely used in predictive analytics because real-world outcomes are rarely influenced by a single factor. For instance, house price prediction may depend on area, location, number of bedrooms, age of the property, and nearby amenities. Multiple linear regression allows all these predictors to be considered simultaneously.

The key benefit of multiple regression is that it provides a more realistic and comprehensive explanation of the dependent variable. It also allows the analyst to measure the individual contribution of each predictor while controlling for the others. However, it requires careful attention to assumptions such as linearity, independence of errors, homoscedasticity, normality of residuals, and absence of multicollinearity. If these assumptions are violated, the accuracy and reliability of the model may be affected.

☛ Check Your Progress 3

Q1. What is Multiple Linear Regression? Explain its importance.

.....

Q2. Write the general form of a Multiple Linear Regression model and explain its components.

.....

Q3. Explain the interpretation of regression coefficients in Multiple Regression.

.....

Q4. What is Multicollinearity? How is it detected?

.....

Q5. Explain model selection techniques used in Multiple Regression.

.....

8.3.3 POLYNOMIAL REGRESSION

Polynomial regression is an important extension of linear regression that is used when the relationship between the independent variable and the dependent variable is not adequately represented by a straight line. In many real-world datasets, the pattern of association between variables is curved rather than linear. In such situations, simple linear regression may fail to provide a satisfactory fit. Polynomial regression addresses this limitation by incorporating higher-order powers of the independent variable into the regression equation, thereby allowing the model to capture non-linear relationships more effectively. Although the relationship between the variables becomes curved, polynomial regression is still considered a type of regression analysis because it remains linear in terms of the coefficients.

The general form of a polynomial regression model of degree d is expressed as:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \dots + \beta_d X^d + \epsilon$$

In this equation, Y represents the dependent variable, X is the independent variable, $\beta_0, \beta_1, \beta_2, \dots, \beta_d$ are the regression coefficients, and ϵ is the error term. The inclusion of squared, cubed, and higher-order terms allows the regression curve to bend and adapt to the underlying structure of the data. A second-degree polynomial, also called a quadratic model, takes the form:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \epsilon$$

A third-degree polynomial, or cubic model, is written as:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \epsilon$$

As the degree of the polynomial increases, the model becomes more flexible and capable of fitting increasingly complex curved patterns. However, greater flexibility must be balanced against the risk of overfitting.

A crucial question is, When to Use Polynomial Regression? It is to be noted that Polynomial regression is appropriate when the relationship between the variables clearly exhibits curvature and cannot be explained well by a simple straight-line model. One indication for its use arises when residual plots from linear regression show systematic patterns rather than random scatter, suggesting that the linear model is missing important non-linear structure. Polynomial regression is also useful when prior domain knowledge suggests that the underlying process is non-linear in nature. In addition, exploratory data analysis, particularly scatter plots, may reveal curved trends in the data that make polynomial modeling a more suitable choice. Thus, polynomial regression is selected when a linear model is too restrictive, and a curved function is needed to represent the relationship more accurately.

The graphical representation of polynomial regression typically begins with a scatter plot of data points, just as in linear regression. However, in this case the observations do not cluster around a straight-line trend; instead, they follow a curved pattern. The fitted model is represented by a smooth curved line, which serves as the polynomial line of best fit. This curve is generated by the polynomial equation and is able to reflect non-linear movement in the dependent variable as the independent variable changes. The degree of the polynomial determines the number of bends that the curve can display. For instance, a quadratic polynomial can capture one curve, while a cubic polynomial can model more complex changes with up to two bends. As a result, polynomial regression provides a more realistic representation of many naturally occurring phenomena than linear regression.

To understand **polynomial regression** clearly, let us consider a numerical example in which the relationship between the independent variable and dependent variable is **curved** rather than linear.

Example 4: Suppose a researcher wants to study the relationship between: X : Hours of practice and Y : Performance score. The observed data are:

Observation Hours of Practice X Performance Score Y

1	1	52
2	2	58
3	3	68
4	4	82
5	5	100

From the pattern of data, it appears that the increase in performance is not constant. The score is increasing at an increasing rate, which suggests a curved relationship. Therefore, instead of simple linear regression, we use a quadratic polynomial regression model.

Solution: The Polynomial Regression Model of degree 2 is written as:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \epsilon$$

where:

- Y = dependent variable
- X = independent variable
- β_0 = intercept
- β_1 = coefficient of X
- β_2 = coefficient of X^2
- ϵ = error term

For this example, the fitted model will be:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2$$

Step 1: Prepare the Calculation Table:

Since this is a quadratic model, we need the values of X^2 .

Observation	X	Y	X^2
1	1	52	1
2	2	58	4
3	3	68	9
4	4	82	16
5	5	100	25

Now compute the required totals:

$$\sum X = 1 + 2 + 3 + 4 + 5 = 15$$

$$\sum Y = 52 + 58 + 68 + 82 + 100 = 360$$

$$\sum X^2 = 1 + 4 + 9 + 16 + 25 = 55$$

$$\sum X^3 = 1 + 8 + 27 + 64 + 125 = 225$$

$$\sum X^4 = 1 + 16 + 81 + 256 + 625 = 979$$

$$\sum XY = 52 + 116 + 204 + 328 + 500 = 1200$$

$$\sum X^2 Y = 52 + 232 + 612 + 1312 + 2500 = 4708$$

Step 2: Form the Normal Equations

For quadratic polynomial regression, the normal equations are:

$$\sum Y = n\beta_0 + \beta_1 \sum X + \beta_2 \sum X^2$$

$$\sum XY = \beta_0 \sum X + \beta_1 \sum X^2 + \beta_2 \sum X^3$$

$$\sum X^2 Y = \beta_0 \sum X^2 + \beta_1 \sum X^3 + \beta_2 \sum X^4$$

Substituting the values:

$$360 = 5\beta_0 + 15\beta_1 + 55\beta_2$$

$$1200 = 15\beta_0 + 55\beta_1 + 225\beta_2$$

$$4708 = 55\beta_0 + 225\beta_1 + 979\beta_2$$

Step 3: Solve the Equations

The system of equations is:

$$5\beta_0 + 15\beta_1 + 55\beta_2 = 360$$

$$15\beta_0 + 55\beta_1 + 225\beta_2 = 1200$$

$$55\beta_0 + 225\beta_1 + 979\beta_2 = 4708$$

Solving these equations gives:

$$\beta_0 = 50$$

$$\beta_1 = 1$$

$$\beta_2 = 2$$

Step 4: Write the Fitted Polynomial Regression Equation

Substituting the estimated coefficients into the model: $\hat{Y} = 50 + 1X + 2X^2$

So, the fitted quadratic polynomial regression equation is: $\hat{Y} = 50 + X + 2X^2$

Step 5: Compute the Predicted Values

Now use the fitted equation to calculate the predicted values.

$$\text{For } X = 1: \hat{Y} = 50 + 1(1) + 2(1^2) = 50 + 1 + 2 = 53$$

$$\text{For } X = 2: \hat{Y} = 50 + 1(2) + 2(2^2) = 50 + 2 + 8 = 60$$

$$\text{For } X = 3: \hat{Y} = 50 + 1(3) + 2(3^2) = 50 + 3 + 18 = 71$$

$$\text{For } X = 4: \hat{Y} = 50 + 1(4) + 2(4^2) = 50 + 4 + 32 = 86$$

$$\text{For } X = 5: \hat{Y} = 50 + 1(5) + 2(5^2) = 50 + 5 + 50 = 105$$

So the predicted values are:

X	Actual Y	Predicted $\hat{Y}$
1	52	53
2	58	60
3	68	71

X	Actual Y	Predicted $\hat{Y}$
4	82	86
5	100	105

Step 6: Calculate Residuals: Residual $= Y - \hat{Y}$

X	Actual Y	Predicted $\hat{Y}$	Residual $Y - \hat{Y}$
1	52	53	-1
2	58	60	-2
3	68	71	-3
4	82	86	-4
5	100	105	-5

These residuals show the deviation of observed values from the fitted polynomial curve.

Step 7: Interpretation of the Polynomial Regression Equation: The fitted polynomial model is: $\hat{Y} = 50 + X + 2X^2$

- Interpretation of the Intercept ($\beta_0 = 50$): The intercept indicates the estimated value of performance score when $X = 0$. Thus, when practice hours are zero, the predicted score is 50.
- Interpretation of the Linear Coefficient ($\beta_1 = 1$): The coefficient of X shows the linear part of the relationship. It contributes positively to the score, meaning that as practice hours increase, the performance score also tends to increase.
- Interpretation of the Quadratic Coefficient ($\beta_2 = 2$): The coefficient of X^2 is positive, which means the curve bends upward. This indicates that the increase in performance score becomes larger as practice hours increase. In other words, the effect of practice is not constant; it accelerates with more hours of practice.

This is the key reason for using polynomial regression here instead of simple linear regression.

Step 8: Interpretation of Results: The fitted model suggests that there is a **non-linear positive relationship** between practice hours and performance score. Initially, the score increases slowly, but as practice hours increase further, the score rises more rapidly. This curved trend is captured by the positive quadratic term $2X^2$.

Thus, the model indicates that:

- performance improves as practice increases,

- the rate of improvement is not constant,
- the improvement becomes stronger at higher levels of practice.

This makes polynomial regression more suitable than linear regression for such data.

Step 9: Prediction Using the Model: Suppose we want to predict the performance score for a student who practices for **6 hours**. Using the fitted equation:

$$\hat{Y} = 50 + 1(6) + 2(6^2)$$

$$\hat{Y} = 50 + 6 + 72 = 128$$

So, the predicted performance score for **6 hours of practice** is:

$$\hat{Y} = 128$$

This numerical example demonstrates how **polynomial regression** is used when the relationship between variables is curved rather than linear. Using the quadratic model,

$$\hat{Y} = 50 + X + 2X^2$$

we were able to model the non-linear relationship between hours of practice and performance score. The positive quadratic term shows that performance increases at an increasing rate as practice hours rise.

Therefore, polynomial regression is especially useful when:

- a straight line does not fit the data well,
- the scatter plot shows curvature,
- the rate of change in the dependent variable is not constant.

This makes polynomial regression an important tool in predictive data analysis for modeling real-world non-linear relationships.

Through this numerical example we learned about the polynomial regression and interpreted the results, now we need to learn about the **Advantages and Applications of Polynomial regression**, it offers several important advantages, a few are listed below:

- One major advantage is that it is capable of capturing non-linear relationships, making it more flexible than simple linear regression. At the same time, it remains relatively simple to implement, especially with modern statistical software.
- Polynomial regression can model a wide range of curve shapes, and when low-degree polynomials are used, the model often remains reasonably interpretable. This balance between flexibility and simplicity makes it a valuable technique in predictive data analysis.

Polynomial regression is applied in many practical fields:

- In **biology**, it is used to model the growth rates of organisms over time, where growth may accelerate and then slow down.

- In **economics**, it helps analyze diminishing returns in production functions, where increases in input do not always lead to proportionate increases in output.
- In **engineering**, polynomial regression is useful for modeling stress-strain relationships in materials, which are often non-linear.
- In **environmental science**, it can be applied to study changes in pollutant concentration over time.
- In **finance**, it is used for modeling situations such as option pricing, where payoffs may follow non-linear structures.

These applications demonstrate that polynomial regression is especially suitable in contexts where the effect of the predictor changes at different levels.

Polynomial regression is especially useful in situations where the process under study changes direction or intensity over time. Like in the following cases:

- In **disease progression modeling**, polynomial regression helps represent epidemic curves by capturing the initial growth phase, the peak level, and the eventual decline of disease outbreaks.
- In **tissue growth analysis**, particularly in medical research, polynomial models are employed to study non-linear growth patterns, such as tumor development or tissue regeneration.
- In **automotive engineering**, polynomial regression can be used in speed control systems of autonomous vehicles to model non-linear relationships between environmental conditions and appropriate speed adjustments.

These examples highlight the practical importance of polynomial regression in fields where simple linear assumptions are too restrictive.

Despite its strengths, *polynomial regression presents several challenges that must be considered carefully. Some of the major challenges are:*

- **Challenge of overfitting.** Higher-degree polynomials may fit the training data extremely well, but such models may fail to generalize to new data and may produce poor predictive performance. This risk can be managed by using cross-validation to select an appropriate polynomial degree, applying regularization methods, and keeping the degree as low as possible while still capturing the essential shape of the data.
- Another challenge is **extrapolation**. Polynomial models may behave unpredictably outside the range of the observed data, often generating unrealistic predictions at extreme values of the independent variable. Therefore, caution must be exercised when using polynomial regression for forecasting beyond the available data range.
- A further limitation concerns **interpretability**. While low-degree polynomial models may still be understandable, higher-degree polynomials become increasingly difficult to interpret in a meaningful way. This reduces one of the main advantages traditionally associated with regression analysis, namely the ability to explain

relationships clearly. Hence, polynomial regression should be applied with care, balancing flexibility, predictive accuracy, and interpretability.

☛ Check Your Progress 4

Q1. What is Polynomial Regression? Why is it used?.

.....

Q2. Write the general form of a Polynomial Regression model.

.....

Q3. How does Polynomial Regression differ from Linear Regression?

.....

Q4. What are the advantages and limitations of Polynomial Regression?

.....

Q5. When should Polynomial Regression be preferred over Linear Regression?

.....

8.3.4 RIDGE AND LASSO REGRESSION

Ridge regression and Lasso regression are regularization techniques used to improve the performance of regression models, especially when there are many independent variables or when multicollinearity exists among the predictors. In ordinary linear regression, highly correlated predictors can make coefficient estimates unstable and may lead to overfitting. Ridge and Lasso address this issue by adding a penalty term to the regression model, which shrinks the coefficient values and helps produce a more reliable model.

The main difference between the two methods is that ridge regression shrinks coefficients but keeps all variables, whereas lasso regression can shrink some coefficients to zero and thus simplify the model. Ridge is preferred when most predictors are relevant but highly correlated, while lasso is more suitable when only a few predictors are truly important and feature selection is needed.

Overall, both ridge and lasso regression are valuable techniques in predictive data analysis because they help reduce overfitting, improve model generalization, and handle multicollinearity effectively.

Now, in this section we will discuss both type of regressions in a bit detail, let's begin with the Ridge regression.

Ridge Regression:

Ridge regression is an advanced form of linear regression that is used when the independent variables are highly correlated with each other or when the model has a large number of predictors. In ordinary least squares regression, the coefficients are estimated in such a way that the sum of squared errors between actual and predicted values is minimized. However, when multicollinearity exists, that is, when the predictor variables are strongly related to one

another, the estimated coefficients may become unstable and may vary greatly with small changes in the data. Ridge regression addresses this problem by introducing a penalty term into the regression objective function. This penalty discourages very large coefficient values and therefore helps produce a more stable and reliable model.

The objective function of ridge regression is written as:

$$\text{Minimize } \sum (Y_i - \hat{Y}_i)^2 + \lambda \sum \beta_j^2$$

In this expression, the first term, $\sum (Y_i - \hat{Y}_i)^2$, is the usual residual sum of squares from ordinary linear regression. The second term, $\lambda \sum \beta_j^2$, is called the ridge penalty or regularization term. Here, λ is known as the **regularization parameter**, and it controls the strength of the penalty. When $\lambda = 0$, ridge regression becomes identical to ordinary least squares regression. As the value of λ increases, the penalty becomes stronger, and the regression coefficients are forced to shrink toward zero. This shrinkage reduces the effect of multicollinearity and helps avoid overfitting. However, unlike lasso regression, ridge regression does not make coefficients exactly zero, which means that all predictor variables remain in the model.

Ridge regression is especially useful when there are several predictors that carry overlapping information. In such cases, instead of trying to assign large and unstable coefficients to correlated variables, ridge regression distributes their influence more smoothly by shrinking the coefficients. This often improves the predictive performance of the model, especially on new data.

For a beginner, ridge regression can be understood in a very simple way. Ordinary linear regression tries to fit the data as closely as possible, but sometimes this results in very large or unstable coefficients, especially when predictors are strongly related to each other. Ridge regression adds a “penalty” for having large coefficients. Because of this penalty, the model prefers smaller and more balanced coefficient values. This makes the model more stable and often better at prediction.

One may think of ridge regression as telling the model: “Do not only fit the data well, but also keep the coefficient values under control.”

Let’s understand the terms associated with Ridge regression by solving a beginner-level Simple Numerical Example of Ridge Regression

Example5: Let us take. Suppose a teacher wants to predict the final marks of students on the basis of two highly related predictors:

- X_1 = Hours studied
- X_2 = Number of practice tests attempted
- Y = Final marks

Assume the following small dataset:

Student Hours Studied (X_1) Practice Tests (X_2) Final Marks (Y)

Student Hours Studied (X_1) Practice Tests (X_2) Final Marks (Y)

1	1	1	52
2	2	2	57
3	3	3	63
4	4	4	68

Here, the two predictors are perfectly related because students who study more hours also attempt more practice tests. In such a case, ordinary regression faces difficulty in deciding how much weight should be given separately to X_1 and X_2 . Ridge regression helps solve this issue by shrinking the coefficients.

For simplicity, suppose the ordinary least squares regression gives the following model:

$$\hat{Y} = 47 + 3X_1 + 2X_2$$

Now let us apply ridge regression with a penalty parameter $\lambda = 1$. For beginner-level understanding, suppose the effect of ridge penalty shrinks the coefficients from 3 and 2 to 2.5 and 1.5, respectively. Then the ridge regression model becomes:

$$\hat{Y}_{ridge} = 47 + 2.5X_1 + 1.5X_2$$

Thus, the coefficients are reduced, but they are not removed from the model.

Now, let's perform Stepwise Prediction Using the Ridge Model. Let us calculate the predicted marks for each student using the ridge regression equation.

For Student 1

$$\begin{aligned}\hat{Y} &= 47 + 2.5(1) + 1.5(1) \\ \hat{Y} &= 47 + 2.5 + 1.5 = 51\end{aligned}$$

For Student 2

$$\begin{aligned}\hat{Y} &= 47 + 2.5(2) + 1.5(2) \\ \hat{Y} &= 47 + 5 + 3 = 55\end{aligned}$$

For Student 3

$$\begin{aligned}\hat{Y} &= 47 + 2.5(3) + 1.5(3) \\ \hat{Y} &= 47 + 7.5 + 4.5 = 59\end{aligned}$$

For Student 4

$$\begin{aligned}\hat{Y} &= 47 + 2.5(4) + 1.5(4) \\ \hat{Y} &= 47 + 10 + 6 = 63\end{aligned}$$

So the predicted values are:

Student	Actual Marks (Y)	Predicted Marks ($\hat{Y}_{ridge}$)
1	52	51

Student	Actual Marks (Y)	Predicted Marks ($\hat{Y}_{ridge}$)
2	57	55
3	63	59
4	68	63

The ridge regression model is: $\hat{Y} = 47 + 2.5X_1 + 1.5X_2$

This equation suggests that both hours studied and practice tests positively affect final marks. Holding the number of practice tests constant, a one-unit increase in hours studied increases the predicted marks by 2.5 marks. Similarly, holding hours studied constant, a one-unit increase in practice tests increases the predicted marks by 1.5 marks. The intercept value of 47 indicates the predicted marks when both predictors are zero.

The main point to observe here is that the coefficients in ridge regression are smaller than those in the ordinary regression model. This shrinkage is intentional. Because the predictors are highly correlated, ordinary regression may produce unstable estimates, whereas ridge regression stabilizes them by reducing their size. In this way, ridge regression reduces the risk of overfitting and improves the reliability of the model.

Thus, it can be concluded from the above numerical that Ridge regression is a useful extension of linear regression that helps deal with multicollinearity and improves the stability of the model. It works by adding a penalty term based on the squared values of the coefficients. As the regularization parameter increases, the coefficients are shrunk more strongly toward zero, but they never become exactly zero. Therefore, all variables remain part of the model. In the above numerical example, ridge regression reduced the sizes of the coefficients for hours studied and practice tests, thereby producing a more balanced model. For students, the key takeaway is that ridge regression is especially helpful when predictor variables are highly related and ordinary regression becomes unreliable.

Lasso Regression

Lasso regression, which stands for **Least Absolute Shrinkage and Selection Operator (Lasso)**, is a regularized regression technique used to improve the performance of prediction models, especially when the dataset contains a large number of independent variables. In ordinary linear regression, the coefficients are estimated only by minimizing the sum of squared errors between the actual values and the predicted values. However, when the number of predictors is large, or when several predictors are weak, irrelevant, or highly correlated, ordinary regression may produce unstable estimates and may overfit the training data. Lasso regression addresses this issue by introducing a penalty term into the objective function, thereby controlling the size of the regression coefficients and simplifying the model.

The objective function of lasso regression is:

$$\text{Minimize } \sum (Y_i - \hat{Y}_i)^2 + \lambda \sum |\beta_j|$$

In this expression, the first term, $\sum (Y_i - \hat{Y}_i)^2$, represents the residual sum of squares, which measures how closely the predicted values match the actual observations. The second term, $\lambda \sum |\beta_j|$, is the **lasso penalty**, where λ is the regularization parameter and $|\beta_j|$ denotes the absolute value of the regression coefficients. The parameter λ controls the strength of the penalty. When $\lambda = 0$, lasso regression becomes the same as ordinary least squares regression. As the value of λ increases, the penalty becomes stronger, and the regression coefficients are shrunk more aggressively.

In simple terms, lasso regression can be understood as a method that tells the model: “Fit the data well, but also remove variables that are not really helping.”

The most distinctive feature of lasso regression is that it can shrink some coefficients **exactly to zero**. This happens because the penalty is based on the absolute values of the coefficients rather than their squares. As a result, lasso not only reduces the size of coefficients but can also completely remove some predictors from the model. In this way, lasso performs **automatic variable selection**. This makes the model simpler, easier to interpret, and often more effective when dealing with datasets that contain many variables but only a few truly important ones.

This property distinguishes lasso regression from ridge regression. Ridge regression shrinks coefficients toward zero but usually keeps all variables in the model, whereas lasso regression may shrink some coefficients completely to zero and exclude the corresponding variables. Therefore, lasso is especially useful when the goal is not only prediction but also identification of the most relevant predictors.

Lasso regression is widely applied in situations where a dataset has a large number of potential explanatory variables. For example, in marketing analytics, a business may have dozens of customer-related features such as age, income, purchase history, browsing behaviour, discount usage, and demographic characteristics. Not all these variables are equally important for predicting customer spending. Lasso regression can identify the variables that truly contribute to the prediction while removing those that add little value. This makes it highly useful in machine learning, business analytics, bioinformatics, finance, and other areas involving high-dimensional data.

Because lasso regression reduces overfitting and improves interpretability, it is particularly valuable in predictive data analysis. By shrinking unimportant coefficients to zero, it creates a more compact model that is easier to understand and often better at generalizing to new data. For this reason, lasso regression is considered one of the most important regularization techniques in modern statistical learning and applied machine learning.

Let’s understand the terms associated with Lasso regression by solving a beginner-level Simple Numerical Example of Lasso Regression

A Simple Numerical Illustration is as follows:

Suppose a researcher wants to predict student performance using the following three predictors:

- X_1 : Hours studied
- X_2 : Attendance
- X_3 : Number of extracurricular activities

Assume that after fitting a regression model, the estimated coefficients before regularization are:

$$\hat{Y} = 40 + 5X_1 + 3X_2 + 0.5X_3$$

Here, the coefficient of extracurricular activities is very small compared with the other two predictors, suggesting that it may not be very important.

Now suppose lasso regression is applied with a suitable value of λ . After penalization, the model becomes:

$$\hat{Y}_{lasso} = 40 + 4.2X_1 + 2.6X_2 + 0X_3$$

This means that lasso has shrunk the coefficient of X_3 exactly to zero. As a result, the variable X_3 is automatically removed from the model, and the final simplified regression equation becomes:

$$\hat{Y}_{lasso} = 40 + 4.2X_1 + 2.6X_2$$

This illustrates the main advantage of lasso regression: it retains important variables while excluding unimportant ones. This combination of coefficient shrinkage and variable selection is what makes lasso regression especially powerful in modern predictive modelling.

The final lasso model indicates that **hours studied** and **attendance** are important predictors of student performance, while **extracurricular activities** do not contribute significantly in the presence of the other variables. By shrinking the third coefficient to zero, lasso produces a simpler and more interpretable model. This not only helps in reducing unnecessary complexity but also improves understanding of which variables truly matter in the prediction task.

One more Simple Numerical Example of Lasso Regression: Let us consider one simpler example suitable for beginner-level students.

Suppose a teacher wants to predict the **final marks** of students using the following three predictors:

- X_1 = Hours studied
- X_2 = Attendance score
- X_3 = Number of extracurricular activities

The dependent variable is:

- Y = Final marks

Assume that, using ordinary linear regression, the following model is obtained:

$$\hat{Y} = 30 + 4X_1 + 3X_2 + 0.5X_3$$

This equation means:

- each extra hour of study increases marks by 4,
- each one-unit increase in attendance increases marks by 3,

- each one-unit increase in extracurricular activities increases marks by only 0.5.

From these coefficients, it already appears that X_3 has a very small contribution compared to the other variables.

Now let us apply **lasso regression** with a penalty parameter $\lambda = 1$. For beginner-level understanding, suppose that after applying the lasso penalty, the coefficients change as follows:

- coefficient of X_1 : from 4 to 3.5
- coefficient of X_2 : from 3 to 2.5
- coefficient of X_3 : from 0.5 to 0

So, the lasso regression model becomes:

$$\hat{Y}_{lasso} = 30 + 3.5X_1 + 2.5X_2 + 0X_3$$

or simply,

$$\hat{Y}_{lasso} = 30 + 3.5X_1 + 2.5X_2$$

This shows that lasso has removed the extracurricular activities variable from the model by shrinking its coefficient exactly to zero.

Let's understand the Stepwise procedure of Prediction Using the Lasso Model, with the help of an example: Suppose we have the following student data:

Student	Hours Studied (X_1)	Attendance (X_2)	Extracurricular Activities (X_3)	Actual Marks (Y)
1	2	3	5	45
2	4	4	2	54
3	5	5	1	60
4	6	6	3	66

Using the lasso regression model:

$$\hat{Y}_{lasso} = 30 + 3.5X_1 + 2.5X_2$$

we calculate the predicted values.

For Student 1

$$\hat{Y} = 30 + 3.5(2) + 2.5(3)$$

$$\hat{Y} = 30 + 7 + 7.5 = 44.5$$

For Student 2

$$\hat{Y} = 30 + 3.5(4) + 2.5(4)$$

$$\hat{Y} = 30 + 14 + 10 = 54$$

For Student 3

$$\hat{Y} = 30 + 3.5(5) + 2.5(5)$$

$$\hat{Y} = 30 + 17.5 + 12.5 = 60$$

For Student 4

$$\hat{Y} = 30 + 3.5(6) + 2.5(6)$$

$$\hat{Y} = 30 + 21 + 15 = 66$$

Thus, the predicted values are:

Student	Actual Marks (Y)	Predicted Marks ($\hat{Y}_{lasso}$)
1	45	44.5
2	54	54
3	60	60
4	66	66

Now, we need to Understand that How the Lasso Penalty Works?

Let us now see how the penalty is applied in a simple way.

From the ordinary regression model:

$$\hat{Y} = 30 + 4X_1 + 3X_2 + 0.5X_3$$

the lasso penalty part is:

$$\lambda \sum |\beta_j|$$

Since $\lambda = 1$,

$$\text{Penalty} = |4| + |3| + |0.5| = 4 + 3 + 0.5 = 7.5$$

Now suppose the lasso model becomes:

$$\hat{Y}_{lasso} = 30 + 3.5X_1 + 2.5X_2 + 0X_3$$

Then the new penalty is:

$$|3.5| + |2.5| + |0| = 3.5 + 2.5 + 0 = 6$$

So the penalty has been reduced from **7.5 to 6**.

This is the key idea of lasso regression. It tries to balance two things at the same time:

1. fit the data well
2. keep the coefficients small

If a variable contributes very little, lasso may reduce its coefficient completely to zero in order to reduce the penalty and simplify the model.

The Interpretation of the Results from the lasso regression model i.e.:

$$\hat{Y}_{lasso} = 30 + 3.5X_1 + 2.5X_2$$

This equation shows that:

- for every additional hour of study, marks increase by **3.5 marks**, keeping attendance constant,
- for every one-unit increase in attendance, marks increase by **2.5 marks**, keeping hours studied constant,
- extracurricular activities do not make a meaningful contribution in the presence of the other variables, so their coefficient becomes zero.

This means that **hours studied** and **attendance** are the truly important predictors of final marks in this example, while **extracurricular activities** do not significantly help in prediction.

It can be understood that, lasso regression can be understood very simply, by understanding the following:

- Ordinary regression says: “Use all variables to fit the data as well as possible.”
- Lasso regression says: “Fit the data well, but also remove variables that are not really useful.”

So, lasso acts like a filter. It keeps the important variables and drops the less important ones. That is why lasso regression is very helpful when there are many predictors and the analyst wants a model that is both **accurate** and **easy to understand**.

In the numerical example above, lasso regression identified that **hours studied** and **attendance** were useful predictors of student marks, while **extracurricular activities** were not important enough to remain in the model. Therefore, the final model became simpler and more interpretable. For students, the main idea to remember is that lasso regression helps build a regression model that is not only predictive but also selective about which variables truly matter.

This section concludes that Lasso regression is a powerful extension of ordinary linear regression. It improves the model by adding a penalty based on the absolute values of the coefficients. This penalty shrinks coefficient values, and in some cases it reduces them exactly to zero. As a result, lasso regression not only reduces overfitting but also performs automatic variable selection.

☛ Check Your Progress 5

Q1. What are Ridge and Lasso Regression? Why are they used?

.....

Q2. Differentiate between Ridge Regression and Lasso Regression.

.....

Q3. Explain the concept of regularization in regression.

.....

Q4. How do Ridge and Lasso Regression help in handling multicollinearity?

.....

Q5. What are the advantages and limitations of Ridge and Lasso Regression?

.....

8.3.5 SUPPORT VECTOR REGRESSION

Support Vector Regression, commonly called **SVR**, is a regression technique derived from the concept of Support Vector Machines. While Support Vector Machines are widely known for classification problems, the same idea can also be applied to regression problems where

the goal is to predict a continuous numerical value. SVR is especially useful when the relationship between variables is complex and when the analyst wants a model that is not overly affected by noise in the data.

The basic idea of Support Vector Regression is different from ordinary linear regression. In linear regression, the model tries to minimize the total prediction error for all data points. In SVR, instead of trying to make every error as small as possible, the model attempts to fit a function in such a way that errors within a certain acceptable range are ignored. This acceptable range is called the **epsilon-insensitive tube**. In simple words, if the predicted value lies within a distance of ϵ above or below the actual value, then that error is treated as acceptable and no penalty is given. Only those points that fall outside this tube contribute to the error.

Thus, SVR tries to find a regression line or curve that is as flat as possible while ensuring that most observations lie within the epsilon margin. The observations that lie on the boundary of the tube or outside it are called **support vectors**, because these are the points that actually determine the position of the regression function.

The Support Vector Regression Model

For a simple linear case, the Support Vector Regression function is written as:

$$f(x) = wx + b$$

where:

- x is the independent variable,
- w is the weight or slope,
- b is the intercept.

The objective of SVR is to minimize:

$$\frac{1}{2} \|w\|^2$$

subject to the condition that the prediction errors remain within the epsilon tube as much as possible. When points fall outside the tube, slack variables are introduced, and the optimization becomes:

$$\text{Minimize } \frac{1}{2} \|w\|^2 + C \sum (\xi_i + \xi_i^*)$$

subject to:

$$\begin{aligned} Y_i - (wx_i + b) &\leq \epsilon + \xi_i \\ (wx_i + b) - Y_i &\leq \epsilon + \xi_i^* \\ \xi_i, \xi_i^* &\geq 0 \end{aligned}$$

Here:

- ϵ defines the width of the error tolerance tube,
- C is the penalty parameter,
- ξ_i and ξ_i^* are slack variables for points outside the tube.

It is important to understand that **SVR allows small errors but penalizes only larger errors**.

A regular question that requires attention is, Why Support Vector Regression is Useful?

The usefulness of SVR will be clearer, after understanding that the SVR balances the two goals, given below i.e.:

- keeping the model as simple and flat as possible,
- allowing small errors that do not significantly affect prediction quality.

This makes SVR effective when the data contains noise or when the relationship is not perfectly linear. It can also be extended to nonlinear regression using kernel functions, but it is best to first study the simple linear idea.

A Simple Numerical Example of Support Vector Regression is as follows:

Example6: Suppose by Using the SVR model gives the regression function:

$\hat{Y} = 5X + 45$ with prediction error $\epsilon = 2$, a teacher wants to predict **student marks** based on **hours of study**.

Let:

- X = Hours studied
- Y = Marks obtained

The observed data are:

Student	Hours Studied (X)	Marks (Y)
1	1	50
2	2	55
3	3	60
4	4	65
5	5	72

Solution: As per the given SVR model the regression function is: $\hat{Y} = 5X + 45$

This means the predicted marks are calculated as:

$$\hat{Y} = 5X + 45$$

Now given prediction error is: $\epsilon = 2$,

This means any prediction error up to **2 marks** is acceptable and will not be penalized.

Step 1: Compute Predicted Values by using the regression equation: $\hat{Y} = 5X + 45$

we get:

For $X = 1$ $\hat{Y} = 5(1) + 45 = 50$

For $X = 2$ $\hat{Y} = 5(2) + 45 = 55$

For $X = 3$ $\hat{Y} = 5(3) + 45 = 60$

For $X = 4$ $\hat{Y} = 5(4) + 45 = 65$

For $X = 5$ $\hat{Y} = 5(5) + 45 = 70$

So, the predicted values are:

X	Actual Y	Predicted $\hat{Y}$
1	50	50
2	55	55

X	Actual Y	Predicted $\hat{Y}$
3	60	60
4	65	65
5	72	70

Step 2: Compute Errors by using the Error equation: $\text{Error} = Y - \hat{Y}$

X	Actual Y	Predicted $\hat{Y}$	Error $Y - \hat{Y}$
1	50	50	0
2	55	55	0
3	60	60	0
4	65	65	0
5	72	70	2

Step 3: Apply the Epsilon Rule

In Support Vector Regression, the value of epsilon defines the acceptable margin of error around the regression line. For this example, $\epsilon = 2$.

This means that if the absolute difference between the actual value and the predicted value is **less than or equal to 2**, then that error is treated as acceptable and **no penalty** is applied.

However, if the absolute error is **greater than 2**, then the observation falls outside the epsilon-insensitive tube and a penalty is imposed.

Mathematically, the rule can be written as:

$$|Y - \hat{Y}| \leq 2 \Rightarrow \text{No penalty}$$

$$|Y - \hat{Y}| > 2 \Rightarrow \text{Penalty is applied}$$

Now let us check the error for each observation:

X	Actual Y	Predicted $\hat{Y}$	Error $Y - \hat{Y}$	Penalized?
1	50	50	0	No
2	55	55	0	No
3	60	60	0	No
4	65	65	0	No
5	72	70	2	No

Since the absolute error for every observation is less than or equal to the epsilon value of 2, all the data points lie within the epsilon-insensitive tube. Therefore, **none of the observations are penalized**. This indicates that the fitted SVR regression line is acceptable, as it keeps all prediction errors within the allowed tolerance range.

Step 4: Identify the Support Vectors: It is to be noted that in SVR, the points that lie on the epsilon boundary or outside it are the most important points. These are called **support vectors**.

Here, the point (5, 72) has error 2, which lies exactly on the epsilon boundary. So, this point acts like a support vector.

The other points lie exactly on the line, so they are also fitted well, but the point on the boundary is particularly important in defining the separation region.

Another Small Variation for Better Understanding

Suppose instead the predicted value for $X = 5$ were 68 instead of 70.

Then error would be:

$$72 - 68 = 4$$

Now:

$$|4| > 2$$

So only the excess beyond epsilon is penalized:

$$4 - 2 = 2$$

This means a penalty would apply for that point.

This shows the main feature of SVR:

- small errors inside epsilon are ignored
- only larger deviations outside epsilon matter

Interpretation of the Results are as follows: The fitted SVR model is:

$$\hat{Y} = 5X + 45$$

This equation suggests that for each additional hour of study, the predicted marks increase by **5 marks**. The intercept value **45** means that if a student studies for zero hours, the predicted marks would be 45.

In the numerical example above, the regression equation

$$\hat{Y} = 5X + 45$$

with $\epsilon = 2$ provided a good fit for the student marks data because all prediction errors remained within the acceptable range. Thus, the model successfully captured the relationship between hours studied and marks obtained.

The most important interpretation in SVR, however, is not only the slope and intercept but also the epsilon tolerance. Since $\epsilon = 2$, the model accepts prediction errors up to 2 marks without considering them serious. In this example, all prediction errors lie within this acceptable range, which means the model fits the data well according to the SVR principle.

This is different from ordinary regression, where every error is counted. SVR is more flexible because it focuses only on errors that are meaningfully large.

So, SVR can be understood in this simple way by understanding the following:

- Linear regression says: “Try to reduce every error.”
- Support Vector Regression says: “Small errors are okay; only worry about big errors.”

So, SVR creates a safe zone or tolerance band around the regression line. If a predicted value falls inside this band, the model is satisfied. Only points outside the band affect the optimization strongly.

That's why SVR is useful when the data has slight noise or variation and we do not want the model to overreact to every small difference.

Referring to the *objective of SVR that is to minimize* $\frac{1}{2} \|w\|^2 + C \sum (\xi_i + \xi_i^*)$, in this expression we need to understand the **Real Meaning of C**, another important parameter in SVR.

- A **large C** means the model gives a strong penalty to points outside the epsilon tube. So it tries harder to fit the data closely.
- A **small C** means the model is more relaxed and allows some larger deviations.

So:

- ϵ : controls how much error is acceptable
- **C** : controls how strict the model is when error becomes too large

The key idea of **SVR** to remember is that Support Vector Regression (SVR) allows small errors, ignores them, and concentrates only on large deviations, making it a useful and practical method for regression problems.

We are not going to further details of Support Vector Regression (SVR), which is a powerful regression method that predicts continuous values while allowing a certain amount of error through the epsilon-insensitive tube. Unlike ordinary regression, it does not try to minimize every small error. Instead, it focuses on finding a simple regression function that keeps most points within an acceptable error range. This makes the model more robust and less sensitive to noise.

☛ Check Your Progress 6

Q1. What is Support Vector Regression (SVR)? Explain its objective.

.....

Q2. Explain the concept of ϵ (epsilon)-insensitive loss in SVR.

.....

Q3. Differentiate between Linear Regression and Support Vector Regression.

.....

Q4. What are the advantages of Support Vector Regression?

.....

Q5. What are the limitations of Support Vector Regression?

.....

8.4 SUMMARY

Unit 8 on Regression Models provides a detailed understanding of one of the most important techniques in predictive analytics. It begins with the introduction to regression, which explains how relationships between variables are modeled to predict outcomes. Regression analysis focuses on identifying how a dependent variable changes with respect to one or more independent variables, making it a powerful tool for forecasting and decision-making in domains such as finance, healthcare, and marketing. The unit establishes the conceptual foundation by highlighting the importance of regression in understanding patterns and making data-driven predictions.

The unit then explores different types of regression models, starting with Linear Regression, which assumes a straight-line relationship between variables and is widely used due to its simplicity and interpretability. This is extended to Multiple Regression, where multiple independent variables are used to make predictions, reflecting real-world scenarios more accurately. When relationships are not linear, Polynomial Regression is introduced to model curved patterns, enabling better representation of complex data trends. These models collectively demonstrate how regression techniques can be adapted based on the nature of the dataset.

Further, the unit covers advanced techniques such as Ridge and Lasso Regression, which are used to address issues like overfitting and multicollinearity. These regularization methods improve model performance by controlling the magnitude of coefficients and, in the case of Lasso, even performing feature selection. Finally, the unit introduces Support Vector Regression (SVR), a powerful method capable of handling non-linear and complex relationships by fitting data within a specified margin of tolerance. Overall, the unit provides a comprehensive overview of regression models, equipping learners with both fundamental and advanced tools to build accurate and reliable predictive models.

8.5 ANSWERS

Check Your Progress 1

Q1. What is Regression Analysis? Explain its purpose in predictive data analysis.

Solution: refer to Section 8.2

Q2. Explain the difference between dependent and independent variables with examples.

Solution: refer to Section 8.2

Q3. Write the general form of a regression equation and explain its components.

Solution: refer to Section 8.2

Q4. Discuss the challenges and limitations of regression analysis.

Solution: refer to Section 8.2

Q5. What is the role of regression in predictive analytics?

Solution: refer to Section 8.2

☛ Check Your Progress 2

Q1. What is a Regression Model? Explain its significance in predictive analytics.

Solution: refer to Section 8.3

Q2. Define Simple Linear Regression and write its mathematical equation.

Solution: refer to Subsection 8.3.1

Q3. Explain the Ordinary Least Squares (OLS) method used in linear regression.

Solution: refer to Subsection 8.3.1

Q4. Interpret the slope and intercept in a linear regression model..

Solution: refer to Subsection 8.3.1

Q5. List the assumptions and limitations of Simple Linear Regression

Solution: refer to Subsection 8.3.1

☛ Check Your Progress 3

Q1. What is Multiple Linear Regression? Explain its importance.

Solution: refer to Subsection 8.3.2

Q2. Write the general form of a Multiple Linear Regression model and explain its components.

Solution: refer to Subsection 8.3.2

Q3. Explain the interpretation of regression coefficients in Multiple Regression.

Solution: refer to Subsection 8.3.2

Q4. What is Multicollinearity? How is it detected?

Solution: refer to Subsection 8.3.2

Q5. Explain model selection techniques used in Multiple Regression.

Solution: refer to Subsection 8.3.2

☛ Check Your Progress 4

Q1. What is Polynomial Regression? Why is it used?.

Solution: refer to Subsection 8.3.3

Q2. Write the general form of a Polynomial Regression model.

Solution: refer to Subsection 8.3.3

Q3. How does Polynomial Regression differ from Linear Regression?

Solution: refer to Subsection 8.3.3

Q4. What are the advantages and limitations of Polynomial Regression?

Solution: refer to Subsection 8.3.3

Q5. When should Polynomial Regression be preferred over Linear Regression?

Solution: refer to Subsection 8.3.3

☛ Check Your Progress 5

Q1. What are Ridge and Lasso Regression? Why are they used?

Solution: refer to Subsection 8.3.4

Q2. Differentiate between Ridge Regression and Lasso Regression.

Solution: refer to Subsection 8.3.4

Q3. Explain the concept of regularization in regression.

Solution: refer to Subsection 8.3.4

Q4. How do Ridge and Lasso Regression help in handling multicollinearity?

Solution: refer to Subsection 8.3.4

Q5. What are the advantages and limitations of Ridge and Lasso Regression?

Solution: refer to Subsection 8.3.4

☛ Check Your Progress 6

Q1. What is Support Vector Regression (SVR)? Explain its objective.

Solution: refer to Subsection 8.3.5

Q2. Explain the concept of ϵ (epsilon)-insensitive loss in SVR.

Solution: refer to Subsection 8.3.5

Q3. Differentiate between Linear Regression and Support Vector Regression.

Solution: refer to Subsection 8.3.5

Q4. What are the advantages of Support Vector Regression?

Solution: refer to Subsection 8.3.5

Q5. What are the limitations of Support Vector Regression?

Solution: refer to Subsection 8.3.5

8.6 REFERENCES/FURTHER READINGS

1. *An Introduction to Statistical Learning with Applications in R* by Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani, published by Springer (2nd Edition, 2021, ISBN: 978-1071614174).
2. *Introduction to Linear Regression Analysis* by Douglas C. Montgomery, Elizabeth A. Peck, and G. Geoffrey Vining, published by Wiley (5th Edition, 2012, ISBN: 978-0470542811).
3. *Applied Regression Analysis and Generalized Linear Models* by John Fox, published by Sage Publications (3rd Edition, 2016, ISBN: 978-1452205663).
4. *Pattern Recognition and Machine Learning* by Christopher M. Bishop, published by Springer (1st Edition, 2006, ISBN: 978-0387310732).
5. *Data Mining: Concepts and Techniques* by Jiawei Han, Micheline Kamber, and Jian Pei, published by Morgan Kaufmann (3rd Edition, 2011, ISBN: 978-0123814791).
6. *An Introduction to Statistical Learning with Applications in R* by Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani, published by Springer (2nd Edition, 2021, ISBN: 978-1071614174).
7. *Practical Statistics for Data Scientists* by Peter Bruce, Andrew Bruce, and Peter Gedeck, published by O'Reilly Media (2nd Edition, 2020, ISBN: 978-1492072942).
8. *Business Analytics: Data Analysis and Decision Making* by S. Christian Albright and Wayne L. Winston, published by Cengage Learning (7th Edition, 2020, ISBN: 978-0357131787).
9. *Introduction to Operations Research* by Frederick S. Hillier and Gerald J. Lieberman, published by McGraw-Hill Education (10th Edition, 2014, ISBN: 978-0073523453).
10. *Decision Analysis for Management Judgment* by Paul Goodwin and George Wright, published by Wiley (5th Edition, 2014, ISBN: 978-1119974017).
11. *Handbook of Statistical Analysis and Data Mining Applications* by Robert Nisbet, John Elder, and Gary Miner, published by Elsevier (2nd Edition, 2017, ISBN: 978-0124166455).
12. *Data Mining and Analysis: Fundamental Concepts and Algorithms* by Mohammed J. Zaki and Wagner Meira Jr., published by Cambridge University Press (1st Edition, 2014, ISBN: 978-0521766333).
13. *Core Concepts in Data Analysis: Summarization, Correlation and Visualization* by Boris Mirkin, published by Springer (1st Edition, 2011, ISBN: 978-0857292872).
14. *Applied Predictive Modeling*, Max Kuhn and Kjell Johnson, 1st Edition, Springer, 2013.
15. *An Introduction to Statistical Learning with Applications in R*, Gareth James, Daniela Witten, Trevor Hastie and Robert Tibshirani, 1st Edition, Springer, 2013.

UNIT 9 MINING BIG DATA

Structure

Page No.

- 9.0 Introduction
- 9.1 Expected Learning Outcomes
- 9.2 Finding Similar Items
- 9.3 Finding Similar Sets
 - 9.3.1 Jaccard Similarity of Sets
 - 9.3.2 Documents Similarity
 - 9.3.3 Collaborative Filtering and Set Similarity
- 9.4 Finding Similar Documents
 - 9.4.1 Shingles
 - 9.4.2 Minhashing
 - 9.4.3 Locality Sensitive Hashing
- 9.5 Distance Measures
 - 9.5.1 Euclidean Distance
 - 9.5.2 Jaccard Distance
 - 9.5.3 Cosine Distance
 - 9.5.4 Edit Distance
 - 9.5.5 Hamming Distance
- 9.6 Introduction to Other Techniques
- 9.7 Summary
- 9.8 Solutions/Answers
- 9.9 References/Further Readings

9.0 INTRODUCTION

In the previous Block, you have gone through the concepts of Big data and Big data handling frameworks. These concepts include the distributed file system, MapReduce and other similar architectures. This Block focuses on some of the techniques that can be used to find useful information from Big data.

This Unit focuses on the issue of finding similar item sets in Big data. The Unit also defines the measures for finding the distances between two data objects. Some of these techniques include Jaccard distance, Hamming's distance etc. In addition, the Unit also discusses finding similarities among the documents using shingles. Finally, this unit introduces you to some of the other techniques that can be used for analysing Big data.

9.1 EXPECTED LEARNING OUTCOMES

After going through this Unit, you will be able to:

- Explain different techniques for finding similar items
- Explain the process of the collaborative filtering process
- Use shingling to find similar documents
- Explain various techniques to measure the distance between two objects
- Define supervised and unsupervised learning

9.2 FINDING SIMILAR ITEMS

Finding similar items for a small set of documents may be a simple problem, but how do you find a set of similar items when the number of items is extremely large? This section defines the basic issues of finding similar items and their applications.

Problem Definition:

Given an extremely large collection of item sets, which may have millions or billions of sets, how to find the set of similar item sets without comparing all the possible combinations of item sets, using the notion that similar item sets may have many common sub-sets.

Purpose of Finding Similar Items:

1. You may like to classify web pages that are using similar words. This information can be used to classify web pages.
2. You may like to find the purchases and feedback of customers on similar items to classify them into similar groups, leading to making recommendations for purchases for these customers.
3. Another interesting use of similar item set is in entity resolution, where you need to find out if it is the same person across different applications like e-Commerce web sites, social media, searches etc.
4. Two or more web pages amongst millions of web pages may be identical. These web pages may be plagiarized or a mirror of a simple website.

Why Finding Similar Items is a problem?

One of the simplest ways to find similar items would be to compare two documents or web pages and determine if they are identical or not by comparing sequences of characters/words used in those documents/web pages. However, considering that you need to find duplicates in 10 documents/web pages, you need to compare $^{10}C_2=45$ pairs. In general, for n documents/webpages, you may need to compare $n \times (n-1)/2$ pairs of documents/web pages. For about 10^6 documents/web pages, you need to make about 5×10^{11} comparisons. Thus, the question is how to find these identical documents/web pages amongst billions of documents/web pages without checking all the combinations.

Next, we first discuss a technique to find the similarity of sets, followed by techniques that can be used to find similar item sets efficiently.

9.3 FINDING SIMILAR SETS

In this section, we discuss one of the measures of finding similarity among the sets and then discuss how this similarity measure can be used for finding the textual similarity of documents; and collaborative filtering, which can be used for finding a similar group of customers.

9.3.1 Jaccard Similarity of Sets

Jaccard similarity is defined in the context of sets. Consider two sets – set A and set B, then the Jaccard similarity $JS_{A,B}$ is defined as a ratio of the cardinality of the set $A \cap B$ and cardinality of the set $A \cup B$. The value of $JS_{A,B}$ can vary from 0 to 1. The following equation represents the Jaccard similarity:

$$JS_{A,B} = \frac{|A \cap B|}{|A \cup B|}$$

For example, consider the sets $A = \{a, b, c, d, e\}$ and set $B = \{c, d, e, f, g\}$, then Jaccard similarity of these two sets would be:

$$JS_{A,B} = \frac{|\{c, d, e\}|}{|a, b, c, d, e, f, g|} = \frac{3}{7}$$

9.3.2 Documents Similarity

Finding document similarity is an interesting domain of use of Jaccard similarity. It can be used to identify a collection of almost similar documents, news items, web pages or reports. This is also referred to as the character-level similarity of documents. Another kind of document similarity also looks at documents having similar meanings. This kind of similarity requires you to identify similar words and sometimes the meaning of the words. This is also a very useful similarity, but it can be solved using different types of techniques. How can you identify, if two documents are identical? This can be done simply by character-by-character matching. However, this algorithm may be of little use as many documents may be mostly similar though not exactly.

The following cases specify this situation:

- Plagiarism Checking: The plagiarized text may differ in small segments, as some portions of the document may have been changed or even the ordering of some of the sentences may be changed.
- Mirror Websites: Even the mirror websites also make changes, as per the local advertisements and requirements.
- The versions of Course Material: Even the versions of older course material and newer course materials, assignments etc. available on different websites may be slightly different.
- Similar News Reports: The news reports about a news item may consist of similar text, which may be appended with supplementary information by each newspaper.

9.3.3 Collaborative Filtering and Set Similarity

In the past decade, e-commerce has gained acceptance by customers. Many of you, who have purchased and have liked various items on these e-commerce websites, get online recommendations for the purchase of certain items. You may have observed that many times those recommendations are very close to purchases that you would like to do in the near future. This is achieved by the process of collaborative filtering, which tries to group you based on your purchases and likes with other users making similar purchases and likes and then recommending you the products that another person in the group has purchased and liked.

Similarity of the sets is used to address this problem. Two customers can be in a similar customer group if they purchase and like similar products. This similarity is determined by the Jaccard set similarity. However, please note that the number of customers, as well as the products that can be purchased on an e-commerce website, are very large. For example, two separate customers may be interested in purchasing books. They may purchase the same books available on Data Science and may rate them similarly. Please note they may also purchase many other books, which may be bought by only one of them. How are the problems of collaborative filtering and document similarity different? The prime difference is in the value of Jaccard similarity. For document similarity, you are looking at $JS_{A,B}$ in the range of 50% or above, whereas for the case of collaborative filtering, you may be interested if the value of $JS_{A,B}$ is as low as 10-25%.

In general, collaborative filtering uses binary data, such as like/dislike or seen/not seen or purchased/not purchased etc. This data can easily be translated into a set such as a set of likes of items; a set of purchased items; etc. Thus, finding Jaccard's similarity is straightforward. However, in many situations, a 5-, 7- or 11-point Likert scale may be used to rate the articles. An interesting way to deal with such data may be to create a bag instead of a set having as many instances of an item as the rating. For example, consider that on a 5-point scale, a customer gave a rating of 2 to product A and a rating of 3 for product B and another customer gave a rating of 1 to product A and a rating of 4 for product B, then the related data would be represented as:

BagCust1 = {a, a, b, b, b}

BagCust2 = {a, b, b, b, b}

$$JS_{A,B} = \frac{|\{a, b, b, b\}|}{|a, a, b, b, b, b|} = \frac{4}{6} = \frac{2}{3}$$

Thus, the set similarity is an important consideration for finding the similarity between two sets. In the next section, we discuss the method to convert a document into subsets of very small strings, which can be used to compute the similarity of two documents.

Check Your Progress 1:

Question 1: What is the major problem in finding similar items?

Question 2: Define Jaccard Similarity with the help of an example.

Question 3: What is collaborative filtering? How can it be addressed using set similarity?

9.4 FINDING SIMILAR DOCUMENTS

In the last section, we discussed the Jaccard similarity in the context of sets. We also explained the issues of document similarity. The document similarity can be checked using the following process:

Step 1: Create Sets of items from the document (Shingles are used)

Step 2: Perform Minhashing and create documents Signatures that can be tested for similarity checking. This reduces the number of shingles to be tested for checking the similarity of two documents.

Step 3: Perform Locality Sense Hashing produces pairs of documents that should be checked for similarity rather than all the possible pairs

In this section, we discuss these three important concepts of document similarity analysis.

9.4.1 Shingles

In order to define the term shingle, let us first try to answer the question: How to represent a document as a set of items so that you can find lexicographically similar documents?

One way would be to identify words in the document. However, the identification of words itself is a time-consuming problem and would be more useful if you are trying to find the semantics of sentences. One of the simplest and efficient ways would be to divide the document into smaller substrings of characters, say of size 3 to 7. The advantage of this division is that for almost common sentences many of these substrings would match despite small changes in those sentences or changes in the ordering of sentences.

A k-shingle is defined as any substring of the document of size k. A document has many shingles, which may occur at least once. For example, consider a document that consists of the following string:

“bit-by-bit”

Assuming the value of $k=3$ and even using a blank character as part of substrings. The following are the possible substrings of this document of size 3:

“bit”, “it-”, “t-b”, “-by”, “by-”, “y-b”, “-bi”, “bit”

Please note that out of these substrings “bit” is occurring twice. Therefore, the 3-shingles for this document would be:

{“bit”, “it-”, “t-b”, “-by”, “by-”, “y-b”, “-bi”}

One of the issues while making shingles is how to deal with white spaces. A possible solution, which is used commonly, would be to replace all the continuous white space characters with a single blank space.

So, how do you use shingles to find similar documents?

You may convert the documents into shingles and check if the two documents share the same shingles.

An interesting issue here would be to determine the size of the shingle. If the size of the shingle is small then most of the documents would have those shingles, even if they are not identical. For example, if you keep a single size of $k=1$, then almost all the characters would be shingles and these shingles would be common in most of the documents, as most of them will have almost all the alphabets. On the other hand, bigger shingles may not be able to distinguish similar items. The ideal size of a shingle is $k=5$ to 9 for small to large documents.

How many different shingles are possible for a character set? Assume that a typical language has n characters and the size of shingles is k , then the possible number of shingles is n^k . Thus, the number of possible shingles in a document may be very large.

Consider that you are using 9-shingles for finding similar research articles amongst very large size articles. The possible character set would require 26 alphabets of the English language and one character for the space character. Therefore, the maximum possible set of shingles would have $(26+1)^9 = 27^9$ 9-shingles with each shingle being 9 bytes long (assuming 1 alphabet = 1 byte). This is a very large set of data.

One of the ways of reducing the data would be to use hash functions instead of shingles. For example, assume that a hash function maps a 9-byte substring to an integral hash bucket number. Assuming that the size of the integer is 4 bytes, then the hash function maps these 27^9 possible 9-shingles to $2^{4*8}-1=2^{32}-1$ possible buckets. You may now use the bucket number as the shingle itself, thus reducing the size of the shingle from 9 bytes to 4 bytes.

For certain applications, such as finding similar news articles, shingles are created based on the words. These shingles are found to be more effective for finding the similarity of news articles.

9.4.2 Minhashing

As you can observe, the set of shingles of a document, in general, is large. Even if we reduce this set using minhashing of shingles to a smaller size, as discussed in the previous section, the size is substantial. In fact, in general, the total size of the set of shingles in Bytes is larger than the size of the document. Thus, it will not be possible to accommodate the shingles in the memory of a computer, so as to find the similarity of the documents. Is it possible to reduce

this stored size of shingles, yet not compromise too much on the estimates of Jaccard similarity?

The method used to do so is called minhashing. The idea here is to convert the set of shingles to a set of signatures of documents using a large number of permutations of rows of the matrix. In order to explain this process, let us use a visual representation of shingles using a matrix. Please note the word visual representation, as it is being used only for explanation. The actual representation would be closer to the representation of a sparse matrix. Consider a universal document that has 5 elements or shingles. The four document sets include the following shingles

Document set 1 = {2, 3, 5}

Document set 2 = {3, 4}

Document set 3 = {1, 4}

Document set 4 = {1, 2, 4}

These documents can be represented using the following matrix of Shingles and the documents:

Shingle/Documents	Set 1	Set 2	Set 3	Set 4
1	0	0	1	1
2	1	0	0	1
3	1	1	0	0
4	0	1	1	1
5	1	0	0	0

Figure 1: Matrix Representation of Documents and associated shingles

You can compute the Jaccard similarity of these documents or sets as follows:

$$JS_{SetA,SetB} = \frac{\text{The number of rows having 1's in columns of BOTH the sets}}{\text{The number of rows having 1' sin sinat least one of the two column of the sets}}$$

Please note that in the equation given above the columns of the sets having 0s in both columns are not counted, as they show that that shingle is NOT present in both columns.

Thus, for computing the similarity of set 1 and set 2 (see Figure 1), you may observe that row 1 has values 0 in the columns of set 1 and set 2 both, therefore, would not be counted. You may also observe that only row 3 has a value of 1 in columns of set 1 and set 2 both. In all the other rows only one of the two columns is 1. This indicates that only shingle 3 is present in both documents. Thus, the similarity would be:

$$JS_{Set1,Set2} = \frac{1}{4}; \quad JS_{Set1,Set3} = \frac{0}{5}; \quad JS_{Set1,Set4} = \frac{1}{5}$$

$$JS_{Set2,Set3} = \frac{1}{3}; \quad JS_{Set2,Set4} = \frac{1}{4}$$

$$JS_{Set3,Set4} = \frac{2}{3}$$

You can verify the Jaccard similarity from the set definitions also. Thus, using the matrix representation, you may be able to compute the similarity of two documents or sets.

After going through the representation and its Jaccard similarity, next, let us define the term minhashing. You can create a large number of orderings of the rows of the shingle/document matrix given in Figure 1, to generate a new sequence of rows or shingles (for our example). The first non-zero shingle of each set is called the minhashed value of that set. For example, consider the following three ordering of the matrix:

ORDER	Shingle/Documents	Set 1	Set 2	Set 3	Set 4
1 st	2	1	0	0	1
2 nd	4	0	1	1	1
3 rd	3	1	1	0	0
4 th	5	1	0	0	0
5 th	1	0	0	1	1
MH ₁ (Set n)	-	1 st	2 nd	2 nd	1 st

ORDER	Shingle/Documents	Set 1	Set 2	Set 3	Set 4
1 st	5	1	0	0	0
2 nd	1	0	0	1	1
3 rd	3	1	1	0	0
4 th	2	1	0	0	1
5 th	4	0	1	1	1
MH ₂ (Set n)	-	1 st	3 rd	2 nd	2 nd

ORDER	Shingle/Documents	Set 1	Set 2	Set 3	Set 4
1 st	1	0	0	1	1
2 nd	2	1	0	0	1
3 rd	4	0	1	1	1
4 th	3	1	1	0	0
5 th	5	1	0	0	0
MH ₁ (Set n)	-	2 nd	3 rd	1 st	1 st

Figure 2: Computation of Minhash Signatures

You may collect these minhashed values, called signatures, into a signature matrix. Thus, a typical signature matrix of the minhashing as above is given in Figure 3.

Set 1	Set 2	Set 3	Set 4
1 st	2 nd	2 nd	1 st
1 st	3 rd	2 nd	2 nd
2 nd	3 rd	1 st	1 st

Figure 3: Minhash Signature Matrix

Likewise, you can create a large number of minhashed values. But how can you compute the Jaccard similarity using these minhashed values?

In order to answer this question, you may think about the probability of finding the same minhash signatures in two columns. This would happen when both the columns have value 1 in the row, which has been selected as the first non-zero row for both columns. Further, the minhash signatures will be different if you have selected a row, which has only one column having a value 1 and the other having a value 0. Thus, going by this logic, if you select a very large number of orderings of the row, the probability of a similar minhash value of two columns would be the same as the Jaccard similarity. Thus, you can reduce the problem of finding the similarity of documents to finding the similarity of minhash signatures. Thus, reducing the complexity of the problem.

For example, you can compute the approximate similarity using Figure 3, as follows:

$$AJS_{Set1,Set2} = \frac{0}{3}; \quad AJS_{Set1,Set3} = \frac{0}{3}; \quad AJS_{Set2,Set4} = \frac{1}{3}$$

$$AJS_{Set1,Set2} = \frac{1}{3}; \quad AJS_{Set2,Set4} = \frac{0}{3}$$

$$AJS_{Set3,Set4} = \frac{2}{3}$$

Thus, you may observe that the Jaccard Similarity can be computed approximately using the signature matrix, which can be used to determine the similarity between two documents.

Now, consider the case when you are computing the signatures for 1 million rows of data. The ordering of these itself is time-consuming and representing an ordering will require large storage space. Thus, for real data, the presented algorithm may not be practical. Therefore, you would like to simulate the ordering using simple hash functions rather than actual ordering. You can decide the number of buckets, say 100, and hash the matrix (see Figure 1) into these buckets. You must select different hash functions such that a column of a row (if it has a value 1) is mapped to a different bucket by a particular hash function. This will ensure that hash functions create different ordering in the buckets. Please remember that hashing may result in collisions, but by using many hash functions, the effect of collision can be minimized. You must select the smallest bucket number to which a column is mapped as its minhash signature value. The following example explains the process with the help of two hash functions. Consider the documents and shingles given in Figure 1, and assume two hash functions given below:

$$h_1(x) = (x + 1) \bmod 5$$

$$h_2(x) = (2x + 3) \bmod 5$$

We assume 5 buckets, numbered 0 to 4. The following signatures can be created by using these hashed functions:

Initial Values:

	Set 1	Set 2	Set 3	Set 4
h_1	∞	∞	∞	∞
h_2	∞	∞	∞	∞

Figure 4: Initial hashed values

Now, apply the hash function on the first row of Figure 1. Since Set 1 and Set 2 have values 0, therefore, the value in the above table will not change.

However, Set 3 and Set 4 would be mapped as follows:

For Set 3 and Set 4:

$$h_1(1) = (1 + 1) \bmod 5 = 2$$

$$h_2(1) = (2 * 1 + 3) \bmod 5 = 0$$

Figure 4 will change now to:

	Set 1	Set 2	Set 3	Set 4
h_1	∞	∞	2	2
h_2	∞	∞	0	0

Figure 5: Hashed signature after hashing the first row

Applying the hash function on the second row of Figure 1:

For Set 1 and Set 4:

$$h_1(2) = (2 + 1) \bmod 5 = 3$$

$$h_2(2) = (2 * 2 + 3) \bmod 5 = 2$$

Since Set 4 already has lower values than these so no change will take place in set 4. Figure 5 will be modified as:

	Set 1	Set 2	Set 3	Set 4
h_1	3	∞	2	2
h_2	2	∞	0	0

Figure 6: Hashed signature after hashing the second row

Applying the hash function on the third row of Figure 1:

For Set 1 and Set 2:

$$h_1(3) = (3 + 1) \bmod 5 = 4$$

$$h_2(3) = (2 * 3 + 3) \bmod 5 = 4$$

Since Set 1 already has lower values than these so no change will take place in set 1. Figure 6 will be modified as:

	Set 1	Set 2	Set 3	Set 4
h_1	3	4	2	2
h_2	2	4	0	0

Figure 7: Hashed signature after hashing the third row

Applying the hash function on the fourth row of Figure 1:

For Set 2, Set 3 and Set 4:

$$h_1(4) = (4 + 1) \bmod 5 = 0$$

$$h_2(4) = (2 * 4 + 3) \bmod 5 = 1$$

This will result in changes in the h_1 values of Set 2, Set 3 and Set 4 and h_2 value of Set 2 only, as h_2 values of Set 3 and Set 4 are already lower. Figure 7 will be modified as:

	Set 1	Set 2	Set 3	Set 4
h_1	3	0	0	0
h_2	2	1	0	0

Figure 8: Hashed signature after hashing the fourth row

Applying the hash function on the fifth row of Figure 1:

For Set 1 only:

$$h_1(5) = (5 + 1) \bmod 5 = 1$$

$$h_2(5) = (2 * 5 + 3) \bmod 5 = 3$$

This will result in changes in the h_1 values of Set 1 only. Figure 8 will be modified as:

	Set 1	Set 2	Set 3	Set 4
h_1	1	0	0	0
h_2	2	1	0	0

Figure 9: Hashed signature after hashing the fifth row

The signature so computed will give almost the same similarity measure as earlier. Thus, we are able to simplify the process of finding the similarity between two documents. However, still one of the problems remains, i.e., there are a very large number of documents between which the similarity is to

be checked. The next section explains the process of minimizing the pairs that should be checked for similarities.

9.4.3 Locality Sensitive Hashing

The signature matrix as shown in Figure 9 can also be very large, as there may be many millions of documents or sets, which are to be checked for similarity. In addition, the number of minhash functions that can be used for these documents may be in the hundreds to thousands. Therefore, you would like to compare only those pairs of documents that have some chance of similarity. Other pairs of documents will be considered non-similar, though there may be small false negatives in this group. Locality-sensitive hashing is a technique to find possible pairs of documents that should be checked for similarity. It takes the signature matrix, as shown in Figure 9, as input and produces the list of possible pairs, which should be checked for similarity. The locality-sensitive hashing uses hash functions to do so.

The basic principle of the Locality-sensitive hashing technique is as follows:

- Step 1: Divide the overall set of the signature matrix into horizontal portions of sets, let us call them horizontal bands. This will reduce the effective size of data that is to be processed at a time.
- Step 2: Use a separate hash function to hash the column of a horizontal band into a hash bucket. It may be noted that if two sets are similar, then there is a very high probability that at least one of their horizontal band will be hashed by some hash function to the same bucket.
- Step 3: Perform the similarity checking on the pairs of sets collected in each bucket. You may please note that the probability of hashing dissimilar sets to the same bucket is low.

Thus, finally reducing the pairs that are to be checked for similarity can be reduced using locality-sensitive hashing.

You may please note that this method is an approximate method, as there would be a certain number of false positives as well as false negatives. However, given the size of the data, a small probability of errors are acceptable in the results.

Check Your Progress 2:

Question 1: What is a shingle? What are its uses? List all the 5-shingle of a document, which contain the text “This is a test”

Question 2: Consider the following Matrix representation of 2 documents and their associated shingles. What is the Jaccard Similarity of the documents?

Shingle/Documents	Set 1	Set 2
1	0	0
2	1	1
3	1	1
4	1	1
5	1	0

Question 3: Consider any three orderings of the sets to compute the minhash signature of the documents.

Question 4: Compute the signature matrix from minhash signatures and compute the similarity using the signature matrix.

Question 5: What is Locality sensitive hashing?

9.5 Distance Measures

In the previous sections, we used the Jaccard similarity to find similarity measures. In this section, we define some of the important distance measures that are used in data science. This list is not exhaustive, you may find more such measures from further readings.

The term distance is defined with reference to space, which is defined as a set of points.

A distance measure, say $distance(x, y)$, is the distance between two points in space. Some of the basic characteristics of this distance are:

1. Distance is always positive. It can be zero, only if x and y are the same points.
2. The distance measured between two points is symmetrical i.e. $distance(x, y)$ is the same as $distance(y, x)$.
3. The distance between two points would be the distance of the shortest path between two points. This can be represented with the help of a third point, say t , by the following equation:

$$distance(x, y) \leq distance(x, t) + distance(t, y)$$

Let us discuss some of the basic distance measures in the following subsections.

9.5.1 Euclidean Distance

In Euclidean space, a point can be represented as an n -dimensional vector. For example, a point x can be represented as an n -dimensional vector as $x(x_1, x_2, x_3, \dots, x_n)$. The distance of two n -dimensional points x and y in Euclidean n -dimensional space, where x is represented as $x(x_1, x_2, x_3, \dots, x_n)$ and y is represented as $y(y_1, y_2, y_3, \dots, y_n)$, can be computed using the following equation:

$$distance(x, y) = \left(\sum_{i=1}^n [|(y_i - x_i)|^m]^{\frac{1}{m}} \right)$$

In the case of $m = 1$, you get the formula:

$$manhattendistance(x, y) = \left(\sum_{i=1}^n |(y_i - x_i)| \right)$$

This is called the Manhattan distance.

In case of $m = 2$, we get the formula:

$$distance(x, y) = \left(\sqrt{\sum_{i=1}^n (y_i - x_i)^2} \right)$$

You can verify that these measures satisfy all the properties of the distance measure.

9.5.2 Jaccard Distance

As discussed earlier, you compute the Jaccard similarity of two sets, namely set A and set B, using the following formula:

$$JS_{A,B} = \frac{|A \cap B|}{|A \cup B|}$$

The Jaccard distance between these two sets is computed as:

$$JaccardDistance(A, B) = 1 - JS_{A,B} = 1 - \frac{|A \cap B|}{|A \cup B|}$$

Does this distance measure fulfil all the criteria of a distance measure?

The value of Jaccard Similarity varies from 0 to 1, therefore, the value of Jaccard distance will be from 1 to 0. The value 0 of Jaccard distance means that the value of $A \cap B$ will be the same as $A \cup B$, which can occur only if set A and set B are identical. In addition, as set intersection and union both are symmetrical operations, therefore the Jaccard distance is also symmetrical. You can also test the third property using certain values for three sets.

9.5.3 Cosine Distance

The Cosine distance is computed for those spaces where points are represented as the direction of vector components. In such cases, the cosine distance is defined as the angle between the two points. You can use the dot product and magnitude of the vectors to compute the cosine distance between two vectors. For example, consider the two vectors A[1,0,1] and B[0,1,0], the cosine distance between the two vectors can be computed as:

The dot product of the two vectors $A \cdot B = 1 \times 0 + 0 \times 1 + 1 \times 0 = 0$

$$A \cdot B = |A| \times |B| \cos\theta$$

$$0 = \sqrt{2} \times 1 \cos\theta$$

$$\cos\theta = 0 \text{ or } \theta = 90^\circ$$

You may check if this measure also fulfils the criteria of the measures. It may be noted that the range of cosine distance is evaluated between 0 and 180 degrees only. You may go through further readings for more details.

9.5.4 Edit Distance

The edit distance may be used to determine the distance between two strings. It can be defined as follows:

Consider two strings, string A consisting of n characters $a_1, a_2, \dots, a_n$ and a string B of m characters $b_1, b_2, \dots, b_m$, then edit distance between the two strings is the minimum number operations that are required to make strings identical. These operations are either the addition of a single character or the deletion of a single character.

For example, the edit distance between the strings: A= "abcdef" and B= "acdfg", would be 3, as you can obtain B from A; by deleting b, deleting e and inserting g at the end in string A.

However, finding these operations may not be easy to code, therefore, edit distance can be computed using the following method:

Step 1: Consider two strings - string A consisting of n characters $a_1, a_2, \dots, a_n$ and a string B of m characters $b_1, b_2, \dots, b_m$. Find a sub-sequence, which is the longest and has the same character sequence in the two strings. Use the deletion of character operation to do so. Assume the size of this sub-sequence is ls

Step 2: Compute the edit distance using the formula:

$$\text{editdistance}(A, B) = n + m - 2 \times ls$$

For example, in the strings A= “abcdef” and B= “acdfg”, the longest common sub-sequence is “acdf”, which is 4 characters long, therefore, the edit distance between the two strings is:

$$\text{editdistance}(A, B) = 6 + 5 - 2 \times 4 = 3$$

You can verify that edit distance is a proper distance measure.

9.5.5 Hamming Distance

Hamming distance is one of the most used distances in digital design and error detection in digital systems, e.g. while mapping of min-terms into Karnaugh’s map or single error correcting code. The Hamming distance is defined as the difference between the components of two vectors, especially when these vectors are of type Boolean.

For example, consider the following two Boolean vectors:

Vector A	1	0	1	1	0	0	1	1
Vector B	1	1	0	1	0	0	0	0
Difference	N	Y	Y	N	N	N	Y	Y

The Hamming distance between the vectors A and B is 4.

You may verify that Hamming distance also satisfies the properties of a distance measure.

You may use any of these distance measures based on the type of data being used or the type of problem.

9.6 INTRODUCTION TO OTHER TECHNIQUES

Big data analytics are the processes that are used to analyse Big data, which include structured and unstructured data, to produce useful information for organisational decision-making. In this section, we introduce some of the techniques that are useful for extracting useful information from Big data. You may refer to the latest research articles from various journals for more details on such techniques.

Textual Analysis:

Textual data is produced by a large number of sources of Big data, e.g. email, blogs, websites, etc. Some of the important types of analytics that you may perform on the textual data are:

1. Structured Information Extraction from the unstructured textual data: The purpose here is to generate information from the data that can be stored for a longer duration and can be reprocessed easily if needed. For example, the Government may find the list of

medicines that are being prescribed by doctors in various cities by analysing the prescriptions given by the doctors to different patients. Such analysis would essentially require recognition of entities, such as doctor, patient, disease, medicine etc. and then relationships among these entities, such as among doctor, disease and medicine.

2. **Meaningful summarization of single or multiple documents:** The basic objective here is to identify and report the key aspects of a large group of documents, e.g. financial websites may be used to generate data that can be summarised to produce information for stock analysis of various companies. In general, the summarization techniques either extract some important portions of the original documents based on the frequency of words, location of words etc.; or semantic abstraction-based summaries, which use Artificial Intelligence to generate meaningful summaries.
3. **Question-Answering:** In the present time, these techniques are being used to create automated query-answering systems. Inputs to such systems are the questions asked in the natural languages. These inputs are processed to determine the type of question, keywords or semantics of the questions and possible focus of the question. Next, based on the type of the Question-Answering system, which can be either an Information Retrieval based or a knowledge-based system, either the related information is retrieved or information is generated. Finally, the best possible answer is sent to the person who has asked the question. Some examples of Question-answering systems are – Siri, Alexa, Google Assistant, Cortona, IBM Watson etc.
4. **Sentiment Analysis:** Such analysis is used to determine the opinion or perception of feedback about a product or service. Sentiment analysis at the document level determines if the given feedback is positive feedback or negative feedback. However, techniques have been developed to perform sentiment analysis at the sentence level or aspect level.

Audio-Video Analysis:

The purpose of audio or video analysis is to extract useful information from speech data or video data. Most of the calls to the customer call centres are recorded for performance enhancement. One of the common approaches to dealing with such data is to transcribe the speech data using automated speech recognition software. Such software converts the speech into text, which is checked in a dictionary to ascertain if such a word exists or not. Even phonetics can be used for converting speech to text.

Video analysis is performed on video or closed-circuit television (CCTV) data. The objective of such data analysis is to check for security breaches, which may be checked by changes in CCTV data if any. In addition, it can be used to perform indexing of videos, so as to find relevant information at a later time.

Social Media Data Analysis:

This is one of the largest chunks of present-day data. Such data can be part of social networks, micro-blogging, wiki-based content and many other related web applications. The challenge here is to obtain structured information from noisy, user-oriented, unstructured data available in a large number of diverse and dispersed web pages to produce information like finding a connected group of people, community detection, social influence, link predictions etc. leading to applications like recommender systems.

Predictive Analysis:

The purpose of such analysis is to predict some of the patterns of the future based on the present and past data. In general, predictive analysis uses some statistical techniques like moving averages, regression and machine learning. However, in the case of Big data, as the size of the data is very large and it has low veracity, you may have to develop newer techniques to perform predictive analysis.

Supervised and Unsupervised Learning:

Some of the Big data problems can also be addressed using machine learning algorithms, which create models by learning from the enormous data. These models are then used to make future predictions. In general, these algorithms can be classified into two main categories – Supervised Learning algorithms and unsupervised learning algorithms.

Supervised Learning:

Supervised learning uses already available knowledge to generate models, one of the most common examples of supervised learning is spam detection in which the word patterns and anomalies of already classified emails (spam or not-spam) are used to develop a model, which is used to detect if a newly arrived email is spam or not.

Supervised learning problems can further be categorised into classification problems and regression problems.

Unsupervised Learning:

Unsupervised learning generates its own set of classes and models, one of the most common examples of unsupervised learning is creating a new categorisation of customers based on their feedback on different types of products. This new categorisation may be used to market different types of products to these customers. Such types of problems are also called clustering problems.

You can obtain more details on supervised and unsupervised learning in the AI and Machine learning course (MCS224).

Check Your Progress 3:

1. What is the purpose of a distance measure? What are the characteristics of distance measures?
2. What is the Cosine distance measure? How is it different from Hamming's distance?
3. Explain the purpose of text analysis.

This unit introduces you to basic techniques for finding similar items. The Unit first explains the methods of finding similarity between two sets. In this context, the concept of Jaccard similarity, document similarity and collaborative similarity are discussed. This is followed by a detailed discussion on one of the important set similarity applications – document similarity. For finding document similarity of a very large number of documents, a document is first converted to a set of shingles, next the minhashing function is used to compute the signatures of document sets against these shingles. Finally, locality-sensitive hashing is used to find the sets that must be compared to find similar documents. The locality-sensitive hashing greatly reduces the number of documents that should be compared to find similar documents. The Unit then describes several common distance measures that may be used in different types of Big data analysis. Some of the distance measures defined are Euclidean distance, Jaccard distance, Cosine distance, Edit distance and Hamming distance. Further, the unit introduces you to some of the techniques that are used to perform analysis of Big data.

9.8 SOLUTIONS/ANSWERS

Check Your Progress 1:

1. For finding similar items, e.g. web pages, the first problem is the representation of web pages into the sets, which can be compared. The major problem, in finding similar items, is the size of the data. For example, if you find duplicate web pages, you may have to compare billions of pairs of web pages.
2. You can define Jaccard's similarity between two sets, viz. Set A and Set B as:

$$JS_{A,B} = \frac{|A \cap B|}{|A \cup B|}$$

Consider two sets $A = \{a, b, c, d, e, f, g\}$ and $B = \{a, c, e, g, i\}$, then Jaccard similarity between the two sets is:

$$JS_{A,B} = \frac{|\{a, c, e, g\}|}{|\{a, b, c, d, e, f, g, i\}|} = \frac{4}{8} = \frac{1}{2}$$

3. Collaborative filtering is the process of grouping objects based on certain criteria and providing possible suggestions to those objects based on the activities of other objects in the group. The similarity is the basis on which these groups are constructed.

Check Your Progress 2:

1. Shingle is a small string of size 3-9 characters, which can be part of a document. Shingles are used to convert a document into sets, which can be checked for similarity. The document containing the text "This is a test document" has the following set of 5-shingles (_ is used to represent a space character):
 $\{\text{This_}, \text{his_i}, \text{is_is}, \text{s_is_a}, \text{_is_a}, \text{is_a_}, \text{s_a_t}, \text{_a_te}, \text{a_tes}, \text{_test}\}$
2. Jaccard Similarity of the document is:

$$JS_{Set1,Set2} = \frac{|\{Row2, Row3, Row4\}|}{|\{Row2, Row3, Row4, Row5\}|} = \frac{3}{4}$$

3. The minhash signatures for the following three orderings are shown:

ORDER	Shingle/Documents	Set 1	Set 2
1 st	1	0	0
2 nd	4	1	1
3 rd	3	1	1
4 th	5	1	0
5 th	2	1	1
MH ₁ (Set n)	-	2 nd	2 nd

ORDER	Shingle/Documents	Set 1	Set 2
1 st	4	1	1
2 nd	1	0	0
3 rd	2	1	1
4 th	5	1	0
5 th	3	1	1
MH ₁ (Set n)	-	1 st	1 st

ORDER	Shingle/Documents	Set 1	Set 2
1 st	5	1	0
2 nd	4	1	1
3 rd	3	1	1
4 th	2	1	1
5 th	1	0	0
MH ₁ (Set n)	-	1 st	2 nd

4. The minhash signature matrix is:

Set 1	Set 2
2 nd	2 nd
1 st	1 st
1 st	2 nd

Similarity using signatures = 2/3

5. The locality sensitive hashing first divides the signature matrix into several horizontal bands and hashes each column using a hash function to hash buckets. The similarity is checked only for the documents that hash into the same bucket. It may be noted that the chances of similar document sets may get to the same hashed bucket by at least one hash function is high, whereas the documents that are not similar have low chances of hashing into the same bucket.

Check Your Progress 3:

- Distance measures are used to find differences between two entities based on some criteria. They are used in problems that measure similarity, which is complementary to distance. The distance measure should be such that it is positive, symmetric and trigonometric (that is the distance between two points is less than or equal to the sum of the distance from the first point to a third point and the distance of the third point to the second point.)
- Cosine distance measures the angular distance from a specific reference point. It is in general in the range from 0 to 180 degrees. Hamming distance is normally used as the binary distance between two Boolean vectors.

3. Some of the common uses of text analysis are the extraction of structured information from unstructured text, meaningful summarization of documents, question-answering systems and sentiment analysis.

9.9 REFERENCES/FURTHER READINGS

1. Leskovec J., Rajaraman R, Ullman J, **Mining of Massive Datasets**, 3rd Edition, available on the website <http://www.mmids.org/>
2. Gandomi A., Haider M., **Beyond the hype: Big data concepts, methods, and analytics**, International Journal of Information Management, Volume 35, Issue 2, 2015, Pages 137-144, ISSN 0268-4012.



UNIT 10 SUPERVISED LEARNING MODELS

10.0 Introduction

10.1 Expected Learning Outcomes

10.2 Supervised Vs Unsupervised Learning

10.3 Supervised Learning models

10.3.1 K-Nearest Neighbour,

10.3.2 Naïve-Bayes,

10.3.3 Logistic regression,

10.3.4 Decision trees

10.3.5 Random Forest

10.3.6 Support vector Machines

10.4 Summary

10.5 Answers

10.6 References/Further Readings

10.0 INTRODUCTION

Data analysis is essential for turning observations into meaningful predictions and actionable insights. In critical fields such as finance and biomedicine, forecasting future outcomes and accurately classifying data play a vital role in decision-making. Supervised learning formalises this predictive process when the outcome variable is known for a portion of the dataset. It enables the development of models that learn from past observations and apply that knowledge to future scenarios, thereby bridging the gap between raw data and intelligent decision systems.

A strong foundation in mathematics and statistics is essential for understanding supervised learning methodologies. Most machine learning frameworks rely heavily on statistical reasoning, probability theory, and optimization techniques. To effectively grasp supervised learning, one must be familiar with key concepts such as statistical models, loss functions, optimization algorithms, and performance evaluation metrics. These elements collectively determine how well a model learns patterns from data and how accurately it performs in real-world applications.

Supervised learning is fundamentally characterized by the use of labelled training data. For every input instance (X), there exists a corresponding output (Y), and the goal of the algorithm is to learn a mapping function, f , such that $f(X) \approx Y$. Once this relationship is learned, the model can generalize to new, unseen inputs (X_{new}) and predict corresponding outputs (Y_{new}). However, the true effectiveness of a model lies not in how well it performs on training data, but in its ability to generalize to new data. This is measured through generalization error. Poor-quality or biased labels can significantly affect this ability, leading to irreducible errors that cannot be corrected even with advanced algorithms.

Supervised learning problems are broadly categorized into two types:

- Regression, and
- Classification.

Regression deals with predicting continuous values, such as stock prices, temperature, or sales figures. In contrast, *Classification* focuses on assigning discrete labels to data, such as identifying whether an email is spam or not. The choice between regression and classification depends on the nature of the target variable and the problem context.

In practical applications, supervised learning is used across a wide range of domains including image recognition, natural language processing, credit scoring, fraud detection, and medical diagnosis. The typical workflow involves splitting the dataset into training and testing sets, where the training data is used to build the model and the testing data evaluates its predictive performance. Data preprocessing steps such as handling missing values, encoding categorical variables, and normalizing features are critical for ensuring reliable results. Additionally, techniques like cross-validation are employed to fine-tune model parameters and reduce the risk of overfitting, thereby improving model robustness.

Another important aspect of this unit is understanding the distinction between supervised and unsupervised learning. While supervised learning uses labeled data to make predictions, unsupervised learning works with unlabeled data to discover hidden patterns or structures, such as clustering similar data points. This distinction is essential in selecting the appropriate analytical approach based on the problem and data availability.

This unit further introduces several important supervised learning models that are widely used in data analysis. The **K-Nearest Neighbors (KNN)** algorithm is a simple, distance-based method that classifies data based on similarity. The **Naïve Bayes** classifier applies probabilistic reasoning using Bayes' theorem and is particularly effective in text classification tasks. **Logistic Regression** is a widely used statistical technique for classification problems, especially binary outcomes, offering interpretability and efficiency.

The unit also covers **Decision Trees**, which provide a rule-based, hierarchical structure for decision-making and are easy to interpret. Building on this, **Random Forest** is an ensemble technique that combines multiple decision trees to improve prediction accuracy and reduce overfitting. Finally, **Support Vector Machines (SVM)** are introduced as powerful models that determine optimal decision boundaries (hyperplanes) to separate different classes, particularly effective in high-dimensional data spaces.

Overall, this unit provides a comprehensive foundation in supervised learning by integrating theoretical concepts with practical relevance. It equips learners with the knowledge required to understand, implement, and evaluate various supervised learning models, enabling them to transform data into meaningful insights and predictive solutions.

10.1 EXPECTED LEARNING OUTCOMES

After completing this Unit, you will be able:

- To differentiate between supervised and unsupervised learning approaches.
- To comprehend the concept of labeled data and mapping functions in supervised learning.
- To identify and distinguish between regression and classification problems.

- To gain knowledge of key supervised learning models such as KNN, Naïve Bayes, Logistic Regression, Decision Trees, Random Forest, and SVM.

10.2 SUPERVISED VERSUS UNSUPERVISED LEARNING

In the previous section, we learned that Supervised learning mainly focuses on two types of problems based on the type of output (Y): classification and regression. These two approaches help in solving different kinds of real-world prediction problems depending on whether the output is categorical or continuous.

Classification tasks are used when we need to predict a category or label. In this case, the output is discrete, meaning it belongs to a fixed set of classes. For example, an email can be classified as spam or not spam, a user may or may not click on an advertisement, or an image can be identified as a specific object. The main goal of classification models is to correctly assign the right label to each input. These models are designed to improve accuracy and clearly separate different classes using techniques like Logistic Regression or Support Vector Machines.

Regression tasks, on the other hand, are used when the output is a continuous value. This means the result can take any value within a range. Examples include predicting house prices, stock market trends, energy consumption, or disease growth over time. In regression, the aim is to minimize the difference between the predicted and actual values. This is usually done using error measures like Mean Squared Error (MSE) or Mean Absolute Error (MAE).

The supervised learning process follows a step-by-step workflow:

- First, an appropriate algorithm is selected based on the problem.
- Then, the model is trained and evaluated, and improvements are made by adjusting parameters or changing the model structure.

This process continues until the model performs well.

While choosing a model, several factors must be considered, such as training speed, memory usage, accuracy, and interpretability. In fields like healthcare or finance, simpler and more interpretable models are often preferred even if they are slightly less accurate, because understanding the decision process is very important.

Improving a model is an iterative process. It involves managing model complexity to avoid problems like overfitting (model learning too much from training data) or underfitting (model not learning enough). This can be done by tuning parameters, handling imbalanced data, or modifying the model structure, such as limiting the depth of a decision tree. These adjustments help maintain a balance between bias and variance.

Finally, before building any model, data preparation is very important. Steps like handling missing values, normalizing data, and creating useful features help improve model

performance. Good preprocessing ensures that the model works accurately and reliably when applied to real-world data.

Now we will understand the distinction between supervised and unsupervised learning.

In machine learning, data can be analyzed using two main approaches: supervised learning and unsupervised learning.

The primary difference between these approaches lies in the type of data used and the objective of the analysis.

- Supervised learning works with labeled data, where both the input (X) and the correct output (Y) are known. The model learns from this data and tries to predict the correct output for new inputs.
- In contrast, unsupervised learning works with unlabeled data, where only input features are available. Instead of predicting outcomes, the model identifies hidden patterns, structures, or groupings within the data.

To understand this difference more clearly, consider a simple example.

- In supervised learning, if we have emails labeled as “spam” or “not spam,” the model learns from these examples and can classify new emails accordingly.
- On the other hand, in unsupervised learning, if we have a large collection of emails without any labels, the model groups similar emails together based on patterns, such as grouping promotional emails separately from personal or work-related messages.

The objectives of these two approaches are also different.

- Supervised learning focuses on prediction, meaning it tries to determine the correct output for given inputs.
- Unsupervised learning, however, focuses on pattern discovery, aiming to uncover relationships or structures that are not immediately visible.

This difference in objectives makes supervised learning suitable for tasks like classification and regression, while unsupervised learning is more useful for clustering and data exploration.

Another important distinction is related to data requirements.

- Supervised learning requires high-quality labeled data, which can be expensive and time-consuming to collect because it often involves human effort.
- In contrast, unsupervised learning uses raw, unlabeled data, which is easier and cheaper to obtain.

The way performance is evaluated also differs between the two.

- In supervised learning, evaluation is straightforward because predicted outputs can be compared directly with actual outputs using metrics such as accuracy, precision, and recall.

- However, in unsupervised learning, there are no true labels available, so evaluation relies on indirect measures such as how well the data is grouped or how meaningful the discovered patterns are.

In real-life terms,

- supervised learning can be compared to learning with a teacher, where correct answers are provided during training.
- Unsupervised learning, on the other hand, is like self-learning, where patterns are discovered without guidance.

Due to the high cost of labeling data, modern approaches are developed, such as:

- semi-supervised and
- self-supervised learning.

These methods combine small amounts of labeled data with large amounts of unlabeled data to improve learning efficiency and performance. We are not going into their details.

Overall, both supervised and unsupervised learning play important and complementary roles in data analysis.

- Supervised learning is best suited for making accurate predictions when labeled data is available, while
- Unsupervised learning is useful for exploring and understanding hidden patterns in data.

Together, they form the foundation of modern machine learning and intelligent data-driven systems.

So, we learned that in machine learning, one of the main differences lies in how the data is used.

- Supervised learning requires **labeled data**, which means that for every input, the correct output is already known. For example, if we want to build a system to recognize handwritten digits, we need images that are already labeled with the correct number. Using these examples, the model learns patterns like shapes and pixel arrangements and then uses this knowledge to predict the correct output for new data.
- On the other hand, unsupervised learning works with **unlabeled data**, where no correct answers are given. Instead of predicting outcomes, the model tries to find hidden patterns or relationships in the data. For example, it can group news articles into categories like sports or crime, or cluster customers based on similar buying behavior.

It is to be noted that, unlike supervised learning, the unsupervised learning does not have predefined outputs to guide it. The main strength of Unsupervised learning is discovering patterns that may not be obvious. This makes it very useful for tasks like data exploration,

anomaly detection, and reducing data complexity. It is especially helpful when labeling data is difficult, expensive, or time-consuming.

The way we measure success is also different in both approaches.

- In supervised learning, we can directly compare predicted results with actual results using metrics like accuracy, precision, and recall.
- However, in unsupervised learning, since there are no correct answers, we use indirect methods such as how well the data is grouped or how meaningful the patterns are.

So, we learned that supervised learning is best when we want to make accurate predictions using labeled data, while unsupervised learning is useful for discovering hidden patterns in unlabeled data. Both approaches are important and complement each other in solving real-world data problems.

☛ Check Your Progress 1

1. What is the main difference between supervised and unsupervised learning?
.....
.....
2. Which type of learning is used for predicting house prices and why?
.....
.....
3. Give one real-life example of unsupervised learning.
.....
.....
4. Why is data preprocessing important in machine learning?
.....
.....
5. How is supervised learning similar to learning with a teacher?
.....
.....

10.3 SUPERVISED LEARNING MODELS

In the above we learned about Supervised learning and compared it with Unsupervised learning, on the basis of various aspects. Now, in this section we are going to discuss about various Supervised learning models, which are widely used in data analysis, and they learn from labeled data to make predictions or classifications.

These models identify patterns between input features and known outputs, enabling them to classify/predict outcomes for new, unseen data. Different supervised learning algorithms are designed to handle various types of problems such as classification and regression, each with its own strengths and applications. The prominent supervised learning models are as follows:

- *K-Nearest Neighbours (KNN)* is a simple and intuitive algorithm that classifies a data point based on the majority class of its nearest neighbours. It relies on distance

measures such as Euclidean or Manhattan distance and does not require an explicit training phase. Its performance depends on the choice of K and proper feature scaling.

- *Naïve Bayes* is a probabilistic classifier based on Bayes' Theorem. It assumes that features are independent of each other and calculates the probability of each class given the input features. Despite its simplicity, it performs well in applications like spam detection and text classification.
- *Logistic Regression* is a statistical model used mainly for binary classification problems. It estimates the probability of an outcome using a logistic (sigmoid) function and provides interpretable results, making it useful in fields like healthcare and finance.
- *Decision Trees* are tree-structured models that make decisions based on a series of conditions or rules derived from the data. They are easy to understand and interpret but may suffer from overfitting if not properly controlled.
- *Random Forest* is an ensemble learning method that combines multiple decision trees to improve prediction accuracy and reduce overfitting. By aggregating the outputs of several trees, it produces more stable and reliable results.
- *Support Vector Machines (SVM)* are powerful models that classify data by finding the optimal boundary (hyperplane) that separates different classes. They are particularly effective in high-dimensional spaces and can handle both linear and non-linear classification using kernel functions.

These supervised learning models provide a comprehensive toolkit for solving real-world classification problems. The choice of model depends on factors such as the nature of the data, interpretability requirements, and computational efficiency.

But, before going to next section wherein we are going to discuss the technical aspects of the various models of the supervised learning (mentioned above), Let's discuss some technicalities of the workflow of the Supervised learning itself, we are going to start with the Basic Idea of Model Training.

It is to be noted that, in supervised learning, the main goal is to build a model that can correctly predict outputs from inputs. To do this, the model uses something called a **loss function**, which measures how far the predicted value is from the actual value. The training process tries to minimize this error so that predictions become more accurate.

However, if a model learns too much from training data, it may start memorizing noise instead of actual patterns. This problem is called **overfitting**. To avoid this, techniques like **regularization** are used (we learned about regularization in our earlier unit), which control the complexity of the model and help it perform better on new data.

Further, to minimize error, optimization techniques such as **Gradient Descent** are used. This method adjusts model parameters step by step in the direction that reduces error. For large datasets, a faster version called **Stochastic Gradient Descent (SGD)** is used, which works on small batches of data and speeds up training.

Another important concept is the **bias-variance trade-off**. Bias occurs when a model is too simple and cannot capture the real pattern (underfitting), while variance occurs when the model is too complex and reacts too much to small changes in data (overfitting). A good model balances both bias and variance to achieve better performance on new data.

Overall, the goal of supervised learning is to create a model that is accurate, stable, and capable of making reliable predictions on unseen data.

There are various Supervised Learning models to be discussed in this unit, which includes, K-Nearest neighbour, naïve-bayes, Logistic regression, decision trees, Random Forest and Support vector Machines.

In the subsequent section we are going to discuss one of the simplest models i.e. K-Nearest Neighbour (K-NN)

☛ Check Your Progress 2

Q1. What is Supervised Learning and how does it support predictive analytics?

.....
.....

Q2. What are regression and classification problems in supervised learning?

.....
.....

Q3. Differentiate between supervised and unsupervised learning..

.....
.....

Q4. Explain the workflow of supervised learning.

.....
.....

Q5. List and briefly explain any three supervised learning models

.....
.....

Q6. What factors should be considered while selecting a supervised learning model?

.....
.....

10.3.1 K-NEAREST NEIGHBORS (K-NN)

The K-Nearest Neighbors (KNN) algorithm is one of the simplest and most intuitive supervised learning techniques used for both classification and regression tasks. It is a **non-parametric and instance-based learning algorithm**, meaning it does not build an explicit model during training but instead makes predictions based on the stored training data. The core idea behind KNN is that similar data points exist close to each other in the feature space. Therefore, the class or value of a new data point can be determined by examining its nearest neighbors.

In simple terms, KNN works on the principle of “**Similarity**” or “**Closeness**.” For example, in real life, if a new student joins a class and we want to guess their performance, we may compare them with students who have similar study habits or background. Similarly, in a

movie recommendation system, if a user likes certain movies, the system recommends movies liked by users with similar preferences. These are practical examples of how KNN operates.

The working of the KNN algorithm involves the following steps:

- first, choose the value of K (the number of nearest neighbors).
- Then, calculate the distance between the new data point and all points in the training dataset.
- Next, select the K closest data points based on the calculated distance.
- Finally, for classification, assign the class that is most frequent among the neighbors, and for regression, compute the average of the values of the neighbors.

The most commonly used distance measure in KNN is the **Euclidean Distance**, which is given by the formula:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

For higher dimensions, the formula extends as:

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Other distance measures such as Manhattan distance or Minkowski distance can also be used depending on the problem.

Note: The **Manhattan distance**, also known as L1 distance, is a method used to measure the distance between two points by calculating the sum of the absolute differences of their coordinates. It is called Manhattan distance because it resembles movement in a city like Manhattan, where one can only travel along streets in horizontal and vertical directions rather than taking a straight diagonal path. The formula for Manhattan distance is given by:

$$d = \sum_{i=1}^n |x_i - y_i|$$

where x_i and y_i represent the values of the i^{th} feature of two data points, and n is the total number of dimensions. The absolute value ensures that all differences are treated as positive distances.

For example, if we consider two points A(2, 3) and B(5, 7), the Manhattan distance between them is calculated as $|2 - 5| + |3 - 7| = 3 + 4 = 7$. This shows that the distance is computed by moving step-by-step along each axis rather than directly.

The **Minkowski Distance** is a more general and flexible distance measure that includes both Manhattan and Euclidean distances as special cases. Its formula is given by:

$$d = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}}$$

where p is a parameter that determines the type of distance, and x_i , y_i , and n is the total number of dimensions.

- When $p = 1$, the Minkowski distance becomes the Manhattan distance, and
- When $p = 2$, it becomes the Euclidean distance.

For example, if we take two points A (2, 3) and B (5, 7) and choose $p = 3$, the distance is calculated as $(|2 - 5|^3 + |3 - 7|^3)^{1/3} = (27 + 64)^{1/3} = 91^{1/3} \approx 4.48$.

This demonstrates how the Minkowski distance can adapt to different scenarios by adjusting the value of p .

Note: It is to be noted that the Lower values of p treat all differences more uniformly, while higher values give more importance to larger differences.

Thus, Minkowski distance provides a flexible framework for measuring similarity between data points in various machine learning applications.

Now, to understand KNN more clearly, consider a simple numerical example. Suppose we have the following training dataset representing students based on two features: study hours and attendance percentage. The target variable indicates whether the student passed (Yes) or failed (No).

Study Hours	Attendance (%)	Result
2	50	No
4	60	No
6	70	Yes
8	80	Yes

Now, suppose a new student has studied for 5 hours and has 65% attendance. We want to predict whether this student will pass or fail using KNN with $K = 3$.

First, we calculate the Euclidean distance between the new student (5, 65) and all training points:

- Distance from (2, 50): $\sqrt{(5 - 2)^2 + (65 - 50)^2} = \sqrt{9 + 225} = \sqrt{234} \approx 15.33$
- Distance from (4, 60): $\sqrt{(5 - 4)^2 + (65 - 60)^2} = \sqrt{1 + 25} = \sqrt{26} \approx 5.10$
- Distance from (6, 70): $\sqrt{(5 - 6)^2 + (65 - 70)^2} = \sqrt{1 + 25} = \sqrt{26} \approx 5.10$
- Distance from (8, 80): $\sqrt{(5 - 8)^2 + (65 - 80)^2} = \sqrt{9 + 225} = \sqrt{234} \approx 15.33$

Next, we select the 3 nearest neighbors (smallest distances):

- (4, 60) → No
- (6, 70) → Yes
- (2, 50) → No (or (8,80) depending on tie, but both far; typically, next nearest is chosen consistently)

Among these 3 neighbors:

- No → 2 times
- Yes → 1 time

Therefore, the predicted class for the new student is **“No” (Fail)**.

The interpretation of this result is that the new student is more similar to students who failed, based on the selected features. Hence, the model predicts that the student is likely to fail. This demonstrates how KNN uses proximity to make decisions rather than learning a mathematical model.

It is very important to understand that the performance of the K-Nearest Neighbors (KNN) algorithm is strongly influenced by the choice of the parameter K, which represents the number of nearest neighbors considered while making a prediction.

- If the value of K is very small, the model may become too sensitive to individual data points or noise, leading to **overfitting**.
- On the other hand, if the value of K is too large, the model may become too generalized and fail to capture important local patterns, leading to **underfitting**.

In addition, KNN is a distance-based algorithm, so the scale of the features plays a major role. If one feature has much larger values than another, it can dominate the distance calculation, which may produce misleading results. Therefore, feature normalization is often necessary.

To understand this clearly, consider the following numerical example. Suppose we want to classify whether a student will **Pass** or **Fail** based on two features: **Study Hours** and **Marks in Internal Assessment**.

The training data is given below:

Student	Study Hours	Internal Marks	Result
A	1	40	Fail
B	2	42	Fail
C	3	45	Fail

Student	Study Hours	Internal Marks	Result
D	4	48	Pass
E	5	50	Pass
F	6	52	Pass
G	7	54	Pass
H	8	90	Fail

Now suppose a new student(X)has joined the class with record of 5 hours of study and 49 as the internal marks i.e. X has following values:

$$X = (5, 49)$$

Now, we want to classify(Using K-NN),whether this student will Pass or Fail using different values of K.

First, let us calculate the Euclidean distance of the new student $X = (5, 49)$ from each training point tabled above.

We know that the Euclidean distance formula is:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Euclidean distance of $X = (5, 49)$ For student A (1, 40):

$$d_A = \sqrt{(5 - 1)^2 + (49 - 40)^2} = \sqrt{16 + 81} = \sqrt{97} \approx 9.85$$

Euclidean distance of $X = (5, 49)$ For student B (2, 42):

$$d_B = \sqrt{(5 - 2)^2 + (49 - 42)^2} = \sqrt{9 + 49} = \sqrt{58} \approx 7.62$$

Euclidean distance of $X = (5, 49)$ For student C (3, 45):

$$d_C = \sqrt{(5 - 3)^2 + (49 - 45)^2} = \sqrt{4 + 16} = \sqrt{20} \approx 4.47$$

Euclidean distance of $X = (5, 49)$ For student D (4, 48):

$$d_D = \sqrt{(5 - 4)^2 + (49 - 48)^2} = \sqrt{1 + 1} = \sqrt{2} \approx 1.41$$

Euclidean distance of $X = (5, 49)$ For student E (5, 50):

$$d_E = \sqrt{(5 - 5)^2 + (49 - 50)^2} = \sqrt{0 + 1} = 1$$

Euclidean distance of $X = (5, 49)$ For student F (6, 52):

$$d_F = \sqrt{(5 - 6)^2 + (49 - 52)^2} = \sqrt{1 + 9} = \sqrt{10} \approx 3.16$$

Euclidean distance of $X = (5, 49)$ For student G (7, 54):

$$d_G = \sqrt{(5 - 7)^2 + (49 - 54)^2} = \sqrt{4 + 25} = \sqrt{29} \approx 5.39$$

Euclidean distance of $X = (5, 49)$ For student H $(8, 90)$:

$$d_H = \sqrt{(5 - 8)^2 + (49 - 90)^2} = \sqrt{9 + 1681} = \sqrt{1690} \approx 41.11$$

Now the distances in ascending order are:

Student	Distance	Result
E	1.00	Pass
D	1.41	Pass
F	3.16	Pass
C	4.47	Fail
G	5.39	Pass
B	7.62	Fail
A	9.85	Fail
H	41.11	Fail

Now let us see what happens for different values of K .

For $K = 1$, only the nearest neighbor is considered. The nearest neighbor is student E, whose class is **Pass**. Therefore, the predicted class is:

Prediction = Pass

This looks correct, but $K = 1$ can be risky. If the nearest point is a noisy or unusual point, the prediction may become unstable. Thus, a very small K can lead to **overfitting**, where the model becomes too dependent on individual training examples.

For $K = 3$, we consider the three nearest neighbors: E, D, and F. Their classes are:

- E → Pass
- D → Pass
- F → Pass

Since all three are Pass, the majority class is:

Prediction = Pass

This result is also Pass, and it is more reliable than $K = 1$ because it is based on a small group of nearby points rather than a single point.

For $K = 5$, we consider E, D, F, C, and G. Their classes are:

- E → Pass
- D → Pass

- $F \rightarrow \text{Pass}$
- $C \rightarrow \text{Fail}$
- $G \rightarrow \text{Pass}$

Pass appears 4 times and Fail appears 1 time, so:

Prediction = Pass

This is still a good prediction because most nearby students belong to the Pass class.

Now let us consider $K = 7$. The seven nearest neighbors are E, D, F, C, G, B, and A. Their classes are:

- $\text{Pass} \rightarrow E, D, F, G = 4$
- $\text{Fail} \rightarrow C, B, A = 3$

So the predicted class is again:

Prediction = Pass

However, the influence of farther points is increasing. These farther points may not truly represent the local pattern around the new student.

Finally, for $K = 8$, all points are included.

The class counts become:

- $\text{Pass} \rightarrow D, E, F, G = 4$
- $\text{Fail} \rightarrow A, B, C, H = 4$

This creates a tie, and the result becomes ambiguous. In some cases, the algorithm may choose one class based on implementation rules, but clearly the prediction becomes less meaningful. Here, we see that a large K ignores the strong local neighborhood around the new student and starts including distant or irrelevant points such as H, which is very far away. This is an example of **underfitting**, where the model becomes too general and loses important local structure.

The above example shows that choosing the right value of K is very important. A very small K may fit noise and give unstable predictions, while a very large K may smooth out meaningful differences and produce weak predictions. In practice, moderate values such as 3, 5, or 7 are often tested, and the best value is selected using validation techniques.

Choosing the right value of K in KNN is important because it directly affects how well the model performs on new (unseen) data. But the question is—how do we decide which value of K is best?

This is where validation techniques come into play.

Validation techniques help us test different values of K and select the one that gives the best performance, not just on training data but on unseen data as well. The most commonly used method is:

- Train-Test Split or
- Cross-Validation.

First, let us understand the idea using a simple approach called **Train-Test Split**. The dataset is divided into two parts:

- **Training Data (e.g., 70–80%)**: used to build the model
- **Testing Data (e.g., 20–30%)**: used to evaluate performance

Now, we try different values of K (say 1, 3, 5, 7, 9) and check how well each performs on the test data.

Example 1: Suppose we try different values of K and get the following accuracy:

K Value	Accuracy (%)
1	85
3	90
5	92
7	88
9	83

Solution: From this table, we can see that:

- $K = 1$ gives good accuracy but may overfit
- $K = 5$ gives the **highest accuracy (92%)**
- Larger K values start reducing performance

So, we select:

$K = 5$ (best performance)

This method works, but it depends on one split of data.

To make it more reliable, we use a better technique called **Cross-Validation**.

Cross-Validation (More Reliable Method)

In **K-Fold Cross-Validation**, the dataset is divided into K equal parts (called folds). For example, if $K = 5$:

- Split data into 5 parts
- Train the model on 4 parts and test on 1 part
- Repeat this process 5 times (each time using a different test part)
- Take the **average accuracy**

Now, we repeat this entire process for different values of K (neighbors). The results are:

Result :

K (neighbors)	Avg. Accuracy (%)
1	84
3	89

K (neighbors)	Avg. Accuracy (%)
5	91
7	87
9	82

Here again, the highest accuracy is for:

$$K = 5$$

So, cross-validation confirms that $K = 5$ is the best choice.

The interpretation of different values of K in the K-Nearest Neighbors (KNN) algorithm is very important for understanding model behavior.

- When the value of K is very small, such as $K = 1$, the model relies heavily on the nearest data point. This makes the model highly sensitive to noise or outliers in the dataset, which can lead to **overfitting**, where the model performs well on training data but poorly on new data.
- On the other hand, when the value of K is very large, such as $K = 9$, the model considers too many neighboring points. As a result, it may ignore important local patterns in the data and produce overly generalized predictions, leading to **underfitting**.
- A moderate value of K , such as 3, 5, or 7, usually provides a good balance because it considers enough neighbors to make stable predictions while still capturing meaningful patterns. This results in better **generalization** on unseen data.

The key idea behind using validation techniques is to determine which value of K performs best on data that the model has not seen before. Instead of selecting K randomly, a systematic approach is followed:

- Multiple values of K are tested
- The performance of each value is measured using validation methods
- The value that gives the best average accuracy is selected

So, we learned that the validation techniques such as train-test split and cross-validation help in choosing an appropriate value of K . These techniques ensure that the selected K :

- Avoids overfitting (caused by very small K)
- Avoids underfitting (caused by very large K)
- Provides the best predictive performance on unseen data

Thus, validation plays a crucial role in finding the optimal balance, making the KNN model more reliable, stable, and accurate.

Now let us understand why **feature scaling or normalization** is also important in KNN. Suppose instead of the previous dataset, we have two features with very different scales: **Study Hours** ranging from 1 to 8, and **Annual Family Income** ranging from 40,000 to 90,000. If we directly calculate distances, the income feature will dominate because its numerical values are much larger.

For example, consider two students, $P = (5, 40000)$ and $Q = (6, 90000)$

The Euclidean distance is:

$$d = \sqrt{(5 - 6)^2 + (40000 - 90000)^2}$$
$$d = \sqrt{1 + 2500000000} \approx 50000$$

Here, the difference in study hours is only 1, but it becomes negligible compared to the huge income difference. As a result, the algorithm pays almost no attention to study hours, even if that feature is very important for prediction. This is why normalization is needed.

A common normalization method is **Min-Max Normalization**, given by:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

This transforms all feature values into a common scale, usually between 0 and 1. After normalization, both study hours and income contribute more fairly to the distance calculation. This improves the reliability of KNN predictions.

Now let us normalize both features.

For **Study Hours**, the minimum and maximum values are:

$$x_{\min} = 1, \text{ and } x_{\max} = 8$$

So the range is: $8 - 1 = 7$

For **Annual Family Income**, the minimum and maximum values are:

$$x_{\min} = 40000, \quad x_{\max} = 90000$$

So the range is: $90000 - 40000 = 50000$

Now normalize student **P** = (5, 40000).

For study hours:

$$5' = \frac{5 - 1}{8 - 1} = \frac{4}{7} \approx 0.571$$

For income:

$$40000' = \frac{40000 - 40000}{90000 - 40000} = \frac{0}{50000} = 0$$

So the normalized coordinates of **P** are:

$$P' = (0.571, 0)$$

Now normalize student **Q** = (6, 90000).

For study hours:

$$6' = \frac{6 - 1}{8 - 1} = \frac{5}{7} \approx 0.714$$

For income:

$$90000' = \frac{90000 - 40000}{90000 - 40000} = \frac{50000}{50000} = 1$$

So the normalized coordinates of **Q** are:

$$Q' = (0.714, 1)$$

Now calculate Euclidean distance using the normalized values:

$$\begin{aligned} d(P', Q') &= \sqrt{(0.571 - 0.714)^2 + (0 - 1)^2} \\ &= \sqrt{(-0.143)^2 + (-1)^2} \\ &= \sqrt{0.0204 + 1} \\ &= \sqrt{1.0204} \approx 1.01 \end{aligned}$$

Now the distance is around **1.01**, and both features contribute in a balanced way.

The study-hours difference is no longer ignored.

To see why this matters in KNN, let us take a simple classification example.

Assume we want to predict whether a student will **Pass** or **Fail** based on Study Hours and Family Income. Consider the following training data:

Student	Study Hours	Income	Result
A	5	40000	Pass
B	6	90000	Fail
C	4	42000	Pass

Now suppose a new student is:

$$X = (5, 45000)$$

We want to classify this student using KNN.

Classify without normalization:

Distance from **X** to **A**:

$$\begin{aligned} d(X, A) &= \sqrt{(5 - 5)^2 + (45000 - 40000)^2} \\ &= \sqrt{0 + 5000^2} = 5000 \end{aligned}$$

Distance from **X** to **B**:

$$\begin{aligned} d(X, B) &= \sqrt{(5 - 6)^2 + (45000 - 90000)^2} \\ &= \sqrt{1 + 45000^2} \\ &= \sqrt{1 + 2025000000} \approx 45000 \end{aligned}$$

Distance from **X** to **C**:

$$d(X, C) = \sqrt{(5 - 4)^2 + (45000 - 42000)^2}$$

$$= \sqrt{1 + 3000^2}$$

$$= \sqrt{1 + 9000000} \approx 3000$$

So nearest neighbours are dominated by income differences. Study hours have almost no effect.

Now, normalize the data and Classify:

For the new student $X = (5, 45000)$:

Study hours:

$$5' = \frac{5 - 1}{7} = \frac{4}{7} \approx 0.571$$

Income:

$$45000' = \frac{45000 - 40000}{50000} = \frac{5000}{50000} = 0.1$$

So,

$$X' = (0.571, 0.1)$$

For student A = (5, 40000):

$$A' = (0.571, 0)$$

For student B = (6, 90000):

$$B' = (0.714, 1)$$

For student C = (4, 42000):

Study hours:

$$4' = \frac{4 - 1}{7} = \frac{3}{7} \approx 0.429$$

Income:

$$42000' = \frac{42000 - 40000}{50000} = \frac{2000}{50000} = 0.04$$

So,

$$C' = (0.429, 0.04)$$

Now compute normalized distances.

Distance from X' to A' :

$$d(X', A') = \sqrt{(0.571 - 0.571)^2 + (0.1 - 0)^2}$$

$$= \sqrt{0 + 0.01} = 0.1$$

Distance from X' to B' :

$$\begin{aligned}d(X', B') &= \sqrt{(0.571 - 0.714)^2 + (0.1 - 1)^2} \\&= \sqrt{(-0.143)^2 + (-0.9)^2} \\&= \sqrt{0.0204 + 0.81} \\&= \sqrt{0.8304} \approx 0.911\end{aligned}$$

Distance from X' to C' :

$$\begin{aligned}d(X', C') &= \sqrt{(0.571 - 0.429)^2 + (0.1 - 0.04)^2} \\&= \sqrt{(0.142)^2 + (0.06)^2} \\&= \sqrt{0.0202 + 0.0036} \\&= \sqrt{0.0238} \approx 0.154\end{aligned}$$

So after normalization:

- $d(X', A') = 0.1$
- $d(X', C') = 0.154$
- $d(X', B') = 0.911$

Now the nearest students are **A** and **C**, both labeled **Pass**. Hence, KNN will classify the new student as:

Predicted Class = Pass

This example clearly shows why normalization is important in KNN. Without normalization, the large income values dominate the distance formula, and smaller-scale features like study hours are almost ignored. This can give misleading neighbors and wrong predictions. After normalization, both study hours and income are brought to the same scale, so each feature contributes fairly to the distance calculation.

Thus, normalization improves the reliability and fairness of KNN predictions.

In general:

- KNN depends fully on distance
- Distance gets distorted when features have different scales
- Normalization puts all features on a common scale
- This helps KNN identify truly similar neighbors

So, before applying KNN, feature scaling is usually an essential preprocessing step.

The interpretation of these results is straightforward. The value of K controls how local or global the decision is. Small K values focus on very close neighbors and may capture noise, causing overfitting. Large K values include too many neighbors, even distant ones, and may ignore real patterns, causing underfitting. At the same time, because KNN depends entirely

on distance, features must be on comparable scales; otherwise, one feature can unfairly dominate the prediction. Thus, good performance in KNN depends on two important choices: selecting an appropriate value of K and properly normalizing the data.

So, we learned that the KNN algorithm is simple and powerful, but its success depends heavily on careful parameter selection and data preparation. The above numerical example clearly shows that the choice of K affects the balance between overfitting and underfitting, while normalization ensures that all features contribute fairly in the distance calculation. These considerations are essential for building an accurate and meaningful KNN model.

Also, the performance of KNN depends heavily on the choice of K . A small value of K may lead to overfitting, where the model becomes too sensitive to noise. A large value of K may lead to underfitting, where important patterns are ignored. Additionally, KNN is sensitive to the scale of data, so feature normalization is often required.

There are some advantages and limitations to KNN, they are as follows:

KNN has several advantages.

- It is simple to understand and implement,
- requires no training phase, and
- can handle both classification and regression tasks.

However, KNN also has limitations.

- It is computationally expensive for large datasets because it stores all training data and calculates distances for every prediction.
- It is also sensitive to irrelevant features and noisy data.

So, in this section, we learned that the K-Nearest Neighbors algorithm is a powerful yet simple supervised learning technique that relies on similarity and distance. It is widely used in applications such as recommendation systems, pattern recognition, and classification problems. Despite its simplicity, careful selection of parameters and preprocessing steps are essential to achieve accurate and reliable results.

☛ Check Your Progress 3

Q1. What is the K-Nearest Neighbors (KNN) algorithm? Explain its basic principle.

.....
.....

Q2. Explain the working steps of the KNN algorithm.

.....
.....

Q3. What is the role of the parameter K in KNN?

.....
.....

Q4. Why is feature scaling important in KNN?

.....
.....

Q5. What are the advantages and limitations of KNN?

10.3.2 NAÏVE BAYES ALGORITHM

The Naïve Bayes algorithm is a popular and simple supervised learning technique used mainly for classification problems. It is based on probability theory, specifically on **Bayes' Theorem**, and is widely used in applications such as spam detection, sentiment analysis, medical diagnosis, and document classification. The term “naïve” comes from the assumption that all features are **independent of each other**, which is rarely true in real life but works surprisingly well in practice.

In simple terms, Naïve Bayes answers the question: “*Given some features, what is the probability that a data point belongs to a particular class?*” For example, in email filtering, if an email contains words like “offer,” “free,” and “discount,” the algorithm calculates the probability that the email is spam based on these features and assigns the class with the highest probability.

The core of the Naïve Bayes algorithm is based on Bayes' Theorem, which is expressed as:

$$P(C | X) = \frac{P(X | C) \cdot P(C)}{P(X)}$$

where $P(C | X)$ is the **posterior probability** (probability of class C given input X), $P(X | C)$ is the **likelihood** (probability of features given the class), $P(C)$ is the **prior probability** of the class, and $P(X)$ is the probability of the input features. Since $P(X)$ is constant for all classes, the formula is often simplified as:

$$P(C | X) \propto P(X | C) \cdot P(C)$$

Under the naïve assumption of independence, the likelihood can be written as:

$$P(X | C) = P(x_1 | C) \cdot P(x_2 | C) \cdot \dots \cdot P(x_n | C)$$

This simplification makes the computation efficient even for large datasets.

To understand this clearly, consider a real-life example of email classification. Suppose we want to classify emails as **Spam** or **Not Spam** based on the presence of two words: “Free” and “Offer.” The training data is as follows:

Email	Free	Offer	Class
1	Yes	Yes	Spam
2	Yes	No	Spam
3	No	Yes	Not Spam
4	No	No	Not Spam

First, we calculate prior probabilities:

$$P(\text{Spam}) = \frac{2}{4} = 0.5, \quad P(\text{Not Spam}) = \frac{2}{4} = 0.5,$$

Now, calculate likelihoods:

For Spam:

$$P(\text{Free} = \text{Yes} \mid \text{Spam}) = \frac{2}{2} = 1,$$

$$P(\text{Offer} = \text{Yes} \mid \text{Spam}) = \frac{1}{2} = 0.5$$

For Not Spam:

$$P(\text{Free} = \text{Yes} \mid \text{Not Spam}) = \frac{0}{2} = 0,$$

$$P(\text{Offer} = \text{Yes} \mid \text{Not Spam}) = \frac{1}{2} = 0.5$$

Now suppose we receive a new email with features:

$$X = (\text{Free} = \text{Yes}, \text{Offer} = \text{Yes})$$

We calculate probabilities:

For Spam:

$$P(\text{Spam} \mid X) \propto 1 \times 0.5 \times 0.5 = 0.25$$

For Not Spam:

$$P(\text{Not Spam} \mid X) \propto 0 \times 0.5 \times 0.5 = 0$$

Since $P(\text{Spam} \mid X) > P(\text{Not Spam} \mid X)$, the email is classified as:Spam

The interpretation of this result is that the presence of the word “Free” strongly indicates spam, and hence the algorithm assigns the email to the spam category. This example shows how Naïve Bayes uses probabilities to make decisions.

Let's understand the Naïve Bayes Algorithm with one more Numerical example

Example 2: A medical diagnostic system is designed to predict whether a patient is suffering from **Flu** or **No Flu** based on three symptoms: **Fever**, **Cough**, and **Body Ache**. From previously collected patient records, the following training data are available:

Patient	Fever	Cough	Body Ache	Class
1	Yes	Yes	Yes	Flu

Patient	Fever	Cough	Body Ache	Class
2	Yes	Yes	No	Flu
3	Yes	No	Yes	Flu
4	No	Yes	Yes	Flu
5	Yes	Yes	Yes	Flu
6	No	Yes	No	No Flu
7	No	No	Yes	No Flu
8	No	No	No	No Flu
9	Yes	No	No	No Flu
10	No	Yes	No	No Flu

Using the **Naïve Bayes Algorithm**, classify a new patient with the following symptoms:

- **Fever = Yes**
- **Cough = Yes**
- **Body Ache = No**

Also show all steps, equations, and interpret the result.

Solution:

Step 1: Understand the Objective

We need to predict the class of a new patient:

$$X = (\text{Fever} = \text{Yes}, \text{Cough} = \text{Yes}, \text{Body Ache} = \text{No})$$

The two possible classes are:

- $C_1 = \text{Flu}$
- $C_2 = \text{No Flu}$

Naïve Bayes tells us to calculate:

$$P(C_k | X) = \frac{P(X | C_k) P(C_k)}{P(X)}$$

Since $P(X)$ is same for both classes, for comparison we use:

$$P(C_k | X) \propto P(X | C_k) P(C_k)$$

Step 2: Equation Used in Naïve Bayes

For multiple features, Naïve Bayes assumes that all features are conditionally independent given the class. Therefore,

$$P(X | C_k) = P(x_1 | C_k) \cdot P(x_2 | C_k) \cdot P(x_3 | C_k)$$

So the decision rule becomes:

$$P(C_k | X) \propto P(C_k) \cdot P(x_1 | C_k) \cdot P(x_2 | C_k) \cdot P(x_3 | C_k)$$

Step 3: Meaning of Variables

- $P(C_k)$: Prior probability of class C_k

- $P(x_i | C_k)$: Conditional probability of feature x_i given class C_k
- $P(C_k | X)$: Posterior probability of class C_k after observing the symptoms
- X : New observation to be classified
- x_1, x_2, x_3 : Individual features of X

Here,

- x_1 = Fever = Yes
- x_2 = Cough = Yes
- x_3 = Body Ache = No

Step 4: Find Prior Probabilities

From the table:

- Total patients = 10
- Number of Flu cases = 5
- Number of No Flu cases = 5

Therefore,

$$P(\text{Flu}) = \frac{5}{10} = 0.5$$

$$P(\text{No Flu}) = \frac{5}{10} = 0.5$$

Step 5: Find Conditional Probabilities for Class = Flu

Consider only the rows where Class = Flu:

Patient	Fever	Cough	Body Ache	Class
1	Yes	Yes	Yes	Flu
2	Yes	Yes	No	Flu
3	Yes	No	Yes	Flu
4	No	Yes	Yes	Flu
5	Yes	Yes	Yes	Flu

Total Flu cases = 5

Now calculate the required probabilities:

1. Probability of Fever = Yes given Flu

Number of Flu patients having Fever = Yes = 4

$$P(\text{Fever=Yes} | \text{Flu}) = \frac{4}{5} = 0.8$$

2. Probability of Cough = Yes given Flu

Number of Flu patients having Cough = Yes = 4

$$P(\text{Cough=Yes} | \text{Flu}) = \frac{4}{5} = 0.8$$

3. Probability of Body Ache = No given Flu

Number of Flu patients having Body Ache = No = 1

$$P(\text{Body Ache=No} \mid \text{Flu}) = \frac{1}{5} = 0.2$$

Step 6: Find Conditional Probabilities for Class = No Flu

Consider only the rows where Class = No Flu:

Patient	Fever	Cough	Body Ache	Class
6	No	Yes	No	No Flu
7	No	No	Yes	No Flu
8	No	No	No	No Flu
9	Yes	No	No	No Flu
10	No	Yes	No	No Flu

Total No Flu cases = 5

Now calculate the required probabilities:

1. Probability of Fever = Yes given No Flu

Number of No Flu patients having Fever = Yes = 1

$$P(\text{Fever=Yes} \mid \text{No Flu}) = \frac{1}{5} = 0.2$$

2. Probability of Cough = Yes given No Flu

Number of No Flu patients having Cough = Yes = 2

$$P(\text{Cough=Yes} \mid \text{No Flu}) = \frac{2}{5} = 0.4$$

3. Probability of Body Ache = No given No Flu

Number of No Flu patients having Body Ache = No = 4

$$P(\text{Body Ache=No} \mid \text{No Flu}) = \frac{4}{5} = 0.8$$

Step 7: Compute Posterior Score for Each Class

We now calculate the Naïve Bayes score for each class.

For Flu

$$P(\text{Flu} \mid X) \propto P(\text{Flu}) \cdot P(\text{Fever=Yes} \mid \text{Flu}) \cdot P(\text{Cough=Yes} \mid \text{Flu}) \cdot P(\text{Body Ache=No} \mid \text{Flu})$$

$$P(\text{Flu} \mid X) \propto 0.5 \times 0.8 \times 0.8 \times 0.2$$

$$P(\text{Flu} \mid X) \propto 0.064$$

For No Flu

$$P(\text{No Flu} | X) \propto P(\text{No Flu}) \cdot P(\text{Fever}=\text{Yes} | \text{No Flu}) \cdot P(\text{Cough}=\text{Yes} | \text{No Flu}) \cdot P(\text{Body Ache}=\text{No} | \text{No Flu})$$

$$P(\text{No Flu} | X) \propto 0.5 \times 0.2 \times 0.4 \times 0.8$$

$$P(\text{No Flu} | X) \propto 0.032$$

Step 8: Compare the Results

- Score for Flu = **0.064**
- Score for No Flu = **0.032**

Since,

$$0.064 > 0.032$$

therefore, the new patient is classified as: **Flu**

That means, a patient with symptoms **Fever = Yes, Cough = Yes, Body Ache = No** is classified as: **Flu**

The Interpretation of the result means that based on the available training data, the combination of symptoms shown by the new patient is **more likely to occur in patients having Flu than in patients having No Flu**. Even though the symptom **Body Ache = No** is less common among Flu patients, the other two symptoms, **Fever = Yes** and **Cough = Yes**, strongly support the Flu class. Hence, after multiplying all probabilities, the posterior score of Flu becomes higher than that of No Flu.

In practical terms, Naïve Bayes does not say with absolute certainty that the patient has Flu. Rather, it says that **among the available classes, Flu is the more probable class** for this symptom pattern.

Short Summarized Calculation of above Numerical:

In exam, you can write the answer in this compact form:

$$P(C_k | X) \propto P(C_k) \prod P(x_i | C_k)$$

For Flu:

$$0.5 \times \frac{4}{5} \times \frac{4}{5} \times \frac{1}{5} = 0.064$$

For No Flu:

$$0.5 \times \frac{1}{5} \times \frac{2}{5} \times \frac{4}{5} = 0.032$$

Since $0.064 > 0.032$, the patient belongs to class **Flu**.

10.3.2.1 TYPES OF NAÏVE BAYES ALGORITHM

Now we extend our discussion to explore further details on Naïve Bayes algorithm, In practical scenarios, Naïve Bayes is often implemented in different forms such as:

- Gaussian Naïve Bayes (for continuous data),
- Multinomial Naïve Bayes (for text data), and
- Bernoulli Naïve Bayes (for binary features).

One common issue is the occurrence of zero probability, which can be handled using techniques like **Laplace smoothing**, where a small value is added to all probabilities to avoid multiplication by zero.

A brief introduction to each of the Naïve Bayes algorithm, which are implemented as different variants, depending on the nature of data, such as Gaussian, Multinomial, and Bernoulli Naïve Bayes is as follows:

1. Gaussian Naïve Bayes (for Continuous Data) : Gaussian Naïve Bayes is used when the input features are continuous in nature, such as height, weight, marks, or income. It assumes that the values of these features follow a normal (Gaussian) distribution. Instead of counting frequencies, this method uses the mean and variance of the data to estimate probabilities. The probability of a feature value given a class is computed using the Gaussian distribution formula:

$$P(x | C_k) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{(x-\mu)^2}{2\sigma^2} \right)$$

Where:

- μ = mean of the feature
- σ^2 = variance
- x = observed value

For example, consider a classification problem where we want to determine whether a person is Fit or Unfit based on their weight. Suppose the average weight of Fit individuals is 65 kg with a variance of 25, and for Unfit individuals, the average weight is 85 kg with a variance of 36. If a new person has a weight of 70 kg, we substitute this value into the Gaussian formula for both classes. The class for which the probability value is higher is selected as the predicted class. This approach essentially checks which distribution curve the given value fits better.

2. Multinomial Naïve Bayes (for Text Data): Multinomial Naïve Bayes is mainly used for text-based applications where features represent the frequency or count of words. It is widely applied in spam detection, sentiment analysis, and document classification. In this method, the probability is calculated based on how often a word appears in a particular class compared to the total number of words in that class.

For instance, consider the problem of classifying emails as Spam or Not Spam. Suppose in Spam emails, the word “Offer” appears 50 times out of a total of 100 words, giving a probability of 0.5. In contrast, in Not Spam emails, the same word appears only 5 times out of 100 words, resulting in a probability of 0.05. Now, if a new email contains the word “Offer” multiple times, the probability of it being Spam increases significantly. Thus, words that frequently occur in a class strongly influence the classification decision.

3. Bernoulli Naïve Bayes (for Binary Features): Bernoulli Naïve Bayes is used when the features are binary, meaning they can take only two values: 0 or 1, representing absence or presence of a feature. Unlike Multinomial Naïve Bayes, it does not consider how many times a feature appears but only whether it appears or not.

For example, consider a sentiment analysis problem where we classify a review as Positive or Negative based on the presence of certain words. If the word “Good” appears in a review, it is represented as 1; otherwise, it is 0. Suppose the probability of the word “Good” appearing in Positive reviews is 0.8, while in Negative reviews it is only 0.2. If a new review contains the word “Good,” the model will assign a higher probability to the Positive class. Thus, the presence or absence of specific features plays a crucial role in classification.

Each variant handles features differently, making the algorithm flexible and widely applicable. Another important practical issue is the **zero-probability problem**, which is addressed using **Laplace smoothing**, a brief of Laplace smoothing is as follows:

Laplace Smoothing (Handling Zero Probability): In practical applications of Naïve Bayes, a common issue arises when a feature value does not appear in the training data for a particular class. In such cases, the conditional probability becomes zero. Since Naïve Bayes multiplies probabilities, even a single zero value makes the entire probability zero, leading to incorrect predictions.

To overcome this problem, Laplace smoothing is applied. It modifies the probability calculation by adding a small constant (usually 1) to each count:

$$P(x_i | C_k) = \frac{\text{Count} + 1}{\text{Total} + n}$$

where Count is the frequency of the feature, Total is the total number of observations in the class, and n is the number of possible feature values.

For example, suppose in Spam emails the word “Lottery” has never appeared. Without smoothing, its probability would be zero, which would incorrectly eliminate the Spam class if this word appears in a new email. Using Laplace smoothing, the probability becomes:

$$P = \frac{0 + 1}{100 + 2} = \frac{1}{102}$$

This ensures that no probability becomes zero and the model can handle unseen data more effectively.

Now, we are in position to conclude this section, and prior to this we need to address the advantages and limitations of Naïve Bayes algorithm, they are as follows:

The Naïve Bayes algorithm offers several advantages.

- It is simple to understand and easy to implement.
- It is computationally efficient and works well even with large datasets.
- It performs particularly well in high-dimensional problems such as text classification.
- It also requires relatively less training data compared to many other algorithms.

However, it also has some limitations.

- The assumption of feature independence is often unrealistic, which can affect accuracy in some cases.
- It may not perform well when features are highly correlated.
- Additionally, if the dataset is very small or biased, the probability estimates may not be reliable.

This section concludes with the understanding that the Naïve Bayes algorithm is a powerful yet simple probabilistic classifier that works effectively in many real-world applications. Despite its simplifying assumptions, it provides fast and reasonably accurate predictions, making it a valuable tool in supervised learning. Apart from this, it is important to understand that the choice of Naïve Bayes variant depends on the type of data being used. Gaussian Naïve Bayes is suitable for continuous numerical data, Multinomial Naïve Bayes is ideal for frequency-based data such as text, and Bernoulli Naïve Bayes is appropriate for binary features. Laplace smoothing acts as a practical enhancement that improves model robustness by preventing zero probability issues. Together, these variations make Naïve Bayes a simple yet powerful algorithm applicable across a wide range of real-world problems.

☛ Check Your Progress 4:

Q1. What is the Naïve Bayes algorithm? Why is it called “naïve”?

.....
.....

Q2. State Bayes’ Theorem and explain its components.

.....
.....

Q3. Explain the working of the Naïve Bayes classifier.

.....
.....

Q4. What are the advantages of the Naïve Bayes algorithm?

.....
.....

Q5. What are the limitations of the Naïve Bayes algorithm?

10.3.3 LOGISTIC REGRESSION

Logistic Regression is one of the most fundamental and widely used supervised learning algorithms for solving **classification problems**, especially when the output is binary in nature (such as Yes/No, 0/1, True/False). Despite its name, it is not a regression algorithm in the traditional sense; instead, it is used to estimate the probability that a given input belongs to a particular class.

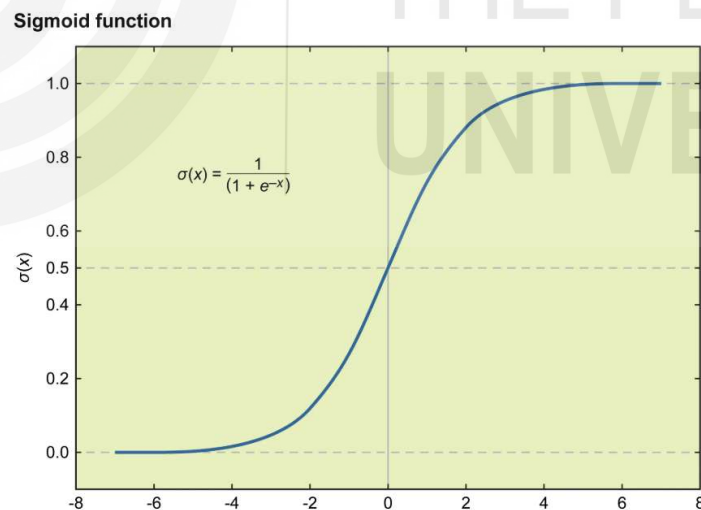
At its core, Logistic Regression builds a relationship between the input features and the probability of an outcome using a mathematical function called the **Sigmoid (Logistic) Function**.

Important Note: The **Sigmoid Function** is a mathematical function used in Logistic Regression to convert any real-valued input into a probability value between **0 and 1**.

Mathematical Expression for Sigmoid Function is: $\sigma(z) = \frac{1}{1+e^{-z}}$

Where:

- z = linear combination of inputs (e.g., $\beta_0 + \beta_1 x$)
- e = Euler's number (≈ 2.718)
- $\sigma(z)$ = output probability between 0 and 1



The sigmoid curve shown above is an S-shaped smooth curve that plays a crucial role in transforming values in Logistic Regression. It maps any real-valued input z into a range between 0 and 1, making it suitable for probability interpretation. When the value of z is very large and negative, the output of the sigmoid function approaches 0, indicating a very low probability. When $z = 0$, the sigmoid function gives an output of 0.5, which represents an uncertain or neutral state. As the value of z becomes large

and positive, the output of the sigmoid function approaches 1, indicating a very high probability.

Mathematically, this behaviour can be observed as $\sigma(0) = 0.5$, $\sigma(z) \rightarrow 1$ as $z \rightarrow +\infty$, and $\sigma(z) \rightarrow 0$ as $z \rightarrow -\infty$.

The sigmoid function can be understood as a probability converter that takes any input value, such as marks, income, or a score, and converts it into a probability between 0 and 1. For example, when $z = -2$, the sigmoid value is approximately 0.12, which indicates a low probability. When $z = 0$, the output is exactly 0.5, representing uncertainty. When $z = 2$, the sigmoid value becomes approximately 0.88, indicating a high probability. This smooth transition from low to high values allows the model to make gradual and interpretable predictions.

In the context of Logistic Regression, the sigmoid function plays a central role by converting the linear combination of input features into a probability. This probability is then used to classify observations based on a threshold value, typically 0.5. Additionally, the sigmoid function is smooth and differentiable, which makes it suitable for optimization techniques used during model training. Overall, the sigmoid function acts as a bridge between linear models and probability theory, ensuring that predictions remain meaningful, interpretable, and bounded within a valid range, thereby making Logistic Regression effective for classification problems.

Unlike linear regression, which can produce any real-valued output, Logistic Regression ensures that the predicted values lie between 0 and 1, making them interpretable as probabilities.

The mathematical expression of Logistic Regression is given by:

$$P(Y = 1 | X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)}}$$

Where:

- $P(Y = 1 | X)$: Probability that the output belongs to class 1
- e : Euler's number (≈ 2.718)
- β_0 : Intercept (bias term)
- $\beta_1, \beta_2, \dots, \beta_n$: Model coefficients
- $x_1, x_2, \dots, x_n$: Input features

The expression inside the exponent is a linear combination of inputs, but the sigmoid function transforms it into a probability between 0 and 1.

The sigmoid function produces an S-shaped curve. When the linear combination of inputs is very large, the output approaches 1; when it is very small, the output approaches 0. A threshold (commonly 0.5) is used to convert probabilities into class labels.

For example,

- If $P \geq 0.5$, classify as 1

- If $P < 0.5$, classify as 0

This makes Logistic Regression very useful for decision-making problems.

If we talk about a real-life example, consider a bank that wants to predict whether a customer will **default on a loan** based on their income.

- Input: Income
- Output: Default (1) or No Default (0)

Suppose the model learns the relationship such that lower income increases the probability of default. Logistic Regression will output a probability (e.g., 0.75), which indicates a high chance of default.

In terms of the numerical example, let us consider a simple Logistic Regression model:

- Intercept (β_0) = -4
- Coefficient (β_1) = 0.05
- Feature: Income (in thousands)

We want to predict the probability of default for a person earning ₹60,000.

Solution:

Step 1: Compute Linear Combination

$$z = \beta_0 + \beta_1 x$$

$$z = -4 + (0.05 \times 60) = -4 + 3 = -1$$

Step 2: Apply Sigmoid Function

$$P = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^1}$$

$$P = \frac{1}{1 + 2.718} \approx 0.268$$

Step 3: Classification

Since $P = 0.268 < 0.5$, the prediction is: No Default

The model predicts that the probability of default is approximately **26.8%**, which is relatively low. Therefore, the customer is classified as **non-risky**. This shows how Logistic Regression not only provides a class label but also gives a meaningful probability that can assist in decision-making.

Let's understand the Logistic Regression algorithm in more detail through a Numerical

Example 4: A bank wants to predict whether a customer will **purchase a term deposit** or **not purchase it** based on the customer's **monthly income**. A Logistic Regression model has already been trained, and the obtained model is:

$$z = \beta_0 + \beta_1 x$$

where,

$$z = -6 + 0.08x$$

Here, x represents monthly income in thousand rupees.

Using this Logistic Regression model, predict whether a customer with monthly income of ₹90,000 will purchase the term deposit or not. Use the classification threshold as 0.5. Also compute the probability and interpret the result.

Solution:

We know that formula for Logistic Regression gives the probability of belonging to class 1 as:

$$P(Y = 1 | X) = \frac{1}{1 + e^{-z}}$$

where,

$$z = \beta_0 + \beta_1 x$$

Meaning of Variables

- Y : output class
- $Y = 1$: customer will purchase the term deposit
- $Y = 0$: customer will not purchase the term deposit
- X : input feature
- x : value of the input feature (monthly income)
- β_0 : intercept or bias term
- β_1 : coefficient of income
- z : linear combination of input and coefficients
- $P(Y = 1 | X)$: probability that the customer will purchase the term deposit

Given Data

- Intercept: $\beta_0 = -6$
- Coefficient of income: $\beta_1 = 0.08$
- Income of customer: $x = 90$ thousand rupees

So, the model is:

$$z = -6 + 0.08x$$

Step 1: Calculate the Linear Value z

Substitute $x = 90$ into the equation:

$$z = -6 + 0.08(90)$$

$$z = -6 + 7.2$$

$$z = 1.2$$

Step 2: Apply the Sigmoid Function

Now use the Logistic Regression equation:

$$P(Y = 1 | X) = \frac{1}{1 + e^{-z}}$$

Substitute $z = 1.2$:

$$P(Y = 1 | X) = \frac{1}{1 + e^{-1.2}}$$

We know that:

$$e^{-1.2} \approx 0.3010$$

Therefore,

$$P(Y = 1 | X) = \frac{1}{1 + 0.3010}$$

$$P(Y = 1 | X) = \frac{1}{1.3010}$$

$$P(Y = 1 | X) \approx 0.7686$$

So,

$$P(Y = 1 | X) \approx 0.769$$

Step 3: Apply Decision Rule

Decision rule in Logistic Regression:

- If $P(Y = 1 | X) \geq 0.5$, classify as **1**
- If $P(Y = 1 | X) < 0.5$, classify as **0**

Here,

$$0.769 > 0.5$$

So the predicted class is: $Y = 1$

Final Answer

The probability that the customer will purchase the term deposit is: 0.769 or 76.9%

Hence, the customer is classified as: *Will Purchase the Tem Deposit*

The interpretation of the results obtained from above numerical reflects that the Logistic Regression model predicts that the customer has approximately **76.9% probability** of purchasing the term deposit. Since this probability is greater than the threshold value of 0.5, the customer is placed in the “purchase” category. This means that based on the trained model, a customer with a monthly income of ₹90,000 is more likely to purchase the term deposit than not purchase it.

In practical terms, this probability-based result is useful because it does not only give a final class label, but also shows the strength of prediction. A value of 76.9% indicates fairly strong confidence in the positive outcome.

Stepwise summary of the steps performed in above numerical

Given: $z = -6 + 0.08x, x = 90$

Step 1: $z = -6 + 0.08(90) = 1.2$

Step 2: $P(Y = 1 | X) = \frac{1}{1 + e^{-1.2}} = \frac{1}{1 + 0.3010} = \frac{1}{1.3010} \approx 0.769$

Step 3: Since $0.769 > 0.5$

the customer belongs to class 1. i.e. Customer will purchase the Term deposit.

Important Note:

In Logistic Regression numerical, always follow the steps, given below:

- First, calculate the linear expression: $z = \beta_0 + \beta_1 x$
- Second, convert this value into probability using the sigmoid function:

$$P(Y = 1 | X) = \frac{1}{1 + e^{-z}}$$

- Third, compare the obtained probability with the threshold, usually 0.5.
- If the probability is greater than or equal to 0.5, classify the observation into class 1. Otherwise, classify it into class 0.

So, Logistic Regression works in two stages: it first forms a linear score, and then converts that score into a probability for classification.

This type of numerical is commonly used in banking, healthcare, and marketing. For example, instead of income, the feature could be exam score, age, blood pressure, or advertisement clicks.

Logistic Regression helps convert such input values into probabilities and then makes a classification decision.

The advantages and the Limitations of the Logistic Regression are as follows:

Advantages of Logistic Regression

- Simple and easy to understand and implement
- Provides probabilistic output (useful for decision-making)
- Works well for linearly separable data
- Computationally efficient
- Less prone to overfitting when regularization is used

Limitations of Logistic Regression

- Assumes a linear relationship between features and log-odds
- Not suitable for complex non-linear data
- Sensitive to outliers
- Performance decreases when features are highly correlated (multicollinearity)
- Requires feature scaling for better performance

From above discussion we learned that, Logistic Regression can be understood as a bridge between linear models and probability theory. It takes a linear equation and transforms it into a probability using the sigmoid function, making it highly interpretable and practical. **The key idea is not just predicting a class but understanding the likelihood of an outcome.** In real-world scenarios such as medical diagnosis, credit scoring, and marketing predictions, this probabilistic interpretation makes Logistic Regression a powerful and reliable tool.

☛ Check Your Progress 5:

Q1. What is Logistic Regression? How is it different from Linear Regression?

.....

.....
Q2. Explain the sigmoid (logistic) function used in Logistic Regression.
.....

.....
Q3. How does Logistic Regression perform classification?
.....

.....
Q4. What are the advantages of Logistic Regression?
.....

.....
Q5. What are the limitations of Logistic Regression?
.....
.....

10.3.4 DECISION TREES

Decision Trees are one of the most intuitive and widely used supervised learning algorithms for both classification and regression problems. The core idea of a Decision Tree is to break down a complex decision-making process into a series of simple, sequential decisions, represented in the form of a tree-like structure. Each internal node of the tree represents a test on a feature, each branch represents the outcome of that test, and each leaf node represents the final prediction or class label. This structure closely resembles human decision-making, making Decision Trees easy to understand and interpret.

In a typical Decision Tree, the algorithm starts from the root node and recursively splits the dataset into smaller subsets based on the feature that best separates the data. The goal is to create pure subsets, where most of the data points belong to a single class. To determine the best feature for splitting, the algorithm uses measures such as **Entropy, Information Gain, or Gini Index**.

The concept of Entropy is used to measure the impurity or randomness in the dataset. It is defined as:

$$Entropy(S) = -\sum p_i \log_2 p_i$$

where p_i represents the probability of each class in the dataset.

If all data points belong to one class, entropy is 0 (pure), whereas if the data is evenly distributed among classes, entropy is maximum (impure).

Information Gain is used to decide which feature should be selected for splitting. It is calculated as:

$$IG(S, A) = Entropy(S) - \sum \frac{|S_v|}{|S|} Entropy(S_v)$$

where S is the dataset, A is the feature, and S_v are the subsets formed after splitting.

Just for example, let's consider a scenario where a bank wants to decide whether to approve a loan based on features such as Income (High/Low) and Credit Score (Good/Poor). A Decision Tree will split the data step by step, for example:

- First split: Credit Score
- If Credit Score = Good → Approve Loan
- If Credit Score = Poor → Check Income

- Income = High → Approve
- Income = Low → Reject

This hierarchical decision-making mimics real-world reasoning and provides clear decision rules.

Example5: Calculate Entropy and Information Gain for Income as a feature in a dataset of 10 customers given below:

Class	Count
Yes (Buy)	6
No (Not Buy)	4

Solution:

Step 1: Calculate Entropy of Dataset

$$Entropy(S) = -\left(\frac{6}{10} \log_2 \frac{6}{10} + \frac{4}{10} \log_2 \frac{4}{10}\right)$$

$$Entropy(S) = -(0.6 \log_2 0.6 + 0.4 \log_2 0.4)$$

$$Entropy(S) = -(0.6 \times -0.737 + 0.4 \times -1.322)$$

$$Entropy(S) = 0.971$$

Step 2: Consider a Feature (e.g., Income)

Suppose:

Income	Yes	No	Total
High	4	1	5
Low	2	3	5

Entropy for High Income

$$Entropy(High) = -\left(\frac{4}{5} \log_2 \frac{4}{5} + \frac{1}{5} \log_2 \frac{1}{5}\right) = 0.722$$

Entropy for Low Income

$$Entropy(Low) = -\left(\frac{2}{5} \log_2 \frac{2}{5} + \frac{3}{5} \log_2 \frac{3}{5}\right) = 0.971$$

Step 3: Calculate Information Gain

$$IG(S, Income) = 0.971 - \left(\frac{5}{10} \times 0.722 + \frac{5}{10} \times 0.971\right)$$

$$IG = 0.971 - (0.361 + 0.485)$$

$$IG = 0.125$$

Step 4: Decision: The feature with the highest Information Gain is selected for splitting. If Income gives the highest gain among all features, it becomes the root node.

It is observed in the above numerical that the Information Gain value of 0.125 indicates that splitting the dataset based on Income reduces uncertainty in the dataset. Although the reduction is not very large, it still improves classification. The Decision Tree algorithm will compare this gain with other features and choose the one that maximizes purity at each step.

Let's understand the Decision Tree Algorithm with one more Comprehensive Numerical citing the use on entropy and information gain only.

Example6: A company wants to predict whether a customer will **Buy a Product** or **Not Buy a Product** based on the following attributes:

- **Age:** Young, Middle, Old
- **Income:** High, Medium, Low
- **Student:** Yes, No

The training dataset is given below:

Record	Age	Income	Student	Buy Product
1	Young	High	No	No
2	Young	High	No	No
3	Middle	High	No	Yes
4	Old	Medium	No	Yes
5	Old	Low	Yes	Yes
6	Old	Low	Yes	No
7	Middle	Low	Yes	Yes
8	Young	Medium	No	No
9	Young	Low	Yes	Yes
10	Old	Medium	Yes	Yes

Using the **Decision Tree algorithm with Entropy and Information Gain**, determine the **best attribute for the root node** and continue the splitting process until the class decision is obtained. Show all calculations step by step and write the final decision rules.

Solution: In this problem, we have to construct a Decision Tree for classification. The target variable is:

$$Y = \text{Buy Product}$$

where:

- **Yes** = Customer buys the product
- **No** = Customer does not buy the product

The Decision Tree chooses the attribute that gives the **highest Information Gain** at each stage.

The used Equations are:

(a) Entropy

Entropy measures the impurity or randomness in the dataset.

$$Entropy(S) = - \sum_{i=1}^n p_i \log_2 p_i$$

For binary classification:

$$Entropy(S) = -p_{yes} \log_2(p_{yes}) - p_{no} \log_2(p_{no})$$

(b) Information Gain

Information Gain tells us how much uncertainty is reduced after splitting on an attribute.

$$IG(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

Where,

- S : Entire dataset
- A : Attribute selected for splitting
- S_v : Subset of dataset for a particular value v of attribute A
- p_{yes} : Proportion of Yes class
- p_{no} : Proportion of No class
- $Entropy(S)$: Impurity of dataset S
- $IG(S, A)$: Information gain obtained by splitting on attribute A

To solve the given problem the steps to be performed are as follows

Step 1: Calculate Entropy of the Full Dataset:

From the table: Total records = 10 ; Yes = 6 and No = 4

So,

$$p_{yes} = \frac{6}{10} = 0.6$$

$$p_{no} = \frac{4}{10} = 0.4$$

Now entropy of the whole dataset:

$$Entropy(S) = -0.6 \log_2(0.6) - 0.4 \log_2(0.4)$$

Using:

$$\log_2(0.6) \approx -0.737$$

$$\log_2(0.4) \approx -1.322$$

So,

$$Entropy(S) = -0.6(-0.737) - 0.4(-1.322)$$

$$Entropy(S) = 0.4422 + 0.5288$$

$$Entropy(S) = 0.971$$

This means the dataset is impure and needs splitting.

Step 2: Calculate Information Gain for Attribute = Age

In the Given table, the attribute Age has three values: Young; Middle; Old

(a) Subset for Age = Young

Refer to Given Table Records: 1, 2, 8, 9

Refer to Given Table Class labels(Buy Product): No, No, No, Yes

- Yes = 1
- No = 3

$$Entropy(Young) = -\frac{1}{4} \log_2 \frac{1}{4} - \frac{3}{4} \log_2 \frac{3}{4}$$

$$Entropy(Young) = -0.25(-2) - 0.75(-0.415)$$

$$Entropy(Young) = 0.5 + 0.311$$

$$Entropy(Young) = 0.811$$

(b) Subset for Age = Middle

Refer to Given Table Records: 3, 7

Refer to Given Table Class labels(Buy Product): Yes, Yes

- Yes = 2
- No = 0

$$Entropy(Middle) = 0$$

(c) Subset for Age = Old

Refer to Given Table Records: 4, 5, 6, 10

Refer to Given Table Class labels(Buy Product): Yes, Yes, No, Yes

- Yes = 3
- No = 1

$$\begin{aligned} Entropy(Old) &= -\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} \\ Entropy(Old) &= -0.75(-0.415) - 0.25(-2) \\ Entropy(Old) &= 0.311 + 0.5 \\ Entropy(Old) &= 0.811 \end{aligned}$$

(d) Weighted Entropy for Age

$$\begin{aligned} Entropy(S, Age) &= \frac{4}{10} (0.811) + \frac{2}{10} (0) + \frac{4}{10} (0.811) \\ Entropy(S, Age) &= 0.3244 + 0 + 0.3244 \\ Entropy(S, Age) &= 0.6488 \end{aligned}$$

(e) Information Gain for Age

$$\begin{aligned} IG(S, Age) &= 0.971 - 0.6488 \\ IG(S, Age) &= 0.3222 \end{aligned}$$

Result: $IG(S, Age) = 0.3222$

Step 3: Calculate Information Gain for Attribute = Income

Income values: High, Medium, and Low

(a) Subset for Income = High

Refer to Given Table Records: 1, 2, 3

Refer to Given Table Class labels(Buy Product): No, No, Yes

- Yes = 1
- No = 2

$$Entropy(High) = -\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3}$$

Using:

$$\begin{aligned} \log_2(1/3) &\approx -1.585, \log_2(2/3) \approx -0.585 \\ Entropy(High) &= -\frac{1}{3}(-1.585) - \frac{2}{3}(-0.585) \\ Entropy(High) &= 0.528 + 0.390 \\ Entropy(High) &= 0.918 \end{aligned}$$

(b) Subset for Income = Medium

Refer to Given Table Records: 4, 8, 10

Refer to Given Table Class labels(Buy Product): Yes, No, Yes

- Yes = 2
- No = 1

$$Entropy(Medium) = 0.918$$

(c) Subset for Income = Low

Refer to Given Table Records: 5, 6, 7, 9

Refer to Given Table Class labels(Buy Product): Yes, No, Yes, Yes

- Yes = 3
- No = 1

$$Entropy(Low) = 0.811$$

(d) Weighted Entropy for Income

$$\begin{aligned}
 Entropy(S, Income) &= \frac{3}{10}(0.918) + \frac{3}{10}(0.918) + \frac{4}{10}(0.811) \\
 Entropy(S, Income) &= 0.2754 + 0.2754 + 0.3244 \\
 Entropy(S, Income) &= 0.8752
 \end{aligned}$$

(e) Information Gain for Income

$$IG(S, Income) = 0.971 - 0.8752$$

$$IG(S, Income) = 0.0958$$

$$\text{Result: } IG(S, Income) = 0.0958$$

Step 4: Calculate Information Gain for Attribute = Student

Student values: Yes, No

(a) Subset for Student = Yes

Refer to Given Table Records: 5, 6, 7, 9, 10

Refer to Given Table Class labels(Buy Product): Yes, No, Yes, Yes, Yes

- Yes = 4
- No = 1

$$\begin{aligned}
 Entropy(Yes) &= -\frac{4}{5} \log_2 \frac{4}{5} - \frac{1}{5} \log_2 \frac{1}{5} \\
 Entropy(Yes) &= -0.8(-0.322) - 0.2(-2.322) \\
 Entropy(Yes) &= 0.258 + 0.464 \\
 Entropy(Yes) &= 0.722
 \end{aligned}$$

(b) Subset for Student = No

Refer to Given Table Records: 1, 2, 3, 4, 8

Refer to Given Table Class labels(Buy Product): No, No, Yes, Yes, No

- Yes = 2
- No = 3

$$\begin{aligned}
 Entropy(No) &= -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} \\
 Entropy(No) &= 0.971
 \end{aligned}$$

(c) Weighted Entropy for Student

$$\begin{aligned}
 Entropy(S, Student) &= \frac{5}{10}(0.722) + \frac{5}{10}(0.971) \\
 Entropy(S, Student) &= 0.361 + 0.4855 \\
 Entropy(S, Student) &= 0.8465
 \end{aligned}$$

(d) Information Gain for Student

$$IG(S, Student) = 0.971 - 0.8465$$

$$IG(S, Student) = 0.1245$$

$$\text{Result: } IG(S, Student) = 0.1245$$

Step 5: Select the Root Node

Now compare all Information Gain values:

- $IG(S, Age) = 0.322$
- $IG(S, Income) = 0.096$
- $IG(S, Student) = 0.125$

The highest Information Gain is for **Age**.

Therefore, the root node is: Age

Step 6: Continue Splitting the Branches

We now split according to Age.

Branch 1: Age = Middle

Records: 3, 7

Both are **Yes**

So this is a pure node. Is Age = Middle → Buy Product = Yes

Branch 2: Age = Young

Subset:

Record	Age	Income	Student	Buy Product
1	Young	High	No	No
2	Young	High	No	No
8	Young	Medium	No	No
9	Young	Low	Yes	Yes

Class counts:

- Yes = 1
- No = 3

$$\text{Entropy(Young)} = 0.811$$

Now among remaining attributes, test **Income** and **Student**.

(a) Split Young subset by Student

Student = No

Records: 1, 2, 8

Class labels: No, No, No

$$\text{Entropy} = 0$$

Student = Yes

Record: 9

Class label: Yes

$$\text{Entropy} = 0$$

Weighted entropy:

$$\text{Entropy(Young, Student)} = \frac{3}{4}(0) + \frac{1}{4}(0) = 0$$

Information Gain:

$$IG(\text{Young, Student}) = 0.811 - 0 = 0.811$$

(b) Split Young subset by Income

Income = High

Records: 1, 2

Both No → Entropy = 0

Income = Medium

Record: 8

No → Entropy = 0

Income = Low

Record: 9

Yes → Entropy = 0

$$\text{Weighted entropy: } \text{Entropy}(\text{Young}, \text{Income}) = 0$$

$$\text{Information Gain: } \text{IG}(\text{Young}, \text{Income}) = 0.811 - 0 = 0.811$$

Both Student and Income perfectly classify this subset. We can choose either. Usually, one may choose **Student** because it gives a simpler rule here.

So:

- Age = Young and Student = No $\rightarrow$ No
- Age = Young and Student = Yes $\rightarrow$ Yes

Branch 3: Age = Old

Subset:

Record Age Income Student Buy Product

4	Old	Medium	No	Yes
5	Old	Low	Yes	Yes
6	Old	Low	Yes	No
10	Old	Medium	Yes	Yes

Class counts:

- Yes = 3
- No = 1

$$\text{Entropy}(\text{Old}) = 0.811$$

Now consider remaining attributes: **Income** and **Student**

(a) Split Old subset by Income

Income = Medium

Records: 4, 10

Class labels: Yes, Yes

$$\text{Entropy} = 0$$

Income = Low

Records: 5, 6

Class labels: Yes, No

$$\text{Entropy} = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = 1$$

Weighted entropy:

$$\text{Entropy}(\text{Old}, \text{Income}) = \frac{2}{4}(0) + \frac{2}{4}(1) = 0.5$$

Information Gain:

$$\text{IG}(\text{Old}, \text{Income}) = 0.811 - 0.5 = 0.311$$

(b) Split Old subset by Student

Student = Yes

Records: 5, 6, 10

Class labels: Yes, No, Yes

- Yes = 2
- No = 1

$$\text{Entropy} = 0.918$$

Student = No

Record: 4

Class label: Yes

$$\text{Entropy} = 0$$

Weighted entropy:

$$Entropy(Old, Student) = \frac{3}{4}(0.918) + \frac{1}{4}(0)$$

$$Entropy(Old, Student) = 0.6885$$

Information Gain:

$$IG(Old, Student) = 0.811 - 0.6885 = 0.1225$$

So the better split is **Income**.

Thus:

- Age = Old and Income = Medium → Yes
- Age = Old and Income = Low → still mixed, so split again

Step 7: Final Split for Age = Old and Income = Low

Subset:

Record	Age	Income	Student	Buy Product
5	Old	Low	Yes	Yes
6	Old	Low	Yes	No

Here all remaining records have Student = Yes, so there is no useful further split with the available attributes. In exam problems, this situation is often handled by assigning the **majority class** of the parent node or noting that data is insufficient to split further uniquely. Since in the Old branch overall majority class is **Yes**, we assign:

Age = Old and Income = Low → Buy Product = Yes (majority)

Final Decision Rules

The Decision Tree can be written as rules:

- Rule 1: If Age = Middle, then Buy Product = Yes
- Rule 2: If Age = Young and Student = No, then Buy Product = No
- Rule 3: If Age = Young and Student = Yes, then Buy Product = Yes
- Rule 4: If Age = Old and Income = Medium, then Buy Product = Yes
- Rule 5: If Age = Old and Income = Low, then Buy Product = Yes (majority rule)

Results of this numerical shows that **Age** is the most important attribute because it gives the highest Information Gain and therefore becomes the root node of the Decision Tree. This means Age reduces uncertainty more than Income and Student in this dataset. After selecting Age, the algorithm continues to divide the data into smaller and purer subsets. For the Young group, Student status becomes important, while for the Old group, Income becomes the better splitting factor. The final tree provides clear and interpretable rules for classification.

In practical terms, the Decision Tree is helpful because it converts raw data into simple human-readable decisions such as “If Age is Young and Student is No, then customer will not buy the product.” This is why Decision Trees are widely used in banking, marketing, healthcare, and business decision-making.

While solving Decision Tree numerical, always follow this sequence:

- First, calculate the entropy of the full dataset.
- Second, calculate the entropy of subsets formed by each attribute.
- Third, compute Information Gain for each attribute.
- Fourth, choose the attribute with the highest Information Gain as the root node.

- Fifth, repeat the same process for each non-pure branch until pure nodes or majority-based terminal nodes are obtained.

So, the full logic is: Entropy $\rightarrow$ Information Gain $\rightarrow$ Best Split $\rightarrow$ Repeat

This stepwise process helps in understanding not only how the tree is built but also why a particular attribute is chosen at each stage.

Here is One more Numerical on Decision Tree Using Gini Index

Example 6: A coaching institute wants to predict whether a student will **Pass** or **Fail** based on the following attributes:

- **Attendance:** High, Low
- **Assignment Submission:** Yes, No

The training dataset is given below:

Record	Attendance	Assignment Submission	Result
1	High	Yes	Pass
2	High	Yes	Pass
3	High	No	Pass
4	High	No	Fail
5	Low	Yes	Pass
6	Low	Yes	Fail
7	Low	No	Fail
8	Low	No	Fail

Using the **Decision Tree algorithm with Gini Index**, determine the **best attribute for the root node**. Show all calculations step by step and write the final decision rules.

Solution: Objective of the given problem is to construct a Decision Tree for classification. The target variable is:

$$Y = \text{Result}$$

where:

- *Pass* = student passes the exam
- *Fail* = student fails the exam

In Gini-based Decision Tree, the best split is the one with the **lowest weighted Gini Index**.

Formula Used for the Gini Index of a dataset is: $Gini(S) = 1 - \sum_{i=1}^n p_i^2$

For binary classification: $Gini(S) = 1 - (p_{pass}^2 + p_{fail}^2)$

Weighted Gini after splitting on attribute A is: $Gini(S, A) = \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \times Gini(S_v)$

Where,

- S: complete dataset

- A : attribute chosen for splitting
- S_v : subset of data for a particular value v of attribute A
- p_{pass} : proportion of Pass cases
- p_{fail} : proportion of Fail cases
- $Gini(S)$: impurity of dataset S
- $Gini(S, A)$: weighted impurity after splitting on attribute A

Note: A smaller Gini value means a purer node.

The stepwise solution of the given problem is as follows:

Step 1: Calculate Gini Index of Full Dataset

From the table:

- Total records = 8
- Pass = 4
- Fail = 4

So,

$$p_{pass} = \frac{4}{8} = 0.5$$

$$p_{fail} = \frac{4}{8} = 0.5$$

Now,

$$Gini(S) = 1 - (0.5^2 + 0.5^2)$$

$$Gini(S) = 1 - (0.25 + 0.25)$$

$$Gini(S) = 1 - 0.5 = 0.5$$

Result: $Gini(S) = 0.5$

This shows that the dataset is impure and needs splitting.

Step 2: Calculate Weighted Gini for Attribute = Attendance

Attendance has two values:

- High
- Low

(a) Subset for Attendance = High

Records: 1, 2, 3, 4

Class labels: Pass, Pass, Pass, Fail

- Pass = 3
- Fail = 1

$$p_{pass} = \frac{3}{4}, p_{fail} = \frac{1}{4}$$

$$Gini(High) = 1 - \left(\left(\frac{3}{4} \right)^2 + \left(\frac{1}{4} \right)^2 \right)$$

$$Gini(High) = 1 - \left(\frac{9}{16} + \frac{1}{16} \right)$$

$$Gini(High) = 1 - \frac{10}{16} = \frac{6}{16} = 0.375$$

(b) Subset for Attendance = Low

Records: 5, 6, 7, 8

Class labels: Pass, Fail, Fail, Fail

- Pass = 1
- Fail = 3

$$p_{pass} = \frac{1}{4}, p_{fail} = \frac{3}{4}$$

$$Gini(Low) = 1 - \left(\left(\frac{1}{4} \right)^2 + \left(\frac{3}{4} \right)^2 \right)$$

$$Gini(Low) = 1 - \left(\frac{1}{16} + \frac{9}{16} \right)$$

$$Gini(Low) = 1 - \frac{10}{16} = \frac{6}{16} = 0.375$$

(c) Weighted Gini for Attendance

$$Gini(S, Attendance) = \frac{4}{8} (0.375) + \frac{4}{8} (0.375)$$

$$Gini(S, Attendance) = 0.1875 + 0.1875 = 0.375$$

Result: $Gini(S, Attendance) = 0.375$

Step 3: Calculate Weighted Gini for Attribute = Assignment Submission

Assignment Submission has two values:

- Yes
- No

(a) Subset for Assignment = Yes

Records: 1, 2, 5, 6

Class labels: Pass, Pass, Pass, Fail

- Pass = 3
- Fail = 1

$$Gini(Yes) = 1 - \left(\left(\frac{3}{4} \right)^2 + \left(\frac{1}{4} \right)^2 \right)$$

$$Gini(Yes) = 1 - \left(\frac{9}{16} + \frac{1}{16} \right) = 0.375$$

(b) Subset for Assignment = No

Records: 3, 4, 7, 8

Class labels: Pass, Fail, Fail, Fail

- Pass = 1
- Fail = 3

$$Gini(No) = 1 - \left(\left(\frac{1}{4} \right)^2 + \left(\frac{3}{4} \right)^2 \right)$$

$$Gini(No) = 1 - \left(\frac{1}{16} + \frac{9}{16} \right) = 0.375$$

(c) Weighted Gini for Assignment Submission

$$Gini(S, Assignment) = \frac{4}{8} (0.375) + \frac{4}{8} (0.375)$$

$$Gini(S, Assignment) = 0.375$$

Result: $Gini(S, Assignment) = 0.375$

Step 4: Compare the Split Values

- $Gini(S, \text{Attendance}) = 0.375$
- $Gini(S, \text{Assignment}) = 0.375$

Both attributes give the same weighted Gini Index.

Decision: Since both splits are equally good, we may choose either attribute as the root node.
Let us choose: Attendance as the root node.

Step 5: Continue Splitting the Branches:

Now split the data based on **Attendance**.

Branch 1: Attendance = High

Subset:

Record	Attendance	Assignment Submission	Result
1	High	Yes	Pass
2	High	Yes	Pass
3	High	No	Pass
4	High	No	Fail

Class counts:

- Pass = 3
- Fail = 1

$$Gini(High) = 0.375$$

Now only one attribute remains: **Assignment Submission**

Split this subset by Assignment Submission

Assignment = Yes

Records: 1, 2

Class labels: Pass, Pass

$$Gini = 1 - (1^2 + 0^2) = 0$$

Assignment = No

Records: 3, 4

Class labels: Pass, Fail

$$Gini = 1 - \left(\left(\frac{1}{2} \right)^2 + \left(\frac{1}{2} \right)^2 \right)$$

$$Gini = 1 - (0.25 + 0.25) = 0.5$$

Weighted Gini for this split

$$Gini(High, \text{Assignment}) = \frac{2}{4}(0) + \frac{2}{4}(0.5)$$

$$Gini(High, \text{Assignment}) = 0 + 0.25 = 0.25$$

So, this split improves purity.

Thus:

- Attendance = High and Assignment = Yes $\rightarrow$ Pass
- Attendance = High and Assignment = No $\rightarrow$ mixed node

Since no further attribute remains, we assign the **majority class** of this node. In this node, one Pass and one Fail are present, so tie handling is needed. In exam settings, we usually use the parent majority class. Parent node majority = Pass.

So,

- Attendance = High and Assignment = No → Pass

Branch 2: Attendance = Low

Subset:

Record Attendance Assignment Submission Result

5	Low	Yes	Pass
6	Low	Yes	Fail
7	Low	No	Fail
8	Low	No	Fail

Class counts:

- Pass = 1
- Fail = 3

$$Gini(Low) = 0.375$$

Now split by **Assignment Submission**

Assignment = Yes

Records: 5, 6

Class labels: Pass, Fail

$$Gini = 1 - \left(\left(\frac{1}{2} \right)^2 + \left(\frac{1}{2} \right)^2 \right) = 0.5$$

Assignment = No

Records: 7, 8

Class labels: Fail, Fail

$$Gini = 1 - (0^2 + 1^2) = 0$$

Weighted Gini for this split

$$Gini(Low, Assignment) = \frac{2}{4}(0.5) + \frac{2}{4}(0)$$

$$Gini(Low, Assignment) = 0.25$$

Thus:

- Attendance = Low and Assignment = No → Fail
- Attendance = Low and Assignment = Yes → mixed node

Again, no further attribute is available, so use majority class of parent node. Parent node majority = Fail.

So,

- Attendance = Low and Assignment = Yes → Fail

Final Decision Tree

Root node: **Attendance**

High → check **Assignment Submission**

- Yes → Pass
- No → Pass (majority rule)

Low → check **Assignment Submission**

- Yes → Fail (majority rule)
- No → Fail

Final Decision Rules

- Rule 1: If Attendance = High and Assignment Submission = Yes, then Result = Pass
- Rule 2: If Attendance = High and Assignment Submission = No, then Result = Pass (by majority rule)
- Rule 3: If Attendance = Low and Assignment Submission = Yes, then Result = Fail (by majority rule)
- Rule 4: If Attendance = Low and Assignment Submission = No, then Result = Fail

Results of this numerical shows how the Gini Index is used to build a Decision Tree by selecting the split with the lowest impurity. Here, both attributes, Attendance and Assignment Submission, produced the same weighted Gini value of 0.375 at the root level. Therefore, either attribute could be chosen as the root node. After selecting Attendance, the remaining attribute helped reduce impurity further within each branch.

The final rules show a simple interpretation: students with high attendance are generally predicted to pass, while students with low attendance are generally predicted to fail. This is a practical and human-readable outcome, which is one of the biggest advantages of Decision Trees.

While solving Gini-based Decision Tree numerical, follow this sequence:

- First, calculate the Gini Index of the full dataset.
- Second, split the dataset by each attribute and calculate the weighted Gini after each split.
- Third, choose the attribute with the smallest weighted Gini as the best split.
- Fourth, repeat the process for each branch until pure or nearly pure nodes are obtained.
- If no further attribute is left and the node is still mixed, apply the majority class rule.

So, the logic is: Gini of dataset → Weighted Gini of each split → Best split → Repeat

The advantages and disadvantages of the decision trees are as follows:

Advantages of Decision Trees

- Decision Trees are easy to understand and interpret, making them highly suitable for decision-making systems.
- They require minimal data preprocessing and can handle both numerical and categorical data.
- They are also capable of capturing non-linear relationships and interactions between features. Additionally, they provide clear decision rules that can be visualized and explained.

Limitations of Decision Trees

- Decision Trees are prone to overfitting, especially when the tree becomes too deep.
- They can be sensitive to small changes in data, which may lead to different tree structures.
- They may also become biased if the dataset is imbalanced.

- Furthermore, single Decision Trees often do not provide the best predictive performance compared to ensemble methods such as Random Forests.

This section concludes with the understanding that the Decision Trees can be understood as a systematic way of asking questions to arrive at a decision. At each step, the algorithm selects the best question (feature) that reduces uncertainty the most. This process continues until the data is sufficiently pure. The use of entropy and information gain provides a mathematical foundation for what appears to be a simple decision-making process. Because of their interpretability and logical structure, Decision Trees form the basis for many advanced algorithms and are an essential concept in machine learning.

☛ Check Your Progress 5 6:

Q1. What is a Decision Tree? Explain its basic structure.

.....

Q2. How does a Decision Tree work in classification problems?

.....

Q3. What are the advantages of Decision Trees?

.....

Q4. What are the limitations of Decision Trees?

.....

Q5. What is overfitting in Decision Trees and how can it be controlled?

.....

10.3.5 RANDOM FOREST

Decision Trees and Random Forest are both supervised learning algorithms based on tree structures, but they differ significantly in their approach and performance.

A Decision Tree is a single model that splits the dataset into smaller subsets based on feature conditions, forming a tree-like structure of decisions. It is simple, easy to understand, and highly interpretable, as the decision-making process can be clearly visualized through rules. However, this simplicity comes with a drawback: Decision Trees are highly prone to overfitting, especially when the tree becomes deep and captures noise in the data. They are also sensitive to small changes in the dataset, which can lead to completely different tree structures.

On the other hand, Random Forest is an ensemble learning technique that builds multiple Decision Trees and combines their predictions to produce a final result. Instead of relying on a single tree, it uses techniques such as bootstrapping (random sampling of data) and random feature selection to create diversity among trees. The final prediction is obtained through majority voting in classification or averaging in regression. This approach significantly improves accuracy and reduces overfitting, making Random Forest more robust and reliable compared to a single Decision Tree.

In terms of performance, Random Forest generally outperforms Decision Trees because it reduces variance and improves generalization. However, this comes at the cost of increased complexity and reduced interpretability. While a Decision Tree provides clear and straightforward decision rules, Random Forest behaves more like a black-box model where individual decisions are harder to interpret. Additionally, Random Forest requires more computational resources and training time due to the construction of multiple trees, whereas Decision Trees are faster and more efficient for smaller datasets.

It is to be noted that the Decision Trees are best suited for problems where interpretability and simplicity are important, while Random Forest is preferred when higher accuracy, robustness, and better generalization are required.

Now, we Understand the Random Forest in more detail,

Random Forest Algorithm is an advanced supervised learning algorithm that improves the performance of Decision Trees by combining multiple trees to make better predictions. It is based on the concept of **ensemble learning**, where instead of relying on a single model, multiple models are trained and their outputs are combined.

The main idea behind Random Forest is that a group of weak learners (decision trees) can come together to form a strong learner. Each tree in the forest is trained on a random subset of the data and uses a random subset of features. This randomness helps in reducing overfitting and improves generalization.

Working of Random Forest: The Random Forest algorithm works in a step-by-step manner to improve prediction accuracy by combining multiple decision trees.

- First, it creates several new datasets from the original dataset using a method called **Bootstrapping (Sampling with replacement)**, where data points are randomly selected with replacement. This means some data points may appear multiple times while others may not appear at all in a particular sample.
- Next, for each of these datasets, a separate **Decision Tree** is built. So instead of having one tree, we now have multiple trees trained on slightly different data.
- While constructing each tree, the algorithm does not consider all features at every split. Instead, it selects a **random subset of features** for splitting. This introduces diversity among the trees and helps reduce overfitting.
- Finally, once all the trees are built, their predictions are combined to produce the final output.
 - In case of classification problems, the final prediction is made using **majority voting**, meaning the class predicted by most trees is selected.
 - In case of regression problems, the final output is calculated by taking the **average of predictions** from all trees.

In this way, Random Forest improves accuracy and stability by combining the results of multiple decision trees instead of relying on just one.

So far as the Mathematical Representation of Random Forest is concerned, in any random Forest the final prediction is obtained by combining the outputs of multiple decision trees.

- For **classification problems**, each tree gives a class prediction, and the final result is decided based on **majority voting**. This means the class that is predicted by the highest number of trees is selected as the final output.

$\hat{Y}$ = Majority vote of all trees

For **regression problems**, each tree gives a numerical value, and the final prediction is calculated by taking the **average of all tree outputs**.

$$\hat{Y} = \frac{1}{N} \sum_{i=1}^N T_i(x)$$

Where:

- N : total number of decision trees in the forest
- $T_i(x)$: prediction made by the i^{th} decision tree for input x
- $\hat{Y}$: final predicted output

In simple terms, Random Forest combines multiple predictions to produce a more accurate and stable result.

Just for example consider a medical diagnosis system where multiple doctors (trees) give their opinion on whether a patient has a disease or not. Instead of trusting one doctor, we take the majority opinion of all doctors. If most doctors say “Yes,” the final decision is “Yes.” This reduces the chances of wrong diagnosis, similar to how Random Forest works.

In terms of the numerical example suppose a Random Forest model consists of **5 decision trees** predicting whether a student will pass or fail.

Predictions of trees:

Tree	Prediction
T1	Pass
T2	Fail
T3	Pass
T4	Pass
T5	Fail

Step 1: Count Votes

- Pass = 3
- Fail = 2

Step 2: Apply Majority Voting: $\hat{Y}$ = Pass

Final Answer is that the student will Pass

The Results of the above numerical can be interpreted as, The Random Forest model predicts that the student will pass because the majority of trees (3 out of 5) voted for the “Pass” class. This demonstrates how combining multiple models reduces the impact of individual errors and leads to more reliable predictions.

Let’s Understand the Random Forest Algorithm with one more Comprehensive Numerical

Example6: A college wants to predict whether a student will **Pass** or **Fail** in the final examination using the **Random Forest algorithm**. After training the Random Forest model, 5 different decision trees are generated. For a new student, the predictions given by these trees are as follows:

Tree No.	Prediction
Tree 1	Pass
Tree 2	Fail

Tree No.	Prediction
Tree 3	Pass
Tree 4	Pass
Tree 5	Fail

For the same student, the probability estimates for the class **Pass** given by the 5 trees are:

Tree No.	Probability of Pass
Tree 1	0.80
Tree 2	0.35
Tree 3	0.75
Tree 4	0.90
Tree 5	0.40

Using the Random Forest algorithm, answer the following:

1. Find the final predicted class using **majority voting**.
2. Find the average probability of the class **Pass**.
3. Interpret the result.
4. Write the final conclusion.

Solution:

In the given problem, we have to understand how Random Forest makes a final prediction by combining the outputs of multiple decision trees. Since Random Forest is an **ensemble learning method**, it does not depend on a single tree. Instead, it takes the predictions of many trees and combines them.

For **classification problems**, the final prediction is usually based on:

- **Majority voting**, and sometimes
- **Average probability** across trees

Formula Used in Random Forest are:

(a) **Majority Voting for Classification**

$$\hat{Y} = \text{Majority vote of all trees}$$

This means the class predicted by the maximum number of trees becomes the final output.

(b) **Average Probability of a Class**

$$P(\text{Pass}) = \frac{1}{N} \sum_{i=1}^N P_i(\text{Pass})$$

Where:

- N = total number of trees
- $P_i(\text{Pass})$ = probability of class Pass predicted by the i^{th} tree
- $P(\text{Pass})$ = final average probability of Pass

If needed, then:

$$P(\text{Fail}) = 1 - P(\text{Pass})$$

For **regression problems**, each tree gives a numerical value, and the final prediction is calculated by taking the **average of all tree outputs**.

$$\hat{Y} = \frac{1}{N} \sum_{i=1}^N T_i(x)$$

Where,

- $\hat{Y}$: final predicted class
- N : total number of trees in the forest
- T_i : prediction of the i^{th} decision tree
- $P_i(\text{Pass})$: probability assigned by the i^{th} tree for class Pass
- $P(\text{Pass})$: final average probability that the student will pass

Step wise Solution to the given problem is as follows:

Step 1: Write the Tree Predictions

From the question, the tree predictions are:

- Tree 1 $\rightarrow$ Pass
- Tree 2 $\rightarrow$ Fail
- Tree 3 $\rightarrow$ Pass
- Tree 4 $\rightarrow$ Pass
- Tree 5 $\rightarrow$ Fail

Now count the votes.

Count of Pass:3

Count of Fail:2

Step 2: Apply Majority Voting

The formula is:

$$\hat{Y} = \text{Majority vote of all trees}$$

Since the number of Pass votes is greater than the number of Fail votes:

$$3 > 2$$

So, the final predicted class is: $\hat{Y} = \text{Pass}$

Step 3: Compute Average Probability of Pass

The probabilities given by the trees are:

- Tree 1: 0.80
- Tree 2: 0.35
- Tree 3: 0.75
- Tree 4: 0.90
- Tree 5: 0.40

Using the formula:

$$P(\text{Pass}) = \frac{1}{N} \sum_{i=1}^N P_i(\text{Pass})$$

Here, $N = 5$

So,

$$P(\text{Pass}) = \frac{0.80 + 0.35 + 0.75 + 0.90 + 0.40}{5}$$

First add all probabilities:

$$0.80 + 0.35 = 1.15$$

$$1.15 + 0.75 = 1.90$$

$$1.90 + 0.90 = 2.80$$

$$2.80 + 0.40 = 3.20$$

Now divide by 5:

$$P(\text{Pass}) = \frac{3.20}{5}$$

$$P(\text{Pass}) = 0.64$$

Thus, the average probability of Pass is: 0.64 or 64%

Step 4: Compute Probability of Fail

If required, probability of Fail can be found as:

$$P(\text{Fail}) = 1 - P(\text{Pass})$$

$$P(\text{Fail}) = 1 - 0.64$$

$$P(\text{Fail}) = 0.36 \text{ or } 36\%$$

Final Answer

Final predicted class: Pass

Average probability of Pass: 0.64 or 64%

Average probability of Fail: 0.36 or 36%

The Results of the Random Forest model in the above numerical, predicts that the student will **Pass** because the majority of trees, that is, 3 out of 5 trees, voted for the class Pass. This is the first basis of the final decision.

The average probability of Pass is **0.64**, which means the model estimates a **64% chance** that the student will pass the exam. Since this value is greater than 0.5, it supports the majority voting result. Therefore, both the class votes and the average probability indicate that the student is more likely to pass than fail.

This also shows one of the biggest strengths of Random Forest i.e. even if some trees make wrong predictions, the combined decision of many trees often leads to a more reliable final result.

To solve a Random Forest numerical, always follow these steps:

- First, list the prediction given by each tree.
- Second, count how many trees voted for each class.
- Third, choose the class with the highest number of votes as the final predicted class.
- Fourth, if probabilities are given, add the probabilities of the required class from all trees and divide by the total number of trees.
- Fifth, interpret both the final class and the probability.

So, the main idea is: Many tree predictions → Combine them → Final decision

Note: Random Forest works on the principle that a group decision is often better than a single decision.

Advantages and Limitation of Random Forests are as follows:

Advantages of Random Forest

- Random Forest provides high accuracy and performs well on large datasets.
- It reduces overfitting by averaging multiple trees.
- It can handle both classification and regression problems.
- It works well even when there are missing values or noisy data.
- It also provides feature importance, which helps in understanding which variables are most influential.

Limitations of Random Forest

- Random Forest models are more complex and less interpretable compared to Decision Trees.
- They require more computational power and memory.
- Training time is higher due to multiple trees.
- It may not be suitable for real-time systems where quick predictions are required.
- Also, the model can become unnecessarily large if too many trees are used.

We learned in this section that Random Forest can be understood as an improved version of Decision Trees. While a single Decision Tree may give unstable or biased results, Random Forest overcomes this by combining many trees and using randomness. The key idea is that diversity among trees leads to better performance. This makes Random Forest one of the most powerful and widely used algorithms in machine learning.

Check Your Progress 7:

Q1. What is Random Forest? Explain its basic concept..

.....

Q2. How does Random Forest improve over Decision Trees?

.....

Q3. Explain the working of Random Forest..

.....

Q4. What are the advantages of Random Forest?.

.....

Q5. What are the limitations of Random Forest?

.....

10.3.6 SUPPORT VECTOR MACHINES (SVM)

Support Vector Machines (SVM) is a powerful supervised learning algorithm used for both classification and regression problems, although it is most commonly applied to classification tasks. The main objective of SVM is to find the optimal boundary (called a hyperplane) that separates different classes of data in such a way that the margin between them is maximized. Unlike many other algorithms that simply try to separate classes, SVM focuses on finding the best possible separation with the maximum distance from the nearest data points of each class.

Before going into the details of SVM, let's understand the Concept of Hyperplane and Margin: In SVM, a hyperplane is a decision boundary that separates data points belonging to different classes. In a two-dimensional space, this hyperplane is simply a straight line. The key idea is not just to separate the classes, but to maximize the margin, which is the distance between the hyperplane and the closest data points from each class. These closest points are called support vectors, and they play a crucial role in defining the position of the hyperplane.

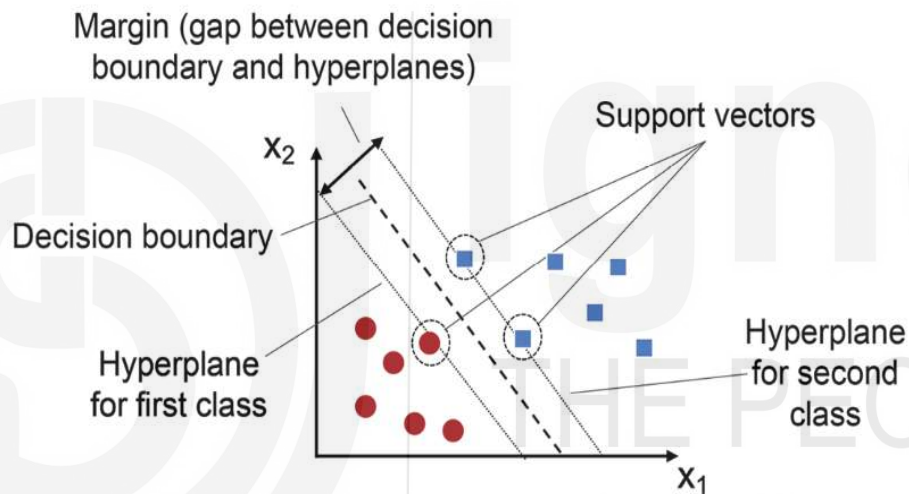


Figure-2 Support Vector Machines (SVM)

The diagram shown above in Figure-2, shows two classes of data points (for example, positive and negative). Multiple lines could separate these classes, but SVM selects the one that maximizes the margin. The points closest to the boundary are the **support vectors**, and the distance between these points and the hyperplane defines the margin. A larger margin generally leads to better generalization and improved model performance.

In Support Vector Machines, the **support vectors** are the most important data points in the dataset because they lie closest to the separating boundary, known as the hyperplane. These points essentially “support” or define the position of the hyperplane. Unlike other points that may lie far away from the boundary, support vectors directly influence how the decision boundary is placed. Even a small change in these points can shift the hyperplane, which shows their critical role in the model.

The concept of **margin** is closely related to support vectors. The margin refers to the distance between the hyperplane and the nearest data points from both classes, which are the support vectors. In SVM, the goal is to maximize this margin so that the separation between classes

becomes as wide as possible, leading to better generalization and reduced chances of misclassification.

Mathematically, the margin is calculated using the formula:

$$\text{Margin} = \frac{2}{\|w\|}$$

where $\|w\|$ represents the magnitude (or length) of the weight vector w . The magnitude of w is computed as:

$$\|w\| = \sqrt{w_1^2 + w_2^2}$$

Here, w_1 and w_2 are the components of the weight vector corresponding to the input features. The value of $\|w\|$ determines how steep or tilted the hyperplane is.

From the formula, we can observe that the margin is inversely proportional to the magnitude of w . This means that if $\|w\|$ is small, the margin becomes larger, and if $\|w\|$ is large, the margin becomes smaller. Therefore, SVM tries to find a hyperplane that minimizes $\|w\|$ in order to maximize the margin.

In simple terms, support vectors are the boundary points that define the decision line, and the margin is the safe distance maintained from these points. A larger margin ensures a more reliable and robust classification model.

The Mathematical Representation of SVM is given by the equation of the hyperplane is given by:

$$w \cdot x + b = 0$$

Where:

- w : weight vector (normal to the hyperplane)
- x : input feature vector
- b : bias term

For classification:

$$\begin{aligned} w \cdot x + b &\geq +1 (\text{Class 1}) \\ w \cdot x + b &\leq -1 (\text{Class 2}) \end{aligned}$$

The margin is given by:

$$\text{Margin} = \frac{2}{\|w\|}$$

SVM aims to maximize this margin.

Just for example consider an email classification system that separates emails into **Spam** and **Not Spam** based on features such as the number of suspicious words and links. SVM will draw a boundary that best separates spam emails from normal emails while ensuring that the closest emails to the boundary are as far apart as possible. This reduces the chances of misclassification.

Let's understand SVM with a simple Numerical Example, given below:

Example 7: Consider the following two data points from two classes:

- Class +1: (2, 2)
- Class -1: (4, 4)

Assume a simple hyperplane:

$$w = [1, -1], \quad b = 0$$

Solution:

Step 1: Check classification for point (2,2)

$$w \cdot x + b = (1)(2) + (-1)(2) + 0 = 2 - 2 = 0$$

Step 2: Check classification for point (4,4)

$$w \cdot x + b = (1)(4) + (-1)(4) = 0$$

Both points lie on the boundary, so this hyperplane is not optimal.

Now consider a better hyperplane:

$$w = [1, -1], \quad b = -1$$

Step 3: Recalculate

- For (2,2): $(1)(2) + (-1)(2) - 1 = -1 \rightarrow$ Class -1
- For (4,4): $(1)(4) + (-1)(4) - 1 = -1 \rightarrow$ Class -1 again (incorrect)

This shows that choosing the correct hyperplane is critical, and SVM mathematically optimizes this selection.

The numerical example highlights that not every separating line is suitable. SVM carefully selects the hyperplane that maximizes the margin while correctly classifying the data. The support vectors define this boundary, and any incorrect choice of parameters leads to misclassification.

Let's understand SVM in more detail through a Numerical given below:

Example 8: Consider the following two classes of data points for a binary classification problem:

- Class +1: $A(3,1), B(3,2)$
- Class -1: $C(1,3), D(2,3)$

Assume that the SVM classifier gives the separating hyperplane:

$$x_1 + x_2 - 4 = 0$$

Using this hyperplane, answer the following:

1. Verify whether the given points are correctly classified.
2. Identify the support vectors.
3. Find the margin of the classifier.
4. Classify a new point $P(4,1)$.
5. Interpret the result.

Solve the problem step by step using the SVM equations, explain all variables used, and present the answer in a self-learning format.

Solution:

We learned that Support Vector Machine is a supervised learning algorithm used mainly for classification. Its goal is to find the best separating boundary, called a hyperplane, between two classes. This hyperplane should separate the classes in such a way that the margin, or the distance between the closest points of the two classes and the hyperplane, is maximum.

In two dimensions, the hyperplane is simply a straight line.

The Equation Used in SVM are as follows:

- The general equation of a hyperplane is: $w \cdot x + b = 0$
- For two features, this becomes: $w_1 x_1 + w_2 x_2 + b = 0$
- For the given problem: $x_1 + x_2 - 4 = 0$
- So, by comparison: $w = (1, 1), b = -4$

Where,

- $x = (x_1, x_2)$: input feature vector
- x_1, x_2 : coordinates of a data point
- $w = (w_1, w_2)$: weight vector, perpendicular to the hyperplane
- b : bias or intercept term
- $w \cdot x + b$: decision function
- If $w \cdot x + b > 0$, point belongs to Class +1
- If $w \cdot x + b < 0$, point belongs to Class -1
- Support vectors are the points closest to the hyperplane with Margin given by:

$$\text{Margin} = \frac{2}{\|w\|} \text{ where } \|w\| = \sqrt{w_1^2 + w_2^2}$$

Stepwise solution to the given problem is as follows:

Step 1: Verify Classification of Given Points

We use the decision function:

$$f(x) = x_1 + x_2 - 4$$

Now calculate for each point.

Point A(3,1), Class +1

$$f(A) = 3 + 1 - 4 = 0$$

So point A lies exactly on the boundary.

Point B(3,2), Class +1

$$f(B) = 3 + 2 - 4 = 1$$

Since $f(B) > 0$, it belongs to Class +1.
Correctly classified.

Point C(1,3), Class -1

$$f(C) = 1 + 3 - 4 = 0$$

So, point C also lies exactly on the boundary.

Point D(2,3), Class -1

$$f(D) = 2 + 3 - 4 = 1$$

Since $f(D) > 0$, this point is classified as Class +1.

But actual class is Class -1. So, point D is misclassified.

Step 2: Check Whether the Given Hyperplane is a Proper SVM Separator

From the above calculations:

- A(3,1) lies on boundary

- $C(1,3)$ lies on boundary
- $D(2,3)$ is wrongly classified

Therefore, this hyperplane is not a correct separating hyperplane for all training points.

However, from an exam point of view, we can still use the given line to demonstrate how SVM calculations are done.

Step 3: Identify Support Vectors

Support vectors are the points nearest to the separating hyperplane. Usually, these satisfy:

$$w \cdot x + b = +1 \text{ or } w \cdot x + b = -1$$

But in this simplified question, the closest points are those lying on or nearest to the boundary.

From the calculations:

- $A(3,1): f(A) = 0$
- $C(1,3): f(C) = 0$

These are the nearest points to the hyperplane.

So, the support vectors are: $A(3,1)$ and $C(1,3)$

Step 4: Calculate the Margin

The margin formula is:

$$\text{Margin} = \frac{2}{\|w\|}$$

First calculate $\|w\|$:

$$\|w\| = \sqrt{1^2 + 1^2}$$

$$\|w\| = \sqrt{2}$$

Now substitute:

$$\text{Margin} = \frac{2}{\sqrt{2}}$$

$$\text{Margin} = \sqrt{2}$$

So, the margin is: $\sqrt{2} \approx 1.414$

Step 5: Classify the New Point $P(4,1)$

Use the decision function:

$$f(P) = 4 + 1 - 4$$

$$f(P) = 1$$

Since $f(P) > 0$, the point belongs to: Class +1

Thus, the Final Answer has Support vectors: $A(3,1)$, $C(1,3)$ and Margin: $\sqrt{2} \approx 1.414$
Classification of new point $P(4,1)$: Class +1

This numerical shows how SVM uses the hyperplane equation to classify points. By substituting the coordinates of each point into the decision function, we can determine on which side of the boundary the point lies. The sign of the result tells the predicted class. The

support vectors are the points closest to the boundary, and the margin tells us how far apart the two classes are separated by the hyperplane.

For the new point $P(4,1)$, the value of the decision function is positive, so it is placed in Class +1. This means the point lies on the positive side of the hyperplane. A larger margin generally indicates better class separation and better generalization.

At the same time, this example also teaches an important lesson: not every given line is a perfect SVM separator. Since one of the original training points is misclassified, the given hyperplane is not the ideal one for the whole dataset. In actual SVM training, the algorithm finds the hyperplane that separates the training data as correctly as possible while maximizing the margin.

While solving SVM numericals, always follow this order:

- First, write the hyperplane equation: $w \cdot x + b = 0$
- Second, identify the values of w and b .
- Third, substitute each data point into: $f(x) = w \cdot x + b$
- Fourth, check the sign of the result:
 - Positive $\rightarrow$ Class +1
 - Negative $\rightarrow$ Class -1
 - Zero $\rightarrow$ Point lies on the boundary
- Fifth, find the support vectors, which are the closest points to the boundary.
- Sixth, calculate the margin using: $\text{Margin} = \frac{2}{\|w\|}$
- Finally, classify any new point using the same decision function.

So, the full logic is:

Hyperplane $\rightarrow$ Decision function $\rightarrow$ Class prediction $\rightarrow$ Support vectors $\rightarrow$ Margin

Before concluding this section on SVM, one more important concept needs attention, and that concept is **Kernel Trick**: In many real-world classification problems, the data is not linearly separable, meaning we cannot draw a straight line (or hyperplane) that perfectly separates the classes. In such cases, a simple SVM with a linear boundary fails to classify the data correctly.

To overcome this limitation, SVM uses a powerful concept known as the Kernel Trick. The idea behind the kernel trick is to transform the original data into a higher-dimensional space where the data becomes linearly separable. Instead of explicitly computing this transformation (which can be computationally expensive), SVM uses a mathematical function called a kernel function to directly compute the relationships in the higher-dimensional space.

In simple terms, the kernel trick allows us to solve complex, non-linear problems by converting them into linear problems in a higher dimension—without actually performing the transformation explicitly.

Let's now talk about an example and imagine that data points are arranged in a circular pattern where one class is inside the circle and another is outside. In two dimensions, no straight line can separate these classes. However, if we project this data into a higher

dimension (for example, adding a new feature like radius), the data can now be separated using a straight line (or plane). This transformation is what the kernel trick achieves.

There are several types of kernel, few of the Common Types of Kernels are as follows:

1. Linear Kernel: The linear kernel is the simplest form and is used when the data is already linearly separable.

$$K(x_i, x_j) = x_i \cdot x_j$$

It computes the dot product between two data points. This kernel does not transform the data and works well for simple datasets.

Example: Classifying emails based on word frequency when the data is already separable using a straight line.

2. Polynomial Kernel: The polynomial kernel allows the algorithm to model more complex relationships by considering polynomial combinations of features.

$$K(x_i, x_j) = (x_i \cdot x_j + 1)^d$$

Where d is the degree of the polynomial.

Example: In a student performance dataset, where marks and study hours have a non-linear relationship, a polynomial kernel can capture curved decision boundaries.

3. Radial Basis Function (RBF) Kernel: The RBF kernel (also called Gaussian kernel) is one of the most widely used kernels. It maps data into an infinite-dimensional space and can handle highly non-linear relationships.

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

Where:

- γ controls how far the influence of a point extends

Example: Image classification or pattern recognition where the data has complex, irregular boundaries.

It is learned that the kernel trick allows SVM to create non-linear decision boundaries while still using linear separation logic internally. Instead of drawing a straight line in the original space, SVM effectively draws a complex curve or boundary by operating in a higher-dimensional space.

It is the kernel trick that makes SVM a powerful and flexible algorithm. It enables SVM to handle both simple linear problems and highly complex non-linear problems without increasing computational complexity significantly. By choosing the appropriate kernel function, SVM can adapt to different types of data and provide accurate classification results.

The Advantages and the Limitations of Support Vector Machines are as follows:

Advantages of SVM:

- SVM is highly effective in high-dimensional spaces and works well when the number of features is large.
- It provides good generalization by maximizing the margin.
- It is robust to overfitting, especially in cases where the margin is large.
- It also works well with clear separation between classes.

Limitations of SVM

- SVM can be computationally expensive for large datasets.
- It is less interpretable compared to Decision Trees.
- Choosing the correct kernel and parameters can be challenging.
- It also performs poorly when there is a lot of noise or overlapping classes in the data.

From the above discussion it is understood that Support Vector Machines combine geometric intuition with mathematical optimization to provide a robust classification method. By focusing on margin maximization and using kernel functions for complex data, SVM becomes a versatile tool in machine learning. It is widely used in applications such as image classification, text categorization, and bioinformatics, where accurate classification is essential.

Actually, SVM is an algorithm that tries to draw the **best possible boundary** between classes, not just any boundary. The idea of maximizing the margin makes it different from other classifiers. Instead of focusing on all data points, SVM focuses only on the most critical ones—the support vectors. This makes it both efficient and powerful.

☛ Check Your Progress 8:

Q1. What is a Support Vector Machine (SVM)? Explain its objective..

.....

Q2. What is a hyperplane and margin in SVM?

.....

Q3. What are Support Vectors? Why are they important?

.....

Q4. How does SVM handle non-linear data?

.....

Q5. What are the advantages and limitations of SVM?

.....

10.4 SUMMARY

This unit provides a comprehensive understanding of machine learning classification techniques, beginning with the fundamental distinction between Supervised and Unsupervised Learning. Supervised learning involves training a model using labelled data, where both input and output are known, enabling the model to learn patterns and make predictions. In contrast, unsupervised learning deals with unlabelled data and focuses on discovering hidden structures or patterns without predefined outputs. This distinction forms the foundation for understanding various supervised learning models discussed in the unit.

The unit then explores several important supervised learning algorithms. It begins with K-Nearest Neighbour (KNN), a simple instance-based algorithm that classifies data points based on the majority class of their nearest neighbors. Next, Naïve Bayes is introduced as a probabilistic classifier based on Bayes' theorem, which assumes independence among features and is widely used in text classification. Logistic Regression is discussed as a statistical method that predicts probabilities using the sigmoid function, making it suitable for binary classification problems.

Further, the unit covers Decision Trees, which model decision-making in a tree-like structure using concepts such as entropy and information gain. While Decision Trees are easy to interpret, they may suffer from overfitting. To address this limitation, the unit introduces Random Forest, an ensemble technique that combines multiple decision trees to improve accuracy and robustness by reducing variance through techniques like bootstrapping and feature randomness.

Finally, the unit focuses on Support Vector Machines (SVM), which aim to find the optimal hyperplane that maximizes the margin between different classes. The concept of support vectors and margin maximization is emphasized as the key strength of SVM. The unit also highlights the importance of the kernel trick, which enables SVM to handle non-linear data by transforming it into higher-dimensional space.

Overall, this unit builds a strong conceptual and practical understanding of major supervised learning algorithms, comparing their working principles, strengths, and limitations. It equips learners with the ability to select appropriate models for different types of classification problems and understand the mathematical intuition behind them.

10.5 ANSWERS

☛ Check Your Progress 1

Q1 What is the main difference between supervised and unsupervised learning?

Solution: Refer to Section 10.2

Q2 Which type of learning is used for predicting house prices and why?

Solution: Refer to Section 10.2

Q3 Give one real-life example of unsupervised learning.

Solution: Refer to Section 10.2

Q4 Why is data preprocessing important in machine learning?

Solution: Refer to Section 10.2

Q5 How is supervised learning similar to learning with a teacher?

Solution: Refer to Section 10.2

☛ Check Your Progress 2

Q1. What is Supervised Learning and how does it support predictive analytics?

Solution: Refer to Section 10.0

Q2. What are regression and classification problems in supervised learning?

Solution: Refer to Section 10.0

Q3. Differentiate between supervised and unsupervised learning..

Solution: Refer to Section 10.2

Q4. Explain the workflow of supervised learning.

Solution: Refer to Section 10.2

Q5. List and briefly explain any three supervised learning models

Solution: Refer to Section 10.3

Q6. What factors should be considered while selecting a supervised learning model?

Solution: Refer to Section 10.3

☛ Check Your Progress 3

Q1. What is the K-Nearest Neighbors (KNN) algorithm? Explain its basic principle..

Solution: Refer to Sub-section 10.3.1

Q2. Explain the working steps of the KNN algorithm.

Solution: Refer to Sub-section 10.3.1

Q3. What is the role of the parameter K in KNN?.

Solution: Refer to Sub-section 10.3.1

Q4. Why is feature scaling important in KNN?.

Solution: Refer to Sub-section 10.3.1

Q5. What are the advantages and limitations of KNN?

Solution: Refer to Sub-section 10.3.1

☛ Check Your Progress 4

Q1. What is the Naïve Bayes algorithm? Why is it called “naïve”?

Solution: Refer to Sub-section 10.3.2

Q2. State Bayes’ Theorem and explain its components.

Solution: Refer to Sub-section 10.3.2

Q3. Explain the working of the Naïve Bayes classifier.

Solution: Refer to Sub-section 10.3.2

Q4. What are the advantages of the Naïve Bayes algorithm?

Solution: Refer to Sub-section 10.3.2

Q5. What are the limitations of the Naïve Bayes algorithm?

Solution: Refer to Sub-section 10.3.2

☛ Check Your Progress 5

Q1. What is Logistic Regression? How is it different from Linear Regression?

Solution: Refer to Sub-section 10.3.3

Q2. Explain the sigmoid (logistic) function used in Logistic Regression.

Solution: Refer to Sub-section 10.3.3

Q3. How does Logistic Regression perform classification?

Solution: Refer to Sub-section 10.3.3

Q4. What are the advantages of Logistic Regression?.

Solution: Refer to Sub-section 10.3.3

Q5. What are the limitations of Logistic Regression?

Solution: Refer to Sub-section 10.3.3

Check Your Progress 6

Q1. What is a Decision Tree? Explain its basic structure.

Solution: Refer to Sub-section 10.3.4

Q2. How does a Decision Tree work in classification problems?

Solution: Refer to Sub-section 10.3.4

Q3. What are the advantages of Decision Trees?

Solution: Refer to Sub-section 10.3.4

Q4. What are the limitations of Decision Trees?.

Solution: Refer to Sub-section 10.3.4

Q5. What is overfitting in Decision Trees and how can it be controlled?

Solution: Refer to Sub-section 10.3.4

Check Your Progress 7

Q1. What is Random Forest? Explain its basic concept..

Solution: Refer to Sub-section 10.3.5

Q2. How does Random Forest improve over Decision Trees?

Solution: Refer to Sub-section 10.3.5

Q3. Explain the working of Random Forest..

Solution: Refer to Sub-section 10.3.5

Q4. What are the advantages of Random Forest?.

Solution: Refer to Sub-section 10.3.5

Q5. What are the limitations of Random Forest?

Solution: Refer to Sub-section 10.3.5

Check Your Progress 8

Q1. What is a Support Vector Machine (SVM)? Explain its objective..

Solution: Refer to Sub-section 10.3.6

Q2. What is a hyperplane and margin in SVM?

Solution: Refer to Sub-section 10.3.6

Q3. What are Support Vectors? Why are they important?

Solution: Refer to Sub-section 10.3.6

Q4. How does SVM handle non-linear data?

Solution: Refer to Sub-section 10.3.6

Q5. What are the advantages and limitations of SVM?

Solution: Refer to Sub-section 10.3.6

10.6 REFERENCES/FURTHER READINGS

1. *Pattern Recognition and Machine Learning* by Christopher M. Bishop, published by Springer (1st Edition, 2006, ISBN: 978-0387310732).
2. *Machine Learning* by Tom M. Mitchell, published by McGraw-Hill (1st Edition, 1997, ISBN: 978-0070428072).

3. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* by Aurélien Géron, published by O'Reilly Media (2nd Edition, 2019, ISBN: 978-1492032649).
4. *Data Mining: Concepts and Techniques* by Jiawei Han, Micheline Kamber, and Jian Pei, published by Morgan Kaufmann (3rd Edition, 2011, ISBN: 978-0123814791)
5. *Practical Statistics for Data Scientists* by Peter Bruce, Andrew Bruce, and Peter Gedeck, published by O'Reilly Media (2nd Edition, 2020, ISBN: 978-1492072942).
6. *Business Analytics: Data Analysis and Decision Making* by S. Christian Albright and Wayne L. Winston, published by Cengage Learning (7th Edition, 2020, ISBN: 978-0357131787).
7. *Decision Analysis for Management Judgment* by Paul Goodwin and George Wright, published by Wiley (5th Edition, 2014, ISBN: 978-1119974017).
8. *Handbook of Statistical Analysis and Data Mining Applications* by Robert Nisbet, John Elder, and Gary Miner, published by Elsevier (2nd Edition, 2017, ISBN: 978-0124166455).
9. *Data Mining and Analysis: Fundamental Concepts and Algorithms* by Mohammed J. Zaki and Wagner Meira Jr., published by Cambridge University Press (1st Edition, 2014, ISBN: 978-0521766333).
10. *Core Concepts in Data Analysis: Summarization, Correlation and Visualization* by Boris Mirkin, published by Springer (1st Edition, 2011, ISBN: 978-0857292872).
11. *Applied Predictive Modeling*, Max Kuhn and Kjell Johnson, 1st Edition, Springer, 2013.
12. *An Introduction to Statistical Learning with Applications in R*, Gareth James, Daniela Witten, Trevor Hastie and Robert Tibshirani, 1st Edition, Springer, 2013.
13. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, Trevor Hastie, Robert Tibshirani and Jerome Friedman, 2nd Edition, Springer, 2009.
14. *An Introduction to Statistical Learning with Applications in R* by Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani, published by Springer (2nd Edition, 2021, ISBN: 978-1071614174).
15. *Time Series Analysis: Forecasting and Control* by George E. P. Box, Gwilym M. Jenkins, Gregory C. Reinsel, and Greta M. Ljung, published by Wiley (5th Edition, 2015, ISBN: 978-1118675021)

UNIT 11 –UNSUPERVISED LEARNING MODELS

11.0 Introduction

11.1 Expected Learning Outcomes

11.2 Unsupervised Learning Models

11.2.1 K-Means Clustering

11.2.2 Hierarchical Clustering

11.2.3 Agglomerative Clustering

11.2.4 Association Rules

11.3 Summary

11.4 Answers

11.5 References/Further Readings

11.0 INTRODUCTION

In the previous units, the focus was on supervised learning techniques where models are trained using labeled data to predict outcomes. However, in many real-world scenarios, labeled data is not available, and the objective is to discover hidden patterns, structures, or relationships within the data itself. This is where **unsupervised learning** plays a crucial role. Unsupervised learning models are designed to analyse and interpret datasets without predefined outputs, enabling the extraction of meaningful insights from raw and unstructured data.

Unsupervised learning is widely used in areas such as customer segmentation, market basket analysis, anomaly detection, and pattern recognition. Unlike supervised learning, where the model learns from known outcomes, unsupervised learning explores the inherent structure of the data. It groups similar data points together, identifies associations among variables, and reveals underlying distributions. This makes it an essential tool in data analysis, especially when dealing with large volumes of unlabelled data.

This unit introduces the fundamental concepts and techniques of unsupervised learning models for data analysis. It begins with an overview of different unsupervised learning approaches and then focuses on clustering techniques such as **K-Means Clustering**, **Hierarchical Clustering**, and **Agglomerative Clustering**, which are used to group similar data points into clusters based on their characteristics. These methods help in identifying natural groupings within the data and are widely applied in business and scientific domains.

In addition to clustering, the unit also covers **Association Rule Learning**, which is used to discover interesting relationships and patterns among variables in large datasets. This technique is particularly useful in applications like market basket analysis, where it helps in identifying combinations of items frequently purchased together.

Overall, this unit aims to build a strong conceptual understanding of unsupervised learning models and their applications in data analysis. By the end of this unit, learners will be able to

understand how patterns are discovered without labelled data and how different unsupervised techniques can be applied to solve real-world problems.

11.1 EXPECTED LEARNING OUTCOMES

After completing this unit, you will be able to:

- understand the concept and significance of unsupervised learning in data analysis.
- explore different unsupervised learning models and their applications.
- learn the working and implementation of K-Means Clustering technique.
- understand Hierarchical and Agglomerative Clustering methods for grouping data.
- analyse patterns and relationships using Association Rule Learning.
- develop the ability to apply clustering and association techniques to real-world datasets.

11.2 UNSUPERVISED LEARNING MODELS

Unsupervised learning models play a crucial role in data analysis by enabling the discovery of hidden patterns, structures, and relationships within data that does not contain predefined labels. Unlike supervised learning, where the outcome is known in advance, unsupervised learning focuses on exploring the inherent organization of the data. This makes it particularly useful in real-world scenarios such as customer segmentation, pattern recognition, and market analysis, where labelled data is often unavailable or difficult to obtain.

One of the most widely used unsupervised learning techniques is **K-Means Clustering**, which aims to partition the dataset into a predefined number of clusters. It works by assigning data points to clusters in such a way that the distance between points within the same cluster is minimized, while the distance between different clusters is maximized. This method is simple, efficient, and highly effective for large datasets, making it a popular choice for grouping similar data points based on their characteristics.

Another important approach is **Hierarchical Clustering**, which builds a hierarchy of clusters rather than forming them in a single step. It organizes data into a tree-like structure called a dendrogram, allowing us to visualize how clusters are formed at different levels of similarity. This method does not require the number of clusters to be specified in advance, providing flexibility in understanding the data structure.

A specific type of hierarchical clustering is **Agglomerative Clustering**, which follows a bottom-up approach. In this method, each data point initially forms its own cluster, and clusters are gradually merged based on similarity until a single cluster is formed or a stopping condition is met. This approach is intuitive and useful for understanding how smaller groups combine to form larger clusters.

In addition to clustering techniques, **Association Rule Learning** is another important unsupervised method that focuses on discovering relationships between variables in large datasets. It identifies patterns such as items that frequently occur together, making it highly valuable in applications like market basket analysis. For example, it can reveal that customers who purchase one product are likely to purchase another, helping businesses make informed decisions.

Together, these unsupervised learning models provide powerful tools for analyzing data without labelled outcomes. While K-Means focuses on partitioning data into distinct groups, hierarchical and agglomerative clustering offer a more structured view of relationships among data points, and association rules uncover meaningful connections between variables. These techniques form the foundation for deeper exploration, and each of them will be studied in detail in the subsequent sections of this unit.

☛ Check Your Progress 1

Q1. What is Unsupervised Learning? How does it differ from supervised learning?

.....
.....

Q2. List the applications of Unsupervised Learning in real-world scenarios.

.....
.....

Q3. What are Unsupervised Learning Models? Explain their significance in data analysis.

.....
.....

Q4. Briefly explain different types of Unsupervised Learning techniques.

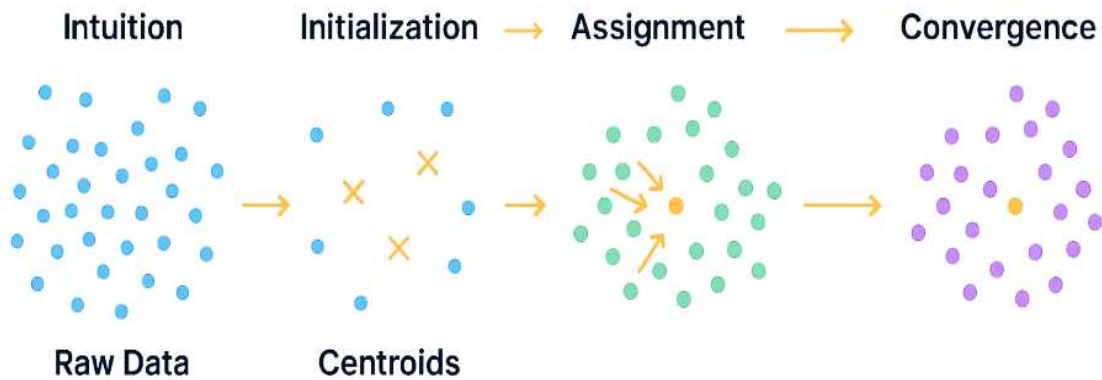
.....
.....

11.2.1 K-MEANS CLUSTERING

K-Means Clustering is one of the most widely used unsupervised learning algorithms for grouping data into clusters based on similarity. The main objective of K-Means is to partition a dataset into **K distinct clusters** such that data points within the same cluster are more similar to each other than to those in other clusters. Since it does not rely on labeled data, K-Means helps in discovering hidden patterns and structures within the dataset.

The algorithm works iteratively by assigning data points to clusters and updating the cluster centers, known as **centroids**, until convergence is achieved. Initially, K cluster centers are selected randomly. Each data point is then assigned to the nearest centroid based on a distance measure, typically the Euclidean distance. After assigning all points, new centroids are calculated as the mean of all points in each cluster. This process repeats until the centroids no longer change significantly.

K-MEANS CLUSTERING



The figure illustrates the working of the K-Means clustering algorithm in a step-by-step visual manner, helping to understand how clusters are formed iteratively. In the initial stage, a set of data points is scattered in the feature space, and a predefined number of cluster centers (centroids) are selected randomly. These initial centroids may not represent the actual grouping of data, but they serve as the starting point for the algorithm.

In the next stage, known as the **assignment step**, each data point is assigned to the nearest centroid based on a distance measure, typically the Euclidean distance. This results in the formation of preliminary clusters, where points closer to a particular centroid are grouped together. At this stage, the clusters may still be uneven or inaccurate because the centroids were chosen randomly.

Following this, the **update step** takes place, where the centroids are recalculated. Each centroid is repositioned to the mean (average) of all the data points assigned to its cluster. This shift in centroid location reflects a better representation of the cluster's center. As a result, the cluster boundaries begin to change.

The algorithm then repeats the assignment and update steps iteratively. With each iteration, data points may switch clusters as centroids move to more optimal positions. Gradually, the centroids stabilize, meaning their positions no longer change significantly. At this point, the clusters become fixed, and the algorithm converges to a final solution.

Overall, the figure demonstrates that K-Means clustering is an **iterative optimization process** where clusters are continuously refined by minimizing the distance between data points and their respective centroids. It visually emphasizes how initial random groupings evolve into well-defined clusters through repeated reassignment and centroid updates, making the underlying concept of the algorithm easy to understand.

Mathematically, the objective of K-Means is to minimize the following function:

$$J = \sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|^2$$

where K is the number of clusters, C_i represents the i^{th} cluster, x is a data point, and μ_i is the centroid of cluster C_i . The term $\|x - \mu_i\|^2$ represents the squared Euclidean distance between a point and its cluster centroid. The algorithm aims to minimize this total within-cluster variance.

To understand this concept intuitively, consider a real-life example of customer segmentation in a shopping mall. Customers can be grouped based on their spending behaviour and visit frequency. K-Means will automatically group customers into clusters such as high spenders, moderate spenders, and low spenders. This helps businesses design targeted marketing strategies.

Let us now understand K-Means with a numerical example.

Consider the following data points:

$$(2,2), (4,4), (6,6), (8,8)$$

Let the number of clusters be $K = 2$.

Step 1: Initialize Centroids

Assume initial centroids:

$$\mu_1 = (2,2), \quad \mu_2 = (8,8)$$

Step 2: Assign Points to Nearest Centroid

Distance formula:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Now calculate distances:

For point (4,4):

Distance to μ_1 :

$$\sqrt{(4-2)^2 + (4-2)^2} = \sqrt{4+4} = \sqrt{8} \approx 2.83$$

Distance to μ_2 :

$$\sqrt{(4-8)^2 + (4-8)^2} = \sqrt{16+16} = \sqrt{32} \approx 5.66$$

So, (4,4) belongs to Cluster 1.

For point (6,6):

Distance to μ_1 :

$$\sqrt{(6-2)^2 + (6-2)^2} = \sqrt{16+16} = \sqrt{32} \approx 5.66$$

Distance to μ_2 :

$$\sqrt{(6-8)^2 + (6-8)^2} = \sqrt{4+4} = \sqrt{8} \approx 2.83$$

So, (6,6) belongs to Cluster 2.

Step 3: Form Clusters

Cluster 1: (2,2), (4,4)

Cluster 2: (6,6), (8,8)

Step 4: Recalculate Centroids

$$\mu_1 = \left(\frac{2+4}{2}, \frac{2+4}{2} \right) = (3,3)$$

$$\mu_2 = \left(\frac{6+8}{2}, \frac{6+8}{2} \right) = (7,7)$$

Step 5: Reassign Points

Now check again—assignments remain same.

So, algorithm converges.

Final Clusters are Cluster 1: (2,2), (4,4) and Cluster 2: (6,6), (8,8) Thus, the Final centroids are: (3,3) and (7,7)

The algorithm successfully divided the dataset into two clusters based on proximity. Points closer to (3,3) belong to one group, and points closer to (7,7) belong to another. This demonstrates how K-Means groups similar data points together. In real-world scenarios, such clustering helps in identifying patterns such as grouping customers, documents, or products based on similarity.

Let's understand the working of K-Means algorithm with one more Numerical

Example 1: Apply the K-Means Clustering algorithm to group the following data points into 2 clusters. The given data points are: A(1,1), B(2,1), C(4,3), D(5,4). Assume the initial centroids are: $C_1 = (1,1)$, $C_2 = (5,4)$.

Using the Euclidean distance measure, perform the clustering step by step until the centroids do not change. Also write the final clusters and interpret the result.

Solution:

In this problem, we have to divide the given data points into **K = 2 clusters** using the K-Means algorithm.

The algorithm works by:

- choosing initial centroids,
- assigning each point to the nearest centroid,
- recalculating the centroids,
- repeating the process until no change occurs.

Formula Used in K-Means Clustering is as follows:

(a) Euclidean Distance Formula

The distance between a data point (x_1, y_1) and a centroid (x_2, y_2) is:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

(b) Centroid Formula

The centroid of a cluster is the mean of all points in that cluster.

$$\mu = \left(\frac{\sum x}{n}, \frac{\sum y}{n} \right)$$

Where,

- K : number of clusters
- C_1, C_2 : centroids of cluster 1 and cluster 2
- d : distance between a point and a centroid
- (x, y) : coordinates of a data point
- μ : new centroid of a cluster
- n : number of points in a cluster

Given Data:

- Data points: $A(1,1)$, $B(2,1)$, $C(4,3)$, $D(5,4)$
- Initial centroids: $C_1 = (1,1)$, $C_2 = (5,4)$

The stepwise solution is as follows:

Iteration 1: Assign Points to Nearest Centroid

We now calculate the distance of each point from both centroids.

Point A(1,1)

Distance from $C_1 = (1,1)$:

$$d(A, C_1) = \sqrt{(1-1)^2 + (1-1)^2} = \sqrt{0+0} = 0$$

Distance from $C_2 = (5,4)$:

$$d(A, C_2) = \sqrt{(1-5)^2 + (1-4)^2} = \sqrt{16+9} = \sqrt{25} = 5$$

Since $0 < 5$, point A belongs to **Cluster 1**.

Point B(2,1)

Distance from $C_1 = (1,1)$:

$$d(B, C_1) = \sqrt{(2-1)^2 + (1-1)^2} = \sqrt{1+0} = 1$$

Distance from $C_2 = (5,4)$:

$$d(B, C_2) = \sqrt{(2-5)^2 + (1-4)^2} = \sqrt{9+9} = \sqrt{18} \approx 4.24$$

Since $1 < 4.24$, point B belongs to **Cluster 1**.

Point C(4,3)

Distance from $C_1 = (1,1)$:

$$d(C, C_1) = \sqrt{(4-1)^2 + (3-1)^2} = \sqrt{9+4} = \sqrt{13} \approx 3.61$$

Distance from $C_2 = (5,4)$:

$$d(C, C_2) = \sqrt{(4-5)^2 + (3-4)^2} = \sqrt{1+1} = \sqrt{2} \approx 1.41$$

Since $1.41 < 3.61$, point C belongs to **Cluster 2**.

Point D(5,4)

Distance from $C_1 = (1,1)$:

$$d(D, C_1) = \sqrt{(5-1)^2 + (4-1)^2} = \sqrt{16+9} = \sqrt{25} = 5$$

Distance from $C_2 = (5,4)$:

$$d(D, C_2) = \sqrt{(5-5)^2 + (4-4)^2} = 0$$

Since $0 < 5$, point D belongs to **Cluster 2**.

Clusters After Iteration 1

So the clusters formed are:

Cluster 1: A(1,1), B(2,1)

Cluster 2: C(4,3), D(5,4)

Recalculate New Centroids

Now we compute the new centroid of each cluster.

New Centroid of Cluster 1

Points in Cluster 1: (1,1), (2,1)

Using centroid formula:

$$C'_1 = \left(\frac{1+2}{2}, \frac{1+1}{2} \right)$$

$$C'_1 = \left(\frac{3}{2}, \frac{2}{2} \right) = (1.5, 1)$$

New Centroid of Cluster 2

Points in Cluster 2: (4,3), (5,4)

$$C'_2 = \left(\frac{4+5}{2}, \frac{3+4}{2} \right)$$

$$C'_2 = \left(\frac{9}{2}, \frac{7}{2} \right) = (4.5, 3.5)$$

Iteration 2: Reassign Points Using New Centroids

Now use the new centroids:

$$C_1 = (1.5, 1), C_2 = (4.5, 3.5)$$

Point A(1,1)

$$d(A, C_1) = \sqrt{(1-1.5)^2 + (1-1)^2} = \sqrt{0.25} = 0.5$$

$$d(A, C_2) = \sqrt{(1-4.5)^2 + (1-3.5)^2} = \sqrt{12.25 + 6.25} = \sqrt{18.5} \approx 4.30$$

So A remains in **Cluster 1**.

Point B(2,1)

$$d(B, C_1) = \sqrt{(2-1.5)^2 + (1-1)^2} = \sqrt{0.25} = 0.5$$

$$d(B, C_2) = \sqrt{(2-4.5)^2 + (1-3.5)^2} = \sqrt{6.25 + 6.25} = \sqrt{12.5} \approx 3.54$$

So B remains in **Cluster 1**.

Point C(4,3)

$$d(C, C_1) = \sqrt{(4 - 1.5)^2 + (3 - 1)^2} = \sqrt{6.25 + 4} = \sqrt{10.25} \approx 3.20$$

$$d(C, C_2) = \sqrt{(4 - 4.5)^2 + (3 - 3.5)^2} = \sqrt{0.25 + 0.25} = \sqrt{0.5} \approx 0.71$$

So C remains in **Cluster 2**.

Point D(5,4)

$$d(D, C_1) = \sqrt{(5 - 1.5)^2 + (4 - 1)^2} = \sqrt{12.25 + 9} = \sqrt{21.25} \approx 4.61$$

$$d(D, C_2) = \sqrt{(5 - 4.5)^2 + (4 - 3.5)^2} = \sqrt{0.25 + 0.25} = \sqrt{0.5} \approx 0.71$$

So D remains in **Cluster 2**.

Check for Convergence: After the second iteration, the cluster assignments did not change. Therefore, the algorithm has **converged**.

So, the Final Answer is:

Final Cluster 1: A(1,1), B(2,1)

Final Cluster 2: C(4,3), D(5,4)

and

Final Centroids: $C_1 = (1.5, 1)$, $C_2 = (4.5, 3.5)$

Results of the above numerical showcased that the K-Means algorithm successfully divided the given four points into two meaningful clusters based on their distance from the centroids. Points A(1,1) and B(2,1) are closer to each other, so they form one cluster. Similarly, points C(4,3) and D(5,4) are grouped into another cluster because they lie close together in the feature space.

This result shows that K-Means works by identifying natural groupings in the data. In real-life applications, such grouping can be useful for customer segmentation, document clustering, image compression, and pattern recognition. The final centroids represent the center of each cluster and summarize the overall position of points in that group.

To solve a K-Means numerical, always follow this sequence:

- First, note the value of K , the number of clusters.
- Second, write the initial centroids.
- Third, calculate the distance of each point from every centroid.
- Fourth, assign each point to the nearest centroid.
- Fifth, recompute the centroids by taking the mean of all points in each cluster.
- Sixth, repeat the process until the assignments no longer change.

So the complete flow is:

Initial Centroids → Distance Calculation → Cluster Assignment → Centroid Update
→ Repeat

This repeated reassignment and updating is the core idea of K-Means clustering.

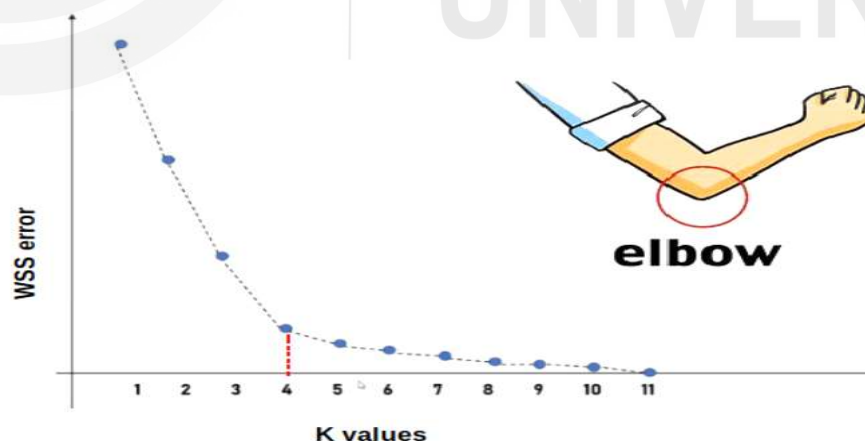
Before concluding this section, we are going to discuss an important method called “Elbow Method” The brief description of this method is as follows:

Elbow Method in K-Means Clustering: The Elbow Method is a widely used technique to determine the optimal number of clusters (K) in K-Means Clustering. Since K-Means requires the number of clusters to be specified beforehand, the elbow method helps in identifying the most suitable value of K by evaluating how well the data fits into different cluster groupings. The method is based on minimizing the Within-Cluster Sum of Squares (WCSS), which measures the total squared distance between data points and their respective cluster centroids. Mathematically, WCSS is expressed as:

$$WCSS = \sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|^2$$

As the number of clusters increases, the value of WCSS decreases because data points are assigned to clusters whose centroids are closer to them. Initially, this decrease is rapid, as adding more clusters significantly improves the grouping of data. However, after a certain point, the reduction in WCSS becomes slower, indicating that adding more clusters does not contribute much to improving the model. This point of diminishing returns is referred to as the “Elbow point.”

Elbow method



In the graph:

- The **X-axis** represents the number of clusters K .
- The **Y-axis** represents the WCSS (error).
- As K increases, WCSS decreases sharply at first.

- After a certain value of K , the decrease becomes gradual.

The point where the curve bends like an “elbow” indicates the best value of K .

In a typical elbow method graph, the number of clusters (K) is plotted on the X-axis, and the WCSS value is plotted on the Y-axis. The curve shows a sharp decline initially and then gradually levels off. The point where the curve bends or forms an “elbow” represents the optimal number of clusters. This is because it balances the trade-off between minimizing error and avoiding unnecessary complexity.

Example 2: To understand this with a numerical example, consider a dataset where the WCSS values are calculated for different values of K .

K (Number of Clusters)	WCSS
1	500
2	300
3	200
4	180
5	170

Solution: From the above table it can be analysed that:

- From $K = 1$ to $K = 2$, WCSS decreases significantly ($500 \rightarrow 300$).
- From $K = 2$ to $K = 3$, there is still a large reduction ($300 \rightarrow 200$).
- From $K = 3$ onwards, the decrease becomes small ($200 \rightarrow 180 \rightarrow 170$).

Thus, the major improvement stops after $K = 3$. So, Optimal number of clusters $K = 3$

Table shown in the above example showcased that for $K = 1$, WCSS = 500; for $K = 2$, WCSS = 300; for $K = 3$, WCSS = 200; for $K = 4$, WCSS = 180; and for $K = 5$, WCSS = 170. It can be observed that there is a significant reduction in WCSS from $K = 1$ to $K = 3$, but after $K = 3$, the decrease becomes minimal.

Therefore, the optimal number of clusters is $K = 3$, as it provides a good balance between accuracy and simplicity.

From an interpretational perspective, the elbow point indicates the division of the data into three clusters which captures most of the inherent structure in the dataset. Increasing the number of clusters beyond this point leads to only marginal improvements while increasing computational complexity. Thus, the elbow method helps prevent both underfitting (too few clusters) and overfitting (too many clusters).

Thus, the elbow method can be understood as a practical way to decide how many groups should be formed in the data. It emphasizes finding a balance where the clusters are meaningful without making the model unnecessarily complex. Overall, the elbow method is a

simple yet effective tool for selecting the optimal value of K in K-Means clustering and plays a crucial role in improving clustering performance.

The advantages and Limitations of K-Means algorithm are as follows:

Advantages of K-Means Clustering

- K-Means is simple and easy to implement.
- It is computationally efficient and works well with large datasets.
- The algorithm converges quickly and is easy to understand.
- It is widely used in practical applications such as customer segmentation and image compression.

Limitations of K-Means Clustering

- K-Means requires the number of clusters (K) to be specified in advance, which may not always be known.
- It is sensitive to the initial selection of centroids, which can affect the final result.
- It does not perform well with clusters of different shapes and sizes.
- It is also sensitive to outliers and noise in the data.

We learned that the K-Means can be understood as a process of repeatedly grouping and adjusting until the best cluster centers are found. The key idea is minimizing the distance between data points and their respective centroids. The algorithm alternates between assigning points to clusters and updating centroids, gradually improving the clustering quality. This iterative refinement makes K-Means a powerful yet simple technique for unsupervised learning.

Also, it is learned the K-Means Clustering is a foundational algorithm in unsupervised learning that helps in discovering natural groupings within data. By minimizing within-cluster variance and iteratively updating centroids, it provides a practical and efficient way to analyze unlabeled datasets. Despite its limitations, its simplicity and effectiveness make it one of the most popular clustering techniques in machine learning.

☛ Check Your Progress 2

Q1. What is K-Means Clustering? Explain its objective.

.....
.....

Q2. Explain the working steps of the K-Means algorithm

.....
.....

Q3. What is the role of centroid in K-Means Clustering?.

.....
.....

Q4. Explain the Elbow Method for selecting optimal number of clusters (K)..

11.2.2 HIERARCHICAL CLUSTERING

Hierarchical Clustering is an important unsupervised learning algorithm used to group similar data points into clusters based on their similarity or distance. Unlike K-Means Clustering, where the number of clusters must be specified in advance, hierarchical clustering builds a **tree-like structure (called a dendrogram)** that represents the arrangement of clusters at different levels of similarity. This allows us to explore the data at multiple levels and choose the number of clusters based on interpretation.

Hierarchical clustering works in two main ways:

- **Agglomerative (bottom-up approach)** and
- **Divisive (top-down approach).**

In this unit, the focus is primarily on the agglomerative approach, where each data point initially forms its own cluster, and clusters are gradually merged step by step based on similarity until all points belong to a single cluster.

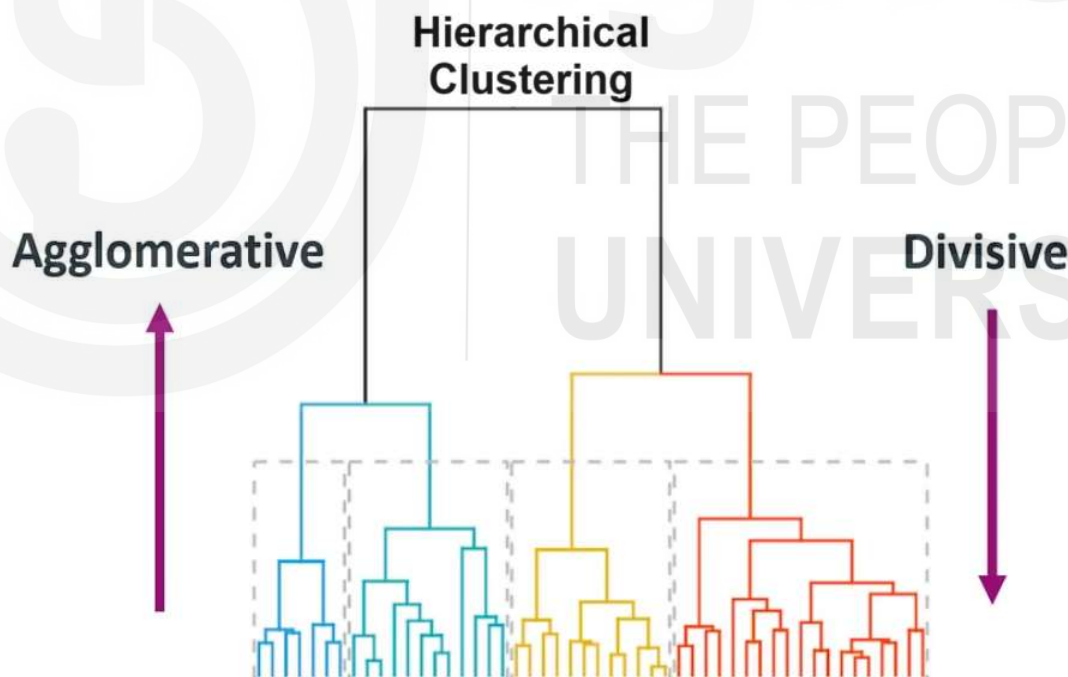


Figure: Dendrogram

The figure shows a **dendrogram**, which is a tree-like diagram used to visualize hierarchical clustering. Each leaf node represents a data point, and branches represent the merging of clusters. The height at which two clusters merge indicates the distance between them. By cutting the dendrogram at a certain level, we can obtain the desired number of clusters.

The agglomerative hierarchical clustering algorithm follows these steps:

- Initially, each data point is treated as a separate cluster.
- Then, the distance between all pairs of clusters is calculated.
- The two closest clusters are merged into one cluster.
- After merging, the distance matrix is updated, and the process repeats.
- This continues until all data points are grouped into a single cluster or until a stopping criterion is reached.

To determine how clusters are merged, hierarchical clustering uses distance measures and linkage criteria.

(a) Distance Measure (Euclidean Distance)

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

(b) Linkage Methods

- Single Linkage: distance between closest points of clusters
- Complete Linkage: distance between farthest points
- Average Linkage: average distance between all points
- Centroid Linkage: distance between cluster centroids

To understand this clearly, consider grouping students based on their marks in subjects. Initially, each student is treated as an individual cluster. Students with similar marks are grouped together step by step, forming larger clusters. Finally, we obtain groups such as high performers, average performers, and low performers. The dendrogram helps visualize how these groups are formed.

Example 3: Consider the following data points: $A(1,1)$, $B(2,1)$, $C(4,3)$; apply Hierarchical clustering and identify the clusters.

Solution:

Step 1: Compute Distance Matrix

Using Euclidean distance:

$$d(A, B) = \sqrt{(1 - 2)^2 + (1 - 1)^2} = 1$$

$$d(A, C) = \sqrt{(1 - 4)^2 + (1 - 3)^2} = \sqrt{9 + 4} = \sqrt{13} \approx 3.61$$

$$d(B, C) = \sqrt{(2 - 4)^2 + (1 - 3)^2} = \sqrt{4 + 4} = \sqrt{8} \approx 2.83$$

Step 2: Merge Closest Clusters

The smallest distance is:

$$d(A, B) = 1$$

So merge A and B:

$$\text{Cluster 1} = \{A, B\}$$

Step 3: Update Distances

Now calculate distance between cluster $\{A, B\}$ and point C using **single linkage**:

$$d(\{A, B\}, C) = \min(d(A, C), d(B, C))$$

$$d = \min(3.61, 2.83) = 2.83$$

Step 4: Final Merge

Merge $\{A, B\}$ and C . Thus, the Final Cluster = $\{A, B, C\}$

Final Clustering Structure

Step 1: A and B merged

Step 2: (A,B) merged with C

The algorithm first grouped A and B because they are closest to each other. Then, it added C to this cluster at a higher distance level. This shows that A and B are more similar compared to C. The dendrogram visually represents this relationship, where A and B merge earlier than C.

The advantages and Limitation of Hierarchical clustering are as follows:

Advantages of Hierarchical Clustering

- Hierarchical clustering does not require the number of clusters to be specified in advance.
- It provides a clear visual representation through dendrograms.
- It is flexible and can capture complex relationships between data points.
- It is useful for exploratory data analysis.

Limitations of Hierarchical Clustering

- The algorithm is computationally expensive for large datasets.
- Once a merge or split is done, it cannot be undone.
- It is sensitive to noise and outliers. The results may vary depending on the choice of linkage method.

So, Hierarchical clustering can be understood as a process of gradually grouping similar data points step by step. Instead of directly forming clusters, it builds a hierarchy of clusters that can be visualized and interpreted at different levels. The dendrogram helps in deciding how many clusters should be formed by cutting the tree at a suitable level.

This section concludes with the understanding that the Hierarchical clustering is a powerful technique for analysing data structure without predefined labels. By building a hierarchy of clusters and representing it through dendrograms, it provides deep insights into relationships among data points. Despite its computational limitations, it remains a valuable tool for clustering and exploratory analysis in machine learning.

In the subsequent section we extend our discussion to Agglomerative Clustering in more detail.

☛ Check Your Progress 3

Q1. What is Hierarchical Clustering? Explain its key concept..

.....

.....

Q2. Differentiate between Agglomerative and Divisive approaches in Hierarchical Clustering.

.....

.....

Q3. What is a dendrogram and how is it used in clustering?

.....

.....

Q4. Explain different linkage methods used in Hierarchical Clustering.

.....

.....

11.2.3 AGGLOMERATIVE CLUSTERING

Agglomerative Clustering is a type of hierarchical clustering that follows a **bottom-up approach**. This means the algorithm starts with the smallest possible units and gradually builds larger clusters. Initially, each data point is considered as a separate cluster. Then, the algorithm looks for the two clusters that are closest to each other and merges them into a single cluster. This merging process continues step by step until all data points are combined into one large cluster. The entire process can be visualized using a **dendrogram**, which shows how clusters are formed and merged at different levels.

In this approach, we often use a method called **single linkage** to measure the distance between clusters. According to single linkage, the distance between two clusters is defined as the **minimum distance between any two points**, where one point belongs to the first cluster and the other belongs to the second cluster. In simple terms, we look for the closest pair of points between two clusters to decide how far apart those clusters are.

To calculate distances, we use a distance formula. Since the data points in this example are one-dimensional, the distance between two points x_i and x_j is simply the absolute difference between them:

$$d(x_i, x_j) = |x_i - x_j|$$

This formula gives us how far apart two individual data points are.

When dealing with clusters instead of individual points, we use the single linkage formula:

$$d(C_1, C_2) = \min \{d(x_i, x_j)\}$$

Here, C_1 and C_2 are two clusters, and x_i and x_j are points belonging to those clusters. The distance between the clusters is the smallest distance among all possible pairs of points taken from the two clusters.

In terms of variables, x_i and x_j represent individual data points, $d(x_i, x_j)$ represents the distance between two points, C_1 and C_2 represent clusters, and $d(C_1, C_2)$ represents the distance between those clusters. The term “min” simply means we select the smallest distance among all possible point pairs.

Overall, agglomerative clustering can be understood as a process of repeatedly merging the closest data points or clusters, using simple distance calculations, until a complete hierarchy of clusters is formed.

In the above section we learned that Agglomerative Clustering is one of the most commonly used approaches in hierarchical clustering, following a bottom-up strategy. It begins by treating each data point as an individual cluster and then progressively merges the closest clusters step by step until all data points are grouped into a single cluster or a stopping criterion is reached. This gradual merging process forms a hierarchy of clusters, which can be visualized using a dendrogram.

Also, we understood that Agglomerative clustering is essentially a type of hierarchical clustering, and in most practical scenarios, when we refer to hierarchical clustering, we are often referring to agglomerative clustering. The key idea is to build a hierarchy by combining smaller clusters into larger ones based on similarity or distance. This makes it intuitive and easy to understand, as it mimics natural grouping processes where similar items are combined to form broader categories.

We learned in our earlier section that the two i.e. Agglomerative and Hierarchical Clustering are closely related in the sense that the Hierarchical clustering is a broader concept that includes two main approaches:

- **Agglomerative Clustering (Bottom-Up)**
- **Divisive Clustering (Top-Down)**

Agglomerative clustering is the **most widely used form** of hierarchical clustering. In this method, the hierarchy is built from the bottom by merging clusters. On the other hand, hierarchical clustering as a concept includes both merging (agglomerative) and splitting (divisive) strategies.

Thus, we can say that Hierarchical clustering is the overall concept whereas the Agglomerative clustering relates to a specific implementation (bottom-up approach)

Now, we extend our discussion on Agglomerative clustering by Comparing it with the Divisive Clustering

It is to be noted that, the Agglomerative clustering works as (bottom-up approach) in contrast to **divisive clustering**, which follows a **top-down approach**. In divisive clustering, all data points initially belong to a single cluster, and the algorithm repeatedly splits the cluster into smaller clusters until each data point becomes a separate cluster or a stopping condition is met.

The key differences between the two are as follows:

- Agglomerative clustering starts with many small clusters and merges them, whereas divisive clustering starts with one large cluster and divides it.
- Agglomerative clustering is more commonly used because it is computationally simpler, while divisive clustering is more complex and less frequently used in practice.
- Agglomerative clustering builds the hierarchy from bottom to top, whereas divisive clustering builds it from top to bottom.

here, we revisit the Working of Agglomerative Clustering, which follows the following steps:

Initially, each data point is treated as an individual cluster.

- The distance between all pairs of clusters is calculated.
- The two closest clusters are merged into one cluster.
- After merging, the distance matrix is updated based on the chosen linkage method.
- This process continues iteratively until all points are grouped into a single cluster or until a desired number of clusters is obtained.

We also learned that the Different linkage methods can be used to measure the distance between clusters, such as:

- Single linkage (minimum distance)
- Complete linkage (maximum distance)
- Average linkage (average distance)
- Centroid linkage (distance between centroids)

Example 4: Consider the following data points representing students' scores:

$$A(1,1), B(2,1), C(5,4), D(6,5)$$

Use Agglomerative Clustering to prepare clusters

Solution:

Step 1: Initial Clusters

Each point is its own cluster:

$$\{A\}, \{B\}, \{C\}, \{D\}$$

Step 2: Compute Distances

- Distance between A and B is small
- Distance between C and D is small

Step 3: Merge Closest Clusters

Merge A and B:

$$\{A, B\}, \{C\}, \{D\}$$

Merge C and D:

$$\{A, B\}, \{C, D\}$$

Step 4: Final Merge

Merge both clusters:

$$\{A, B, C, D\}$$

In this example, points A and B are grouped together first because they are closest to each other, and similarly, C and D form another cluster. Finally, these two clusters are merged at a higher distance level. This shows how agglomerative clustering builds a hierarchy of clusters step by step based on similarity.

If we represent this using a dendrogram, A and B will merge at a lower level, while the merge between the two larger clusters will occur at a higher level, indicating lesser similarity.

One more example for better understanding of Agglomerative clustering

Example 5: Apply the Agglomerative Clustering algorithm on the following four one-dimensional data points using single linkage distance:

$$A = 2, B = 4, C = 10, D = 12$$

Perform the clustering step by step until all points are merged into one cluster. Also show the order in which clusters are formed and interpret the result.

Solution:

Step 1: Write the Initial Clusters

Initially, each point is considered as a separate cluster:

$$\{A\} = \{2\}, \{B\} = \{4\}, \{C\} = \{10\}, \{D\} = \{12\}$$

So the initial clusters are:

$$\{2\}, \{4\}, \{10\}, \{12\}$$

Step 2: Compute the Distance Matrix

Using

$$d(x_i, x_j) = |x_i - x_j|$$

we calculate the pairwise distances.

$$\text{Distance between A and B: } d(A, B) = |2 - 4| = 2$$

$$\text{Distance between A and C: } d(A, C) = |2 - 10| = 8$$

$$\text{Distance between A and D: } d(A, D) = |2 - 12| = 10$$

$$\text{Distance between B and C: } d(B, C) = |4 - 10| = 6$$

$$\text{Distance between B and D: } d(B, D) = |4 - 12| = 8$$

$$\text{Distance between C and D: } d(C, D) = |10 - 12| = 2$$

So, the distance table is:

Pair	Distance
A, B	2
A, C	8

Pair	Distance
A, D	10
B, C	6
B, D	8
C, D	2

Step 3: Find the Smallest Distance and Merge the Closest Clusters

The minimum distance is: 2

This occurs for:

- A and B
- C and D

Since both are equally close, we can merge either pair first. Let us merge A and B first.

So the first merged cluster is:

$$\{A, B\} = \{2, 4\}$$

Now the clusters become:

$$\{2, 4\}, \{10\}, \{12\}$$

Step 4: Update Distances After First Merge

Now we compute the distance between cluster $\{A, B\}$ and the remaining points using single linkage.

Distance between $\{A, B\}$ and C

$$d(\{A, B\}, C) = \min \{d(A, C), d(B, C)\}$$

$$d(\{A, B\}, C) = \min \{8, 6\} = 6$$

Distance between $\{A, B\}$ and D

$$d(\{A, B\}, D) = \min \{d(A, D), d(B, D)\}$$

$$d(\{A, B\}, D) = \min \{10, 8\} = 8$$

Distance between C and D

$$d(C, D) = 2$$

So the updated distances are:

Pair	Distance
$\{A, B\}, C$	6
$\{A, B\}, D$	8
C, D	2

Step 5: Merge the Next Closest Clusters

Again, the smallest distance is: 2

So merge C and D: $\{C, D\} = \{10, 12\}$

Now the clusters become: $\{2, 4\}, \{10, 12\}$

Step 6: Compute Distance Between the Two New Clusters

Now calculate the distance between clusters $\{A, B\}$ and $\{C, D\}$ using single linkage.

$$d(\{A, B\}, \{C, D\}) = \min \{d(A, C), d(A, D), d(B, C), d(B, D)\}$$

Substitute the values:

$$d(\{A, B\}, \{C, D\}) = \min \{8, 10, 6, 8\}$$

$$d(\{A, B\}, \{C, D\}) = 6$$

Step 7: Final Merge

Now only two clusters remain:

$$\{2, 4\}, \{10, 12\}$$

Their distance is 6, so they are merged.

$$\{A, B, C, D\} = \{2, 4, 10, 12\}$$

Now all data points belong to one cluster.

The order of Final Cluster Formation is as follows:

The order of merging is:

1. A and B merge at distance 2
2. C and D merge at distance 2
3. $\{A, B\}$ and $\{C, D\}$ merge at distance 6

Dendrogram-Level Interpretation is as follows:

If we imagine the dendrogram:

- A and B join at height 2
- C and D join at height 2
- The two clusters $\{A, B\}$ and $\{C, D\}$ join at height 6

This indicates that A and B are very similar to each other, and C and D are also very similar to each other. However, the two resulting groups are farther apart from one another.

Thus, the Final Answer involves the Cluster merging sequence as follows:

$$\{2\}, \{4\}, \{10\}, \{12\}$$

$$\{2, 4\}, \{10\}, \{12\}$$

$$\{2, 4\}, \{10, 12\}$$

$$\{2, 4, 10, 12\}$$

Merge distances:

- A and B at distance 2
- C and D at distance 2
- $\{A, B\}$ and $\{C, D\}$ at distance 6

Results of the above numerical reflects that the Agglomerative Clustering algorithm first grouped the closest points together. Since $A = 2$ and $B = 4$ are very near, they formed one cluster. Similarly, $C = 10$ and $D = 12$ formed another cluster because they are also close. Finally, these two clusters merged at a larger distance of 6. This clearly shows that the data naturally forms **two smaller groups**: one around 2 and 4, and another around 10 and 12.

From a practical point of view, if we cut the dendrogram before the final merge, we would obtain two meaningful clusters:

$$\{2, 4\} \text{ and } \{10, 12\}$$

Thus, the numerical demonstrates how Agglomerative Clustering helps in discovering the hierarchical grouping structure in data.

To solve an Agglomerative Clustering numerical, always follow this order.

- First, treat each point as an individual cluster.
- Second, calculate the distance between all pairs of clusters.
- Third, merge the two nearest clusters.
- Fourth, update the distance matrix according to the chosen linkage method.
- Fifth, repeat the process until all points are merged into one cluster.

So the learning flow is:

Individual points → Compute distances → Merge nearest clusters → Update distances
→ Repeat

The main idea is that agglomerative clustering builds the hierarchy from the bottom upward by combining the most similar objects first.

The advantages and limitations of Agglomerative clustering are as follows:

Advantages of Agglomerative Clustering

- Agglomerative clustering does not require the number of clusters to be specified in advance.
- It provides a hierarchical structure that can be visualized using dendrograms.
- It is easy to understand and interpret.
- It is also flexible, as different linkage methods can be used to define similarity.

Limitations of Agglomerative Clustering

- Agglomerative clustering is computationally expensive and not suitable for very large datasets.
- Once clusters are merged, the process cannot be reversed.
- It is sensitive to noise and outliers, and the results may vary depending on the choice of linkage method.

We learned that the agglomerative clustering can be understood as a process of gradually combining similar data points into larger groups. It starts with the most basic units (individual points) and builds up a structure by merging the closest elements. This bottom-up approach makes it intuitive and closely aligned with how humans naturally group similar items.

Also, we understood that Agglomerative clustering is a fundamental technique within hierarchical clustering that builds clusters step by step from individual data points. By comparing it with divisive clustering, we understand two opposite strategies for forming clusters—merging versus splitting. Due to its simplicity and interpretability, agglomerative clustering is widely used in exploratory data analysis and forms a strong foundation for understanding clustering techniques in machine learning.

☛ Check Your Progress 4

Q1. What is Agglomerative Clustering? Explain its basic approach.

.....

.....

Q2. Explain the step-by-step working of Agglomerative Clustering.

.....

.....

Q3. What is single linkage in Agglomerative Clustering?

.....

.....

Q4. Differentiate between Agglomerative and Divisive Clustering.

.....

.....

11.2.4 ASSOCIATION RULES

Association Rule Learning is an important unsupervised learning technique used to discover interesting relationships, patterns, or associations among variables in large datasets. Unlike clustering, which groups similar data points, association rules focus on identifying **co-occurrence relationships**, that is, how items or events are related to each other. This technique is widely used in applications such as market basket analysis, recommendation systems, and customer behaviour analysis.

The basic idea of association rules is to find rules of the form:

$$X \Rightarrow Y$$

where X and Y are sets of items. This rule means that if itemset X occurs, then itemset Y is likely to occur as well. The goal is to identify such rules that are strong and meaningful based on certain evaluation measures.

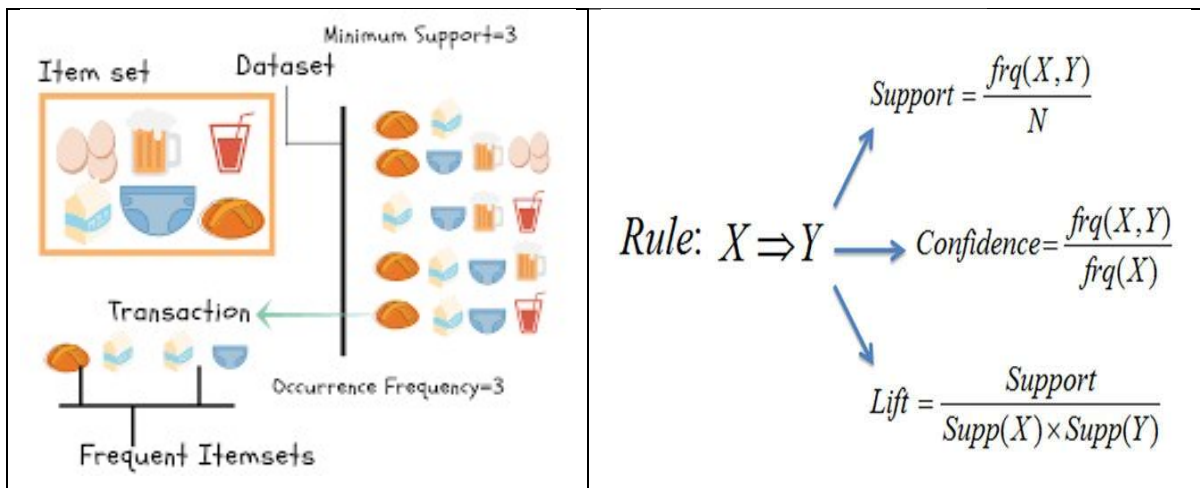


Figure: Association Rule Concept

The figure represents a typical **market basket analysis** scenario. For example, customers who buy bread may also buy butter. This relationship can be expressed as:

$$\text{Bread} \Rightarrow \text{Butter}$$

Such rules help businesses understand purchasing patterns and make better decisions such as product placement, cross-selling, and promotions.

Following are the Key Measures in Association Rules that are used to evaluate the strength of association rules, three important metrics are:

1. Support

Support measures how frequently an itemset appears in the dataset.

$$\text{Support}(X) = \frac{\text{Number of transactions containing } X}{\text{Total number of transactions}}$$

For a rule:

$$\text{Support}(X \Rightarrow Y) = \frac{\text{Transactions containing both } X \text{ and } Y}{\text{Total transactions}}$$

2. Confidence

Confidence measures how often items in Y appear in transactions that contain X .

$$\text{Confidence}(X \Rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

3. Lift

Lift measures how much more likely Y is to occur given X , compared to its general occurrence.

$$\text{Lift}(X \Rightarrow Y) = \frac{\text{Confidence}(X \Rightarrow Y)}{\text{Support}(Y)}$$

Example 6: Consider a supermarket where customers frequently buy items like bread, milk, and butter. By analysing transaction data, we may discover that customers who buy bread often buy butter as well. This insight can help the store place these items close to each other or offer bundled discounts.

Consider the following transaction dataset:

Transaction ID	Items Purchased
T1	Bread, Milk
T2	Bread, Butter
T3	Milk, Butter
T4	Bread, Milk, Butter
T5	Bread, Milk

Total transactions = 5

Solution:

Step 1: Calculate Support

Let us calculate support for:

Support(Bread)

Bread appears in T1, T2, T4, T5 → 4 transactions

$$\text{Support}(\text{Bread}) = \frac{4}{5} = 0.8$$

Support(Bread ∪ Butter)

Bread and Butter appear together in T2 and T4 → 2 transactions

$$\text{Support}(\text{Bread} \Rightarrow \text{Butter}) = \frac{2}{5} = 0.4$$

Step 2: Calculate Confidence

$$\begin{aligned} \text{Confidence}(\text{Bread} \Rightarrow \text{Butter}) &= \frac{\text{Support}(\text{Bread} \cup \text{Butter})}{\text{Support}(\text{Bread})} \\ &= \frac{0.4}{0.8} = 0.5 \end{aligned}$$

Step 3: Calculate Lift

First find Support(Butter):

Butter appears in T2, T3, T4 → 3 transactions

$$\text{Support}(\text{Butter}) = \frac{3}{5} = 0.6$$

Now,

$$\text{Lift}(\text{Bread} \Rightarrow \text{Butter}) = \frac{0.5}{0.6} \approx 0.83$$

Final Results

- Support = 0.4
- Confidence = 0.5
- Lift ≈ 0.83

The support value of 0.4 indicates that 40% of transactions contain both bread and butter. The confidence value of 0.5 means that 50% of customers who buy bread also buy butter. However, the lift value is less than 1, which suggests that buying bread does not strongly increase the likelihood of buying butter compared to random chance. Therefore, this rule is not very strong.

Let's explore the working of Association Rule with one more Numerical Example

Example 7: A mobile accessories store wants to analyze customer buying patterns using **Association Rule Learning**. The transaction data of 5 customers is given below:

Transaction ID	Items Purchased
T1	Mobile Cover, Earphones
T2	Mobile Cover, Charger
T3	Earphones, Charger
T4	Mobile Cover, Earphones, Charger
T5	Mobile Cover, Earphones

Using the above data, find the **Support, Confidence, and Lift** for the association rule:

Mobile Cover $\Rightarrow$ Earphones

Also interpret whether this rule is strong or not.

Solution: In this numerical, we have to study the rule: $X \Rightarrow Y$
where:

- X = Mobile Cover
- Y = Earphones

This rule means:

If a customer buys a Mobile Cover, then the customer is likely to buy Earphones.

To evaluate this rule, we calculate three important measures:

- **Support**
- **Confidence**
- **Lift**

Equations Used are as follows:

(a) Support of the Rule

Support tells us how frequently both items occur together in the total dataset.

$$Support(X \Rightarrow Y) = \frac{\text{Number of transactions containing both } X \text{ and } Y}{\text{Total number of transactions}}$$

(b) Confidence of the Rule

Confidence tells us how often Y occurs when X has already occurred.

$$Confidence(X \Rightarrow Y) = \frac{Support(X \cup Y)}{Support(X)}$$

It can also be written as:

$$Confidence(X \Rightarrow Y) = \frac{\text{Number of transactions containing both } X \text{ and } Y}{\text{Number of transactions containing } X}$$

(c) Lift of the Rule

Lift tells us how much more likely Y is to occur when X occurs, compared to the normal occurrence of Y .

$$Lift(X \Rightarrow Y) = \frac{Confidence(X \Rightarrow Y)}{Support(Y)}$$

Meaning of Variables

- X : antecedent itemset or starting item
- Y : consequent itemset or resulting item
- $X \cup Y$: transactions containing both X and Y
- $Support(X)$: proportion of transactions containing X
- $Support(Y)$: proportion of transactions containing Y
- $Confidence(X \Rightarrow Y)$: probability of buying Y when X is bought
- $Lift(X \Rightarrow Y)$: strength of association between X and Y

Given Data

Transaction ID	Items Purchased
T1	Mobile Cover, Earphones
T2	Mobile Cover, Charger
T3	Earphones, Charger
T4	Mobile Cover, Earphones, Charger
T5	Mobile Cover, Earphones

Total number of transactions:

$$N = 5$$

Rule to analyze:

Mobile Cover $\Rightarrow$ Earphones

Step 1: Find Transactions Containing Mobile Cover

Mobile Cover appears in: Transaction T1, T2, T4, T5

So, number of transactions containing Mobile Cover = 4

$$\text{Support}(\text{Mobile Cover}) = \frac{4}{5} = 0.8$$

Step 2: Find Transactions Containing Earphones

Earphones appear in: Transaction T1, T3, T4, T5

So, number of transactions containing Earphones = 4

$$\text{Support}(\text{Earphones}) = \frac{4}{5} = 0.8$$

Step 3: Find Transactions Containing Both Mobile Cover and Earphones

Mobile Cover and Earphones appear together in: Transaction T1, T4, T5

So, number of transactions containing both = 3

$$\text{Support}(\text{Mobile Cover} \Rightarrow \text{Earphones}) = \frac{3}{5} = 0.6$$

Thus, the support of the rule is: 0.6 or 60%

Step 4: Calculate Confidence

Using the formula: $\text{Confidence}(\text{Mobile Cover} \Rightarrow \text{Earphones}) = \frac{\text{Support}(\text{Mobile Cover} \cup \text{Earphones})}{\text{Support}(\text{Mobile Cover})}$

Substitute the values:

$$\text{Confidence}(\text{Mobile Cover} \Rightarrow \text{Earphones}) = \frac{0.6}{0.8}$$

$$\text{Confidence}(\text{Mobile Cover} \Rightarrow \text{Earphones}) = 0.75$$

Thus, the confidence of the rule is: 0.75 or 75%

Step 5: Calculate Lift

Using the formula:

$$Lift(\text{Mobile Cover} \Rightarrow \text{Earphones}) = \frac{Confidence(\text{Mobile Cover} \Rightarrow \text{Earphones})}{Support(\text{Earphones})}$$

Substitute the values:

$$Lift(\text{Mobile Cover} \Rightarrow \text{Earphones}) = \frac{0.75}{0.8}$$

$$Lift(\text{Mobile Cover} \Rightarrow \text{Earphones}) = 0.9375$$

Thus, the lift of the rule is: 0.9375 or 93.75%

Final Answer: For the rule

Mobile Cover $\Rightarrow$ Earphones

the results are:

- Support = 0.6 or 60%
- Confidence=0.75 or 75%
- Lift=0.9375 or 93.75%

The support value of **0.6** means that 60% of all transactions contain both Mobile Cover and Earphones together. The confidence value of **0.75** means that 75% of customers who buy a Mobile Cover also buy Earphones. This suggests a fairly frequent co-occurrence between the two products.

However, the lift value is **0.9375**, which is less than 1. This means that although these two items appear together in several transactions, buying a Mobile Cover does not increase the probability of buying Earphones beyond their usual occurrence in the dataset. Therefore, this association rule is **not very strong**.

While solving Association Rule numerical, always follow this sequence. First, identify the rule $X \Rightarrow Y$. Then count the transactions containing X , the transactions containing Y , and the transactions containing both X and Y . After that, calculate support, confidence, and lift using the standard formulas. Finally, interpret the lift value carefully. If lift is greater than 1, the association is positive and strong; if it is equal to 1, there is no special association; and if it is less than 1, the rule is weak.

So the learning flow is:

Count transactions $\rightarrow$ Find Support $\rightarrow$ Find Confidence $\rightarrow$ Find Lift $\rightarrow$ Interpret

The Advantages and Limitations of Association Rules are as follows:

Advantages of Association Rules:

- Association rule learning is simple and easy to understand.
- It helps uncover hidden relationships in large datasets.
- It is widely used in business applications such as recommendation systems and marketing strategies.
- It can handle large volumes of data efficiently.

Limitations of Association Rules

- The algorithm can generate a large number of rules, many of which may not be useful.
- It requires careful selection of minimum support and confidence thresholds.
- It may not perform well with sparse datasets.
- Interpretation of rules can sometimes be complex when many variables are involved.

Thus, Association rules can be understood as discovering “if-then” relationships in data. The key idea is to identify patterns that frequently occur together and evaluate their strength using support, confidence, and lift. By understanding these measures, one can easily determine whether a rule is meaningful or not.

We learned that the Association Rule Learning is a powerful unsupervised technique that helps in discovering relationships between items in a dataset. By using measures like support, confidence, and lift, it allows us to evaluate the usefulness of these relationships. It plays a crucial role in real-world applications such as market basket analysis, recommendation systems, and decision-making processes.

☛ Check Your Progress 5

Q1. What is Association Rule Learning? Explain its purpose.

.....
.....

Q2. Explain the basic form of an association rule with an example.

.....
.....

Q3. What are the key measures used in Association Rule Learning?

.....
.....

Q4. List applications of Association Rule Learning in real-world scenarios..

.....
.....

11.3 SUMMARY

This unit introduces the concept of **unsupervised learning models**, which are used to analyse data without predefined labels or outcomes. Unlike supervised learning, where models learn from known input-output pairs, unsupervised learning focuses on discovering hidden patterns, structures, and relationships within the data. These models are particularly useful in scenarios such as customer segmentation, pattern recognition, and market analysis, where the goal is to extract meaningful insights from raw and unstructured data.

The unit begins with an overview of unsupervised learning models and their significance in data analysis. It then explores **K-Means Clustering**, a widely used partitioning technique that groups data into a predefined number of clusters by minimizing the distance between data

points and their respective centroids. K-Means is simple, efficient, and suitable for large datasets, making it one of the most popular clustering algorithms.

Further, the unit discusses **Hierarchical Clustering**, which builds a hierarchy of clusters in the form of a dendrogram. This approach allows data to be analyzed at different levels of granularity without requiring the number of clusters to be specified in advance. A key method within this category is **Agglomerative Clustering**, which follows a bottom-up approach by initially treating each data point as a separate cluster and then merging the closest clusters step by step. This method provides a clear understanding of how clusters are formed and related to each other.

Finally, the unit covers **Association Rules**, which focus on discovering relationships and patterns among variables in large datasets. Using measures such as support, confidence, and lift, association rule learning identifies how items are related and how frequently they occur together. This technique is widely used in applications like market basket analysis and recommendation systems.

Overall, this unit provides a comprehensive understanding of key unsupervised learning techniques, highlighting their working principles, applications, and significance in data analysis. It equips learners with the ability to uncover hidden patterns in data and apply appropriate models to solve real-world problems effectively.

11.4 ANSWERS

☛ Check Your Progress 1

Q1. What is Unsupervised Learning? How does it differ from supervised learning?

Solution: refer to Section 11.0

Q2. List the applications of Unsupervised Learning in real-world scenarios.

Solution: refer to Section 11.0

Q3. What are Unsupervised Learning Models? Explain their significance in data analysis.

Solution: refer to Section 11.2

Q4. Briefly explain different types of Unsupervised Learning techniques..

Solution: refer to Section 11.2

☛ Check Your Progress 2

Q1. What is K-Means Clustering? Explain its objective.

Solution: refer to Sub-section 11.2.1

Q2. Explain the working steps of the K-Means algorithm

Solution: refer to Sub-section 11.2.1

Q3. What is the role of centroid in K-Means Clustering?.

Solution: refer to Sub-section 11.2.1

Q4. Explain the Elbow Method for selecting optimal number of clusters (K).

Solution: refer to Sub-section 11.2.1

☛ Check Your Progress 3

Q1. What is Hierarchical Clustering? Explain its key concept.

Solution: refer to Sub-section 11.2.2

Q2. Differentiate between Agglomerative and Divisive approaches in Hierarchical Clustering.

Solution: refer to Sub-section 11.2.2

Q3. What is a dendrogram and how is it used in clustering?

Solution: refer to Sub-section 11.2.2

Q4. Explain different linkage methods used in Hierarchical Clustering.

Solution: refer to Sub-section 11.2.2

☛ Check Your Progress 4

Q1. What is Agglomerative Clustering? Explain its basic approach.

Solution: refer to Sub-section 11.2.3

Q2. Explain the step-by-step working of Agglomerative Clustering.

Solution: refer to Sub-section 11.2.3

Q3. What is single linkage in Agglomerative Clustering?

Solution: refer to Sub-section 11.2.3

Q4. Differentiate between Agglomerative and Divisive Clustering.

Solution: refer to Sub-section 11.2.3

☛ Check Your Progress 5

Q1. What is Association Rule Learning? Explain its purpose.

Solution: refer to Sub-section 11.2.4

Q2. Explain the basic form of an association rule with an example.

Solution: refer to Sub-section 11.2.4

Q3. What are the key measures used in Association Rule Learning?

Solution: refer to Sub-section 11.2.4

Q4. List applications of Association Rule Learning in real-world scenarios.

Solution: refer to Sub-section 11.2.4

11.5 REFERENCES/FURTHER READINGS

1. *Pattern Recognition and Machine Learning* by Christopher M. Bishop, published by Springer (1st Edition, 2006, ISBN: 978-0387310732).
2. *Machine Learning* by Tom M. Mitchell, published by McGraw-Hill (1st Edition, 1997, ISBN: 978-0070428072).

3. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* by Aurélien Géron, published by O'Reilly Media (2nd Edition, 2019, ISBN: 978-1492032649).
4. *Data Mining: Concepts and Techniques* by Jiawei Han, Micheline Kamber, and Jian Pei, published by Morgan Kaufmann (3rd Edition, 2011, ISBN: 978-0123814791)
5. *Practical Statistics for Data Scientists* by Peter Bruce, Andrew Bruce, and Peter Gedeck, published by O'Reilly Media (2nd Edition, 2020, ISBN: 978-1492072942).
6. *Business Analytics: Data Analysis and Decision Making* by S. Christian Albright and Wayne L. Winston, published by Cengage Learning (7th Edition, 2020, ISBN: 978-0357131787).
7. *Decision Analysis for Management Judgment* by Paul Goodwin and George Wright, published by Wiley (5th Edition, 2014, ISBN: 978-1119974017).
8. *Handbook of Statistical Analysis and Data Mining Applications* by Robert Nisbet, John Elder, and Gary Miner, published by Elsevier (2nd Edition, 2017, ISBN: 978-0124166455).
9. *Data Mining and Analysis: Fundamental Concepts and Algorithms* by Mohammed J. Zaki and Wagner Meira Jr., published by Cambridge University Press (1st Edition, 2014, ISBN: 978-0521766333).
10. *Core Concepts in Data Analysis: Summarization, Correlation and Visualization* by Boris Mirkin, published by Springer (1st Edition, 2011, ISBN: 978-0857292872).
11. *Applied Predictive Modeling*, Max Kuhn and Kjell Johnson, 1st Edition, Springer, 2013.
12. *An Introduction to Statistical Learning with Applications in R*, Gareth James, Daniela Witten, Trevor Hastie and Robert Tibshirani, 1st Edition, Springer, 2013.
13. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, Trevor Hastie, Robert Tibshirani and Jerome Friedman, 2nd Edition, Springer, 2009.
14. *An Introduction to Statistical Learning with Applications in R* by Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani, published by Springer (2nd Edition, 2021, ISBN: 978-1071614174).
15. *Time Series Analysis: Forecasting and Control* by George E. P. Box, Gwilym M. Jenkins, Gregory C. Reinsel, and Greta M. Ljung, published by Wiley (5th Edition, 2015, ISBN: 978-1118675021)

UNIT-12 PERFORMANCE EVALUATION MEASURES

- 12.0 Introduction
- 12.1 Expected Learning Outcomes
- 12.2 Confusion matrix,
- 12.3 Accuracy, Precision and Recall
- 12.4 Sensitivity and Specificity
- 12.5 F1-Score,
- 12.6 ROC Curve,
- 12.7 PR Curve
- 12.8 Summary
- 12.9 Answers
- 12.10 References/Further Readings

12.0 INTRODUCTION

Performance evaluation is essential in predictive and classification models because building a model alone does not guarantee that it is reliable, accurate, or useful in real-life situations. A model may produce outputs, but without proper evaluation, we cannot know whether those outputs are correct, consistent, and meaningful. Performance evaluation helps measure how well a model is learning from data and how effectively it makes predictions on new, unseen cases. It also allows us to compare different models and choose the one that performs best for a given problem.

In predictive modelling, evaluation tells us how close the predicted values are to the actual values. In classification models, it helps us understand whether the model is correctly identifying categories or classes. This becomes especially important in practical applications such as medical diagnosis, fraud detection, email spam filtering, or loan approval, where wrong predictions may lead to serious consequences. Therefore, performance evaluation is not just a technical step, but a necessary process to ensure trust, accuracy, and decision-making quality.

For example, consider a medical test model designed to classify whether a patient has a disease or not. Suppose the model is applied to 100 patients and predicts 90 patients as disease-free and 10 as diseased. At first glance, the model may appear accurate. However, if in reality 15 patients actually had the disease and the model correctly identified only 8 of them, then it failed to detect 7 diseased patients. In healthcare, such errors are critical because missed cases may delay treatment. This example shows that simply looking at the number of correct predictions is not enough; proper performance evaluation is needed to fully understand the strengths and weaknesses of a model.

Thus, performance evaluation helps answer important questions such as: Is the model dependable? Is it identifying the right cases? Can it be trusted in real-world use? Hence, it forms a vital part of every predictive and classification modelling process.

In the previous units, different categories of data analysis models were introduced, each designed to solve a particular type of problem. **Unit 8 – Regression Models** focused on predicting continuous numerical values, while **Unit 9 – Time Series Models** dealt with data observed over time and the identification of trends, seasonality, and future patterns. Moving further, **Unit 10 – Supervised Learning Models for Data Analysis** explained how labelled data can be used for prediction and classification, and **Unit 11 – Unsupervised Learning Models for Data Analysis** discussed techniques for discovering hidden structures, patterns, and groupings in unlabelled data. Together, these units provide a broad understanding of how different analytical models are built and applied to real-world problems.

However, building a model is only one part of the analytical process. It is equally important to assess how well the model performs and whether its predictions are reliable enough for practical use. This is the focus of **Unit 12 – Performance Evaluation Measures**. Performance evaluation helps compare models, identify their strengths and weaknesses, and determine their suitability for a given task. In predictive analytics, especially in classification problems, evaluation measures provide a systematic way to judge whether a model is making correct decisions or not.

This unit introduces the fundamental concepts and tools used to evaluate model performance, with particular emphasis on classification-based measures. Among the most important of these are the **confusion matrix**, **accuracy**, **precision**, **recall**, **sensitivity**, **specificity**, and **F1-score**, which help quantify different aspects of prediction quality. In addition, graphical evaluation techniques such as the **ROC curve** and **PR curve** provide deeper insight into a model's behaviour across different decision thresholds. These measures will be discussed in detail later in this unit, forming the basis for understanding how to interpret and compare analytical models effectively.

12.1 EXPECTED LEARNING OUTCOMES

After studying this unit, you would be able to:

- understand the need and importance of performance evaluation in predictive and classification models.
- introduce the **confusion matrix** as a basic tool for comparing actual and predicted outcomes.
- explain key evaluation measures such as **accuracy**, **precision**, **recall**, **sensitivity**, **specificity**, and **F1-score**.
- develop an understanding of how different performance measures reflect different aspects of model effectiveness.
- explain the role of **ROC curve** and **PR curve** in evaluating classification performance at different decision thresholds.

- enable learners to compare, interpret, and validate models more effectively for practical data analysis applications.

12.2 CONFUSION MATRIX

A confusion matrix is a simple and important table used to evaluate the performance of a classification model. It compares the actual outcomes with the predicted outcomes given by the model. In this way, it helps us clearly understand how many predictions are correct and how many are incorrect. It is called a “confusion” matrix because it shows where the model gets confused between classes. The confusion matrix is especially useful because it provides the foundation for almost all classification performance measures. Instead of giving only one overall number, it presents a complete picture of the model’s prediction behaviour.

Basic Structure of a Confusion Matrix: For a **binary classification problem**, the confusion matrix is generally represented as follows:

Actual / Predicted	<u>Predicted Positive</u>	<u>Predicted Negative</u>
<u>Actual Positive</u>	<i>True Positive (TP)</i>	<i>False Negative (FN)</i>
<u>Actual Negative</u>	<i>False Positive (FP)</i>	<i>True Negative (TN)</i>

A confusion matrix has four main components:

1. *True Positive (TP)*: Cases where the actual class is positive and the model also predicts positive. Example: A diseased patient is correctly predicted as diseased.
2. *True Negative (TN)*: Cases where the actual class is negative and the model also predicts negative. Example: A healthy patient is correctly predicted as healthy.
3. *False Positive (FP)*: Cases where the actual class is negative but the model predicts positive. Example: A healthy patient is wrongly predicted as diseased.
4. *False Negative (FN)*: Cases where the actual class is positive but the model predicts negative. Example: A diseased patient is wrongly predicted as healthy.

The confusion matrix is directly related to **Type-I Error** and **Type-II Error** in statistics:

<u>Error Type</u>	<u>Confusion Matrix Component</u>	<u>Meaning</u>
Type-I Error	False Positive (FP)	Rejecting something that is actually true in negative class terms; predicting positive when actual is negative

<u>Error Type</u>	<u>Confusion Matrix Component</u>	<u>Meaning</u>
Type-II Error	False Negative (FN)	Failing to detect something that is actually positive; predicting negative when actual is positive

Important Note:

- *Type-I Error relates to False Positive (FP):* This happens when the model gives a false alarm. Example: A healthy person is predicted to have a disease.
- *Type-II Error relates to False Negative (FN):* This happens when the model misses a real case. Example: A diseased person is predicted to be healthy.
- In many real-life situations, Type-II error may be more dangerous than Type-I error, especially in medical diagnosis, fraud detection, or security systems.

The Performance measures that can be calculated from the confusion matrix, they are as follows:

1. Accuracy
2. Precision,
3. Recall
4. Sensitivity,
5. Specificity,
6. F1-Score,
7. False Positive Rate (FPR)
8. False Negative Rate (FNR)

A brief introduction to these performance measures is given below:

CONFUSION MATRIX

	Predicted Class	
Actual Class	<i>Positive</i>	<i>Negative</i>
<i>Actual Positive</i>	TP	FN
<i>Actual Negative</i>	FP	TN

1. **Accuracy:** Measures the overall correctness of the model.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

2. **Precision:** Measures how many predicted positive cases are actually positive.

$$Precision = \frac{TP}{TP + FP}$$

3. **Recall / Sensitivity / True Positive Rate:** Measures how many actual positive cases are correctly identified.

$$\text{Recall} = \text{Sensitivity} = \frac{TP}{TP + FN}$$

4. **Specificity / True Negative Rate:** Measures how many actual negative cases are correctly identified.

$$\text{Specificity} = \frac{TN}{TN + FP}$$

5. **F1-Score:** Harmonic mean of precision and recall.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

6. **False Positive Rate (FPR):**

$$FPR = \frac{FP}{FP + TN}$$

7. **False Negative Rate (FNR)**

$$FNR = \frac{FN}{FN + TP}$$

These measures help in understanding different aspects of classification performance.

Example 1: Suppose a hospital develops a machine learning model to predict whether a patient has diabetes or not. Out of **100 patients** tested:

- 40 patients actually have diabetes
- 60 patients are actually healthy

Solution: The model gives the following results:

- 35 diabetic patients are correctly predicted as diabetic i.e. **TP = 35**
- 5 diabetic patients are wrongly predicted as healthy i.e. **FN = 5**
- 10 healthy patients are wrongly predicted as diabetic i.e. **FP = 10**
- 50 healthy patients are correctly predicted as healthy i.e. **TN = 50**

So, the confusion matrix becomes:

<u>CONFUSION MATRIX</u>		
		Predicted Class
		<i>Diabetic</i> <i>Healthy</i>
<i>Actual</i>	<i>Diabetic</i>	TP=35 FN=5
	<i>Healthy</i>	FP=10 TN=50

From this confusion matrix, it can be interpreted that:

- The model correctly identified **35 diabetic patients**
- The model missed **5 diabetic patients**
- The model wrongly alarmed **10 healthy patients**
- The model correctly classified **50 healthy patients**

Now some of the performance measures that can be calculated for the above example are:

Accuracy

$$Accuracy = \frac{35 + 50}{100} = \frac{85}{100} = 0.85 = 85\%$$

Precision

$$Precision = \frac{35}{35 + 10} = \frac{35}{45} = 77.78\%$$

Recall / Sensitivity

$$Recall = \frac{35}{35 + 5} = \frac{35}{40} = 87.5\%$$

Specificity

$$Specificity = \frac{50}{50 + 10} = \frac{50}{60} = 83.33\%$$

This example shows that the confusion matrix gives a much deeper understanding than only saying the model is **85% accurate**.

Thus, we can say that the **confusion matrix** is a basic yet powerful tool for comparing **actual outcomes** with **predicted outcomes** in classification problems. It helps identify correct predictions and different types of errors, especially **Type-I error (False Positive)** and **Type-II error (False Negative)**. It also forms the basis for calculating key performance measures such as **accuracy, precision, recall, sensitivity, specificity, and F1-score**. Therefore, learning the confusion matrix is the first and most important step in understanding classification model evaluation.

Now, we will discuss the performance evaluation metrics in a bit detail in the subsequent sections of this unit. Let's begin with Accuracy Precision & Recall.

☛ Check Your Progress 1

Q1. What is a Confusion Matrix? Why is it important in classification models?

.....

Q2. Explain the four components of a Confusion Matrix.

.....

Q3. What is the difference between Type-I and Type-II errors in the confusion matrix?

.....

Q4. How does a confusion matrix provide better insight than accuracy alone?.

.....

12.3 ACCURACY, PRECISION AND RECALL

In classification problems, a model is not judged only by its ability to generate predictions, but also by how correctly and meaningfully those predictions represent the actual class labels. This is why **accuracy, precision, and recall** are among the most important performance evaluation measures in data analysis. These measures are derived from the confusion matrix and are widely used to understand the quality, reliability, and usefulness of a classification model. Each metric conveys a different aspect of model performance, and together they help in developing a more complete understanding of both the model and the dataset on which it is applied.

We learned about these performance evaluation parameters in the above section; here we are going to discuss them in a bit detail. We begin with Accuracy first and later we will take up other performance evaluation parameters viz. Precision and Recall.

Accuracy is the proportion of total predictions that are correct out of all predictions made by the model. It is one of the simplest and most commonly used evaluation measures because it gives an overall idea of how often the model is right. Mathematically, accuracy is defined as:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where TP represents true positives, TN represents true negatives, FP represents false positives, and FN represents false negatives.

Accuracy mainly conveys the **overall correctness** of the model. If a model has high accuracy, it means that it is able to classify a large number of observations correctly. This makes accuracy a useful starting point when evaluating a classifier. It is particularly meaningful when the dataset is balanced, that is, when the number of instances in each class is approximately equal, and when the cost of false positives and false negatives is similar.

However, the usefulness of accuracy becomes limited in the case of **imbalanced datasets**. For example, if 95 out of 100 cases belong to one class, a model may achieve 95% accuracy simply by always predicting the majority class, even though it completely fails to identify the minority class. Therefore, while analysing a model or dataset, accuracy must not be interpreted in isolation. It gives a broad picture of performance, but it does not explain the kind of mistakes the model is making.

For instance, suppose a model is used to classify 100 email messages, and it correctly classifies 80 of them. Then the accuracy is:

$$Accuracy = \frac{80}{100} = 0.80 = 80\%$$

This means the model is correct 80% of the time. Although this seems good, it still does not tell us whether the model is correctly identifying the important class, such as spam emails, and whether it is making too many false alarms.

Precision is the proportion of predicted positive cases that are actually positive. It measures the correctness of positive predictions made by the model. Precision is calculated as:

$$Precision = \frac{TP}{TP + FP}$$

Precision conveys the **reliability of positive predictions**. In other words, when the model predicts that an observation belongs to the positive class, precision tells us how often that prediction is truly correct. A high precision means that the model makes fewer false positive errors. Therefore, precision becomes especially important in situations where false positives are costly or undesirable.

For example, in a fraud detection system, low precision would mean that many normal transactions are incorrectly flagged as fraudulent. In email spam filtering, low precision would mean that many genuine emails are wrongly classified as spam. In medical diagnosis, low precision may cause healthy individuals to be incorrectly identified as diseased, creating unnecessary stress, cost, and further testing.

Precision is useful in understanding whether the model is over-predicting the positive class and whether its positive predictions can be trusted. If precision is low, it may suggest that the model is confusing negative cases with positive ones. This may occur due to class overlap, noisy features, poor model design, or an inappropriate decision threshold. Thus, precision helps in analysing the **quality of positive decisions** made by the model.

Consider a disease prediction model that predicts 50 patients as diseased, but among them only 40 actually have the disease and 10 are healthy. Then the precision is:

$$Precision = \frac{40}{40 + 10} = \frac{40}{50} = 0.80 = 80\%$$

This means that when the model predicts disease, it is correct 80% of the time. Therefore, precision focuses not on all predictions, but specifically on how accurate the positive predictions are.

Recall is the proportion of actual positive cases that are correctly identified by the model. It is also called **sensitivity** or the **true positive rate**. The formula for recall is:

$$Recall = \frac{TP}{TP + FN}$$

Recall conveys the **ability of the model to capture actual positive cases**. It tells us how many of the truly positive observations were successfully identified by the classifier. A high recall means that the model is missing very few positive cases, and therefore false negatives are low. Recall is particularly important in situations where missing a positive case may have serious consequences.

For example, in medical diagnosis, low recall means that diseased patients may go undetected. In fraud detection, low recall means that many fraudulent activities remain unnoticed. In security systems, low recall may lead to actual threats being missed. In such

cases, recall becomes more important than precision, because identifying as many real positive cases as possible is the primary concern.

Recall is useful for understanding whether the classifier is too conservative in assigning the positive label. If recall is low, it may indicate that the model is failing to capture many true positives. This may happen because the threshold for predicting the positive class is too strict, the positive class is underrepresented in the data, or the features are not sufficiently informative. Thus, recall provides insight into the **completeness of positive detection**.

For example, suppose there are 60 patients who actually have a disease, but the model correctly identifies only 48 of them while missing 12. Then the recall is:

$$Recall = \frac{48}{48 + 12} = \frac{48}{60} = 0.80 = 80\%$$

This means the model is able to detect 80% of the actual diseased cases. Hence, recall is concerned with how well the model finds all relevant positive instances.

Combined Understanding of Accuracy, Precision and Recall provides a much deeper understanding of model behaviour than any one of them alone. Accuracy gives the overall correctness of the model, precision shows how reliable the positive predictions are, and recall indicates how effectively the model identifies the actual positive cases. But it is to be noted that:

- A model may have high accuracy but low recall if it mostly predicts the majority class correctly while missing many positives.
- Similarly, a model may have high recall but low precision if it predicts many cases as positive in order to capture most of the real positives, thereby increasing false positives.
- A model may also have high precision but low recall if it predicts positive only when it is very certain, thereby missing many actual positive cases.

Therefore, these measures must always be interpreted together and in the context of the specific application. Their importance depends on the type of problem and on whether false positives or false negatives are more serious in that setting.

Example 2: Consider a binary classification model tested on 100 observations with the following confusion matrix values: true positives = 30, true negatives = 50, false positives = 10, and false negatives = 10.

The accuracy of the model is:

$$Accuracy = \frac{30 + 50}{30 + 50 + 10 + 10} = \frac{80}{100} = 80\%$$

The precision of the model is:

$$Precision = \frac{30}{30 + 10} = \frac{30}{40} = 75\%$$

The recall of the model is:

$$Recall = \frac{30}{30 + 10} = \frac{30}{40} = 75\%$$

This example shows that although the model is 80% accurate overall, its positive predictions are correct only 75% of the time, and it is able to identify 75% of the actual positives. This gives a more balanced understanding of performance than accuracy alone.

Now, it's time to Learn about the **Role of Accuracy, precision, and recall in Understanding the Dataset**: Accuracy, precision, and recall are not only useful for evaluating the model, but also for understanding the dataset itself.

For example:

- if accuracy is very high but recall is low, it may indicate that the dataset is imbalanced and the model is biased towards the majority class.
- If precision is low, it may suggest that there is overlap between the classes, noisy observations, or insufficiently informative features.
- If recall is low, it may indicate that the positive class is difficult to detect or that it is under-represented in the data.

Thus, these measures help reveal the structure and challenges of the dataset in addition to describing the model's behaviour.

Another important point is that precision and recall are sensitive to the classification threshold used by the model. A change in threshold may increase recall while reducing precision, or increase precision while reducing recall. This threshold-based behaviour is one of the reasons why graphical evaluation tools such as ROC and PR curves are important.

Later in this unit, we will learn about ROC curve and PR curves in detail

Now, we extend our discussion to understand the **Relation of Performance evaluation metrics Viz. Accuracy, precision, and recall with the Receiver Operating Characteristic (ROC) Curve and Precision–Recall (PR) curve**

- **Relation with ROC curve:** The ROC curve, or Receiver Operating Characteristic curve, is a graphical method used to evaluate the performance of a classification model across different threshold values. It plots the True Positive Rate (TPR) on the vertical axis against the False Positive Rate (FPR) on the horizontal axis.

It is important to note that, sincerecall is the same as the true positive rate, recall is directly connected to the ROCcurve:

$$TPR = Recall = \frac{TP}{TP + FN}$$

The false positive rate is defined as:

$$FPR = \frac{FP}{FP + TN}$$

Thus, each point on the ROC curve corresponds to a particular decision threshold and represents a trade-off between detecting actual positives and incorrectly classifying negatives as positives.

Recall is directly represented in the ROC curve, whereas accuracy and precision are not directly plotted. However, since all of these measures come from the same confusion matrix values, they are indirectly related. At each threshold, a different confusion matrix is formed, and from that confusion matrix, accuracy, precision, recall, and false positive rate can all be computed.

ROC analysis is useful for understanding how well the model separates the positive and negative classes overall. A better model achieves higher recall at lower false positive rates.

- **Relation with PR Curve:** The PR curve, or Precision–Recall curve, is another graphical evaluation tool that plots precision on the vertical axis and recall on the horizontal axis for different threshold values. Thus, both precision and recall are directly involved in the construction of the PR curve.

The PR curve shows the trade-off between precision and recall at different threshold values. Usually:

- when recall increases, precision may decrease,
- when precision increases, recall may decrease.

This is because making the model more sensitive to capture more positives often also increases false positives.

Unlike the ROC curve, which uses recall and false positive rate, the PR curve focuses entirely on the trade-off between how many positive cases are captured and how reliable those positive predictions are. This makes the PR curve especially useful when dealing with **imbalanced datasets**, where the positive class is rare and where accuracy may be misleading.

As the threshold changes, precision and recall usually move in opposite directions. Increasing recall often means identifying more actual positives, but it may also increase false positives, thereby reducing precision. On the other hand, increasing precision may require stricter conditions for predicting positive, which may reduce recall. The PR curve provides a visual understanding of this balance.

Accuracy is not directly represented in the PR curve, but it still depends on the same confusion matrix from which precision and recall are computed. Therefore, while accuracy provides a general summary, the PR curve offers a deeper understanding of positive class performance.

Finally, we need to address the Conceptual Relationship Among the Metrics and Curves: The confusion matrix forms the basis for all these measures. From the values of TP, TN, FP, and FN, we calculate accuracy, precision, and recall. The ROC curve uses recall and false positive rate derived from the confusion matrix at different thresholds, while the PR curve uses precision and recall derived from the confusion matrix at different thresholds. In this sense, ROC and PR curves are visual extensions of the same underlying evaluation framework.

Thus, recall plays a direct role in both ROC and PR curve analysis, precision plays a direct role in PR curve analysis, and accuracy remains an important overall measure, though it is not directly plotted in either curve.

Together, these metrics and curves help analysts understand not only whether a model is performing well, but also **how** and **why** it is performing in a particular way.

Following table tabulates the Main focus and the best to use cases for the Performance evaluation metrics and the Curves

<u>Measure/Curve</u>	<u>Main Focus</u>	<u>Best Used When</u>
Accuracy	Overall correctness	Classes are balanced
Precision	Reliability of positive predictions	False positives are costly
Recall	Ability to detect positives	False negatives are costly
ROC Curve	Trade-off between recall and false positive rate	General classification comparison
PR Curve	Trade-off between precision and recall	Imbalanced datasets

This section concludes with the understanding that the Accuracy, precision, and recall are fundamental performance evaluation measures used in classification problems. Accuracy conveys the overall correctness of the model, precision conveys the reliability of positive predictions, and recall conveys the ability of the model to identify actual positive cases. These metrics are highly valuable in understanding both the performance of a model and the characteristics of the dataset, such as class imbalance, overlap, and threshold sensitivity.

They are also closely related to graphical evaluation methods. Recall directly contributes to the ROC curve as the true positive rate, while both precision and recall together form the PR curve. Therefore, a proper understanding of accuracy, precision, and recall provides the conceptual foundation for interpreting ROC and PR curves and for performing meaningful evaluation of predictive classification models.

☛ Check Your Progress 2

Q1. What is Accuracy? When is it most useful in model evaluation?

.....
.....

Q2. Define Precision and explain its importance.

.....
.....

Q3. What is Recall? Why is it important in critical applications?

.....
.....

Q4. Why should Accuracy, Precision, and Recall be interpreted together?

.....
.....

12.4 SENSITIVITY AND SPECIFICITY

The discussion of Accuracy, Precision, and Recall made in the last section, naturally leads to two more important performance evaluation measures: Sensitivity and Specificity. These metrics are especially useful when we want to understand not only how many predictions are correct, but also we want to understand that what kind of errors the model is making.

In many real-life classification problems, such as disease detection, fraud identification, spam filtering, or defect detection in manufacturing, a model must be evaluated from both perspectives: how well it identifies the positive cases and how well it rejects the negative cases. This is where Sensitivity and Specificity become essential.

As discussed earlier, a confusion matrix forms the foundation for these metrics. For a binary classification problem, the confusion matrix contains four outcomes: True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN). Accuracy gives the overall correctness of the model, Precision tells us how many predicted positives are truly positive, and Recall tells us how many actual positives are correctly detected. Sensitivity is closely related to Recall, while Specificity complements it by focusing on the negative class.

In this section we are going to discuss about Sensitivity and Specificity in detail. Let's begin with Sensitivity, and later Specificity.

Sensitivity is the ability of a model to correctly identify actual positive cases. It answers the question: Out of all real positive instances, how many did the model correctly detect? Because of this meaning, Sensitivity is also called the True Positive Rate (TPR) or simply Recall in many machine learning contexts.

The formula for Sensitivity is:

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

Here, $TP + FN$ represents all actual positive cases. So, Sensitivity measures the proportion of actual positives that are successfully recognized by the model.

A high Sensitivity means the model misses very few positive cases. This is very important in situations where, failing to detect a positive case is costly or dangerous.

For example, in medical diagnosis for a serious disease, if a sick patient is predicted as healthy, it becomes a False Negative, which may delay treatment and create severe consequences. Therefore, screening tests are often designed to have very high Sensitivity.

Whereas, **Specificity** is the ability of a model to correctly identify actual negative cases. It answers the question like: Out of all real negative instances, how many did the model correctly classify as negative? It is also called the True Negative Rate (TNR).

The formula for Specificity is:

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Here, $TN + FP$ represents all actual negative cases. So, Specificity measures the proportion of actual negatives that are correctly rejected by the model.

A high Specificity means the model generates very few False Positives. This is important in situations where incorrectly labelling a negative case as positive causes unnecessary cost, anxiety, or intervention.

For instance, if a fraud detection system wrongly flags too many normal banking transactions as fraudulent, customers may face inconvenience and the bank may waste resources in manual verification. In such cases, Specificity becomes highly valuable.

Relationship of Sensitivity and Specificity with Accuracy, Precision, & Recall: Sensitivity and Specificity should be understood as part of the same family of performance measures, the usual purpose is listed below:

- Accuracy shows overall correctness.
- Precision focuses on the reliability of positive predictions.
- Recall measures how many actual positives are detected.
- Sensitivity is effectively the same as Recall for the positive class.
- Specificity measures how well the model identifies actual negatives.

Thus, if Recall tells us how well the model captures the positive class, Specificity tells us how well the model protects the negative class from being incorrectly classified. Together, Sensitivity and Specificity provide a more balanced understanding of classifier behaviour, especially when the dataset is imbalanced.

A model can have high Accuracy but poor Sensitivity if the positive class is rare. For example, if only 5 out of 100 patients have a disease, a model that predicts “healthy” for

everyone will still achieve 95% Accuracy, but its Sensitivity will be 0, because it fails to detect any actual patient. This shows why Sensitivity and Specificity are often more informative than Accuracy alone.

Important Note: Recall and Sensitivity are not different in terms of calculation or meaning. Both represent the model's ability to correctly identify actual positive cases. In other words, both answer the same question: out of all the real positive instances present in the dataset, how many did the model correctly predict as positive.

Mathematically, both Recall and Sensitivity are calculated using the same formula:

$$\text{Recall} = \text{Sensitivity} = \frac{TP}{TP + FN}$$

where **TP** stands for True Positives and **FN** stands for False Negatives. This means both metrics focus only on the actual positive class and measure how effectively the model captures it.

The main difference lies only in the terminology and context of usage. The term **Recall** is more commonly used in machine learning, data science, and information retrieval, while **Sensitivity** is more commonly used in medical diagnosis, healthcare testing, and statistics. For example, in a disease detection problem, a doctor may say that the test has high sensitivity, whereas in a machine learning classification problem, a data scientist may describe the same result as high recall.

Thus, Recall and Sensitivity are essentially the same metric with different names used in different domains. There is no numerical difference between them; only the field of application changes the preferred term.

Example 3: Let us consider a medical screening test for detecting a disease among 100 people. Suppose the confusion matrix is as follows:

- True Positive (TP) = 32
- False Negative (FN) = 8
- True Negative (TN) = 50
- False Positive (FP) = 10

Solution: This means:

- 40 people actually have the disease (32+8)
- 60 people are actually healthy (50+10)

Now let us compute the metrics, Sensitivity

$$\text{Sensitivity} = \frac{TP}{TP + FN} = \frac{32}{32 + 8} = \frac{32}{40} = 0.80$$

So, the Sensitivity is 0.80, or 80%.

The results can be interpreted that Out of all actual diseased patients, the model correctly identifies 80% of them. However, it misses 20% of the truly diseased patients. In a real-life medical setting, this means that the screening test is fairly good at detecting disease, but some patients may still go undiagnosed.

Whereas the computation of Specificity reveals that:

$$\text{Specificity} = \frac{TN}{TN + FP} = \frac{50}{50 + 10} = \frac{50}{60} \approx 0.8333$$

So, the Specificity is about 83.33%.

The results can be Interpretated that Out of all healthy people, the model correctly identifies 83.33% as healthy. The remaining 16.67% are wrongly classified as diseased. In a medical context, these are people who may experience unnecessary fear, additional tests, and extra cost even though they are actually healthy.

This numerical example helps us understand how both measures work together.

- Sensitivity = 80% means the test is reasonably effective at finding sick patients.
- Specificity = 83.33% means the test is also reasonably effective at avoiding false alarms among healthy people.

Now suppose this is a cancer screening test. In such a case, doctors may prefer to maximize Sensitivity, because missing a cancer patient is far more dangerous than sending a healthy person for an extra confirmatory test. On the other hand, if this were a fraud detection system, too many false alarms could create customer dissatisfaction. In that case, Specificity would also need to be carefully controlled.

So, the “best” metric depends on the application domain and the cost of the two error types.

Example 4: Consider a fraud detection model is used by a bank and following are the outcomes:

- TP = 18 fraudulent transactions correctly detected
- FN = 2 fraudulent transactions missed
- TN = 160 genuine transactions correctly approved
- FP = 20 genuine transactions wrongly flagged as fraud

Solution: The computation of Sensitivity reveals that:

$$\text{Sensitivity} = \frac{TP}{TP + FN} = \frac{18}{18 + 2} = \frac{18}{20} = 0.90$$

So, Sensitivity = 90%.

The results can be Interpretated that the model catches 90% of fraudulent transactions. This is good, because very few fraud cases are missed.

Whereas, the computation of Specificity reveals that:

$$\text{Specificity} = \frac{TN}{TN + FP} = \frac{160}{160 + 20} = \frac{160}{180} \approx 0.8889$$

So, Specificity = 88.89%.

The results can be Interpretated that the model correctly approves almost 89% of genuine transactions, but about 11% of normal customer transactions are wrongly blocked.

In banking, this may be frustrating for customers, so even though high Sensitivity is desirable, the bank must also improve Specificity to reduce inconvenience.

This real-life example shows that Sensitivity protects against missed positives, while Specificity protects against false alarms.

Trade-off between Sensitivity and Specificity can be understood as follows:

In many classification systems, Sensitivity and Specificity have an inverse relationship. When the classification threshold is adjusted to make the model more sensitive, it may classify more borderline cases as positive. This increases Sensitivity but may also increase False Positives, thereby reducing Specificity. Conversely, if the threshold is made stricter, Specificity may improve, but Sensitivity may decrease because more actual positives are missed.

For example, in a medical test:

- A low threshold may detect almost all diseased patients, giving very high Sensitivity, but it may also wrongly label many healthy people as diseased, lowering Specificity.
- A high threshold may reduce false alarms and improve Specificity, but it may fail to detect some real patients, lowering Sensitivity.

Hence, model evaluation often requires a balance between the two rather than maximizing only one.

Relationship of Sensitivity and Specificity with ROC Curve: Sensitivity and Specificity are directly connected to the ROC (Receiver Operating Characteristic) Curve. In fact, the ROC curve is built using these two measures across different classification thresholds.

The ROC curve plots:

- True Positive Rate (TPR) on the Y-axis, which is the same as Sensitivity
 - False Positive Rate (FPR) on the X-axis,
- Where,

$$FPR = \frac{FP}{FP + TN} = 1 - \text{Specificity}$$

Thus:

- Sensitivity = TPR
- 1 - Specificity = FPR

This means every point on the ROC curve represents a pair of values: one for Sensitivity and one for False Positive Rate. A classifier with better performance has points closer to the top-left corner of the ROC space, because that indicates high Sensitivity and low False Positive Rate, or equivalently, high Specificity.

Thus, an ROC curve can be interpreted in terms of Sensitivity and Specificity, as follows:

- A model near the top-left corner has high Sensitivity and high Specificity.
- A model near the diagonal line performs like random guessing.

- The Area Under the ROC Curve (AUC-ROC) summarizes the model's ability to distinguish between positive and negative classes across all thresholds.

Important Note: So, ROC analysis is essentially a threshold-wise study of the balance between Sensitivity and Specificity.

Relationship of Sensitivity and Specificity with PR Curve: The Precision-Recall (PR) Curve focuses on:

- Precision on the Y-axis
- Recall (Sensitivity) on the X-axis

Thus, Sensitivity is directly part of the PR curve because Recall and Sensitivity are equivalent in binary classification.

However, Specificity does not appear directly in the PR curve. The PR curve is more concerned with how accurately positive predictions are made and how many actual positives are captured. Therefore:

- Sensitivity has a direct relationship with PR curves
- Specificity has an indirect relationship, because if False Positives increase, Precision decreases, and Specificity also decreases

So although Specificity is not explicitly plotted in the PR curve, it still influences the curve through its effect on False Positives.

Practically speaking, the key points from application point of view are as follows:

- In ROC curves, Sensitivity and Specificity are the central concepts.
- In PR curves, Sensitivity remains important, but the counterpart is Precision instead of Specificity.
- For imbalanced datasets, PR curves are often more informative than ROC curves, especially when the positive class is rare. But if we want to understand how well the model distinguishes both positives and negatives, Sensitivity and Specificity through ROC analysis are very useful.

This section concludes with the understanding that Sensitivity and Specificity are powerful performance evaluation measures that deepen our understanding beyond Accuracy, Precision, and Recall.

Sensitivity tells us how effectively the model identifies actual positive cases, while Specificity tells us how effectively it identifies actual negative cases. Together, they reveal the model's behaviour from both sides of the classification task.

In real-life applications, these measures must be interpreted in the context of error costs:

- In healthcare, high Sensitivity is often critical to avoid missing patients.
- In fraud detection, both Sensitivity and Specificity matter because the system must catch fraud while not disturbing genuine users.

- In industrial quality control, Sensitivity helps detect defective products, while Specificity prevents good products from being wrongly discarded.

The connection of Sensitivity and Specificity with ROC curves is very strong, since ROC plots Sensitivity against $(1 - \text{Specificity})$. Their connection with PR curves is also important, especially because Sensitivity appears directly as Recall. Therefore, Sensitivity and Specificity do not stand alone; rather, they form an essential continuation of the broader framework of classification performance evaluation.

Now we are going to discuss the next important performance evaluation metrics i.e. F-1 Score in the next section.

☛ Check Your Progress 3

Q1. What is Sensitivity? Why is it important in classification problems?

.....

Q2. Define Specificity and explain its significance.

.....

Q3. Explain the relationship between Sensitivity and Recall.

.....

Q4. Why is there a trade-off between Sensitivity and Specificity?

.....

12.5 F1-SCORE

After understanding Accuracy, Precision, Recall (Sensitivity), and Specificity, the next important performance evaluation metric is the F1-Score. This metric becomes especially useful when we do not want to judge a model only by overall correctness, but rather by how well it balances the detection of positive cases and the reliability of positive predictions.

In many real-world classification problems, a model may achieve high Accuracy simply because one class dominates the dataset, yet still perform poorly on the class that actually matters.

Similarly, a model may have high Recall but low Precision, or high Precision but low Recall. In such situations, F1-Score provides a single balanced measure that combines both Precision and Recall into one value.

This makes F1-Score a natural continuation of the previous discussion, where:

- Accuracy tells us the overall correctness of the classifier.

- Recall or Sensitivity tells us how many actual positive cases are correctly detected.
- Specificity tells us how well actual negative cases are correctly rejected.
- Precision tells us how trustworthy the positive predictions are.

Based on these performance evaluation metrics the F1-Score is derived to answer a very practical question like: how good is the model at identifying positive cases while also keeping its positive predictions reliable? And this is a valid question in general.

Mathematical the Meaning of F1-Score states that the F1-Score is the harmonic mean of Precision and Recall. It is used because the harmonic mean gives a balanced score only when both values are reasonably high. If one of them is very low, the F1-Score also becomes low. This makes it a better summary measure than simply taking the arithmetic average of Precision and Recall.

The formula for F1-Score is:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Since Recall is the same as Sensitivity in binary classification, the formula can also be written as:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Sensitivity}}{\text{Precision} + \text{Sensitivity}}$$

Where,

$$\text{Precision} = \frac{TP}{TP + FP}$$

and

$$\text{Recall} = \frac{TP}{TP + FN}$$

Thus, F1-Score depends on False Positives as well as False Negatives, because Precision is affected by FP and Recall is affected by FN. This is what makes F1-Score especially important in applications where both kinds of errors matter.

There might be a question in your mind that *why harmonic mean is used to compute F1-Score?* The Question is quite valid and the answer is as follows:

The F1-Score uses the harmonic mean, not the arithmetic mean, because the harmonic mean penalizes imbalance between Precision and Recall.

For example, if Precision = 1.0 and Recall = 0.1, Then arithmetic average would be:

$$\frac{1.0 + 0.1}{2} = 0.55$$

This seems moderate.

But F1-Score would be:

$$F1 = 2 \times \frac{1.0 \times 0.1}{1.0 + 0.1} = \frac{0.2}{1.1} \approx 0.1818$$

This value is much lower and more realistic, because the model is clearly poor if it performs well on only one metric and badly on the other. So, the harmonic mean ensures that the F1-Score becomes high only when both Precision and Recall are high.

Now, there might be another question in your mind that when we already have so many metrics for performance evaluation and again F1-score is also derived from those metrics only, so what is the need of F1-Score or Why do we need F1-Score?

The answer is as follows:

Suppose a model predicts many cases as positive. It may detect most actual positives, giving a high Recall, but it may also produce many false alarms, lowering Precision. On the other hand, a model may be very strict and predict positive only when it is highly certain. In that case, Precision may be high, but many true positive cases may be missed, causing low Recall. If we look at only one of these metrics, we may get a misleading impression of model performance.

F1-Score solves this problem by combining both into one number. A high F1-Score means the model has found a good balance between:

- capturing actual positives
- avoiding incorrect positive predictions

That is why F1-Score is widely used in medical diagnosis, spam detection, information retrieval, fraud detection, and defect detection, especially when the positive class is important and class imbalance exists.

Now it's time to understand the Relationship of F1-Score with Accuracy, Precision, Recall, Sensitivity, and Specificity. F1-Score is closely related to the previous evaluation measures:

- Accuracy gives overall correctness, but may hide poor positive-class performance in imbalanced data.
- Precision tells us how many predicted positives are actually correct.
- Recall/Sensitivity tells us how many actual positives are successfully captured.
- Specificity focuses on the negative class.

F1-Score does not directly include Specificity, because it focuses mainly on the quality of positive-class prediction. However, it strongly depends on Precision and Recall, which themselves are affected by the confusion matrix values.

So, if the study objective is to measure how well the classifier handles the positive class, then F1-Score is often more meaningful than Accuracy. This is especially true when false negatives and false positives are both important.

The Interpretation scale for F1-Score, describes that the value of F1-Score lies between 0 and 1, the meaning of these values of F1-Score are as follows:

- F1 = 1 means perfect Precision and perfect Recall
- F1 = 0 means the model completely fails on either Precision or Recall
- A higher F1-Score indicates a better balance between Precision and Recall

Example 5: *Case of Disease Screening:* Let us continue with an example for medical diagnosis so that the discussion remains connected with Sensitivity and Specificity. Compute F1-Score for the following case and interpret the results.

Suppose a disease screening model gives the following confusion matrix for 100 people:

- True Positive (TP) = 32
- False Positive (FP) = 10
- False Negative (FN) = 8
- True Negative (TN) = 50

Solution:

Step 1: Calculate Precision

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{32}{32 + 10} = \frac{32}{42} \approx 0.7619$$

So, Precision = 76.19%

From the results of Precision, it can be interpreted that Out of all people predicted as diseased, about 76.19% are actually diseased. The remaining predicted positives are false alarms.

Step 2: Calculate Recall / Sensitivity

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{32}{32 + 8} = \frac{32}{40} = 0.80$$

So, Recall or Sensitivity = 80%

From the results of Recall it can be interpreted that Out of all actual diseased people, the model correctly identifies 80%.

Step 3: Calculate F1-Score

$$\begin{aligned} F1 &= 2 \times \frac{0.7619 \times 0.80}{0.7619 + 0.80} \\ F1 &= 2 \times \frac{0.6095}{1.5619} \\ F1 &\approx 0.7805 \end{aligned}$$

So, the F1-Score is approximately 0.78 or 78.05%

From the results of F1-Score it can be interpreted that this value tells us that the disease screening model has a fairly good balance between identifying sick patients and ensuring that predicted positive results are trustworthy. In medical screening, this is important because:

- If Recall is high but Precision is low, too many healthy people may be incorrectly told they are sick.
- If Precision is high but Recall is low, many real patients may go undetected.

An F1-Score of 78.05% indicates that the test is reasonably balanced, but there is still room for improvement. In a real hospital setting, whether this is acceptable depends on the seriousness of the disease. For life-threatening conditions, doctors may tolerate more false positives in order to increase Recall. For confirmatory diagnostic tests, they may want both Precision and Recall to be high, which would increase the F1-Score.

Example 6:Case of Spam Email Detection: Compute F1-Score for the following case and interpret the results.

Consider a spam filter with the following confusion matrix:

- TP = 90 spam emails correctly identified
- FP = 30 normal emails wrongly classified as spam
- FN = 10 spam emails missed
- TN = 170 normal emails correctly classified

Solution:

Compute Precision

$$\text{Precision} = \frac{90}{90 + 30} = \frac{90}{120} = 0.75$$

$$\text{Precision} = 75\%$$

Compute Recall

$$\text{Recall} = \frac{90}{90 + 10} = \frac{90}{100} = 0.90$$

$$\text{Recall} = 90\%$$

Computer F1-Score

$$F1 = 2 \times \frac{0.75 \times 0.90}{0.75 + 0.90}$$

$$F1 = 2 \times \frac{0.675}{1.65}$$

$$F1 \approx 0.8182$$

So, F1-Score = 81.82%

From the results of Precision, Recall and F1-Score it can be interpreted that This spam filter catches most spam emails, because Recall is high at 90%. However, its Precision is only 75%, meaning some genuine emails are wrongly sent to spam. The F1-Score of 81.82%

shows that overall, the model performs well in balancing spam detection with prediction reliability.

This means that the filter is effective, but users may still occasionally lose important emails in the spam folder. If the business goal is aggressive spam blocking, this may be acceptable. If the goal is to avoid misclassifying important client emails, Precision should be improved, which would also improve the F1-Score.

Example 7: Case of Fraud Detection with Imbalanced Data. Compute Accuracy, and F1-Score for the following case and interpret the results. Consider a bank fraud detection system with the following confusion matrix:

- TP = 18
- FP = 20
- FN = 2
- TN = 160

Solution: Compute Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{18 + 160}{200} = \frac{178}{200} = 89\%$$

$$\text{Accuracy} = 89\%$$

This seems high Accuracy.

Compute Precision

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{18}{18 + 20} = \frac{18}{38} \approx 0.4737$$

$$\text{Precision} = 47.37\%$$

Compute Recall

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{18}{18 + 2} = \frac{18}{20} = 0.90$$

$$\text{Recall} = 90\%$$

Compute F1-Score

$$F1 = 2 \times \frac{0.4737 \times 0.90}{0.4737 + 0.90}$$

$$F1 = 2 \times \frac{0.4263}{1.3737}$$

$$F1 \approx 0.6207$$

So, F1-Score = 62.07%

This is a very important example. From the results of Accuracy, Precision, Recall and F1-Score it can be interpreted that Accuracy is 89%, which looks excellent. Recall is also very

high at 90%, meaning the model catches most fraudulent transactions. But Precision is poor at only 47.37%, which means that more than half of the transactions flagged as fraud are actually genuine.

The F1-Score of 62.07% reveals the true picture much better than Accuracy. It shows that although fraud detection is good, the model creates too many false alarms. In a real banking system, this would lead to customer frustration, transaction blocking, and increased manual verification workload. Thus, the F1-Score provides a more realistic evaluation than Accuracy alone.

Now it's time to discuss the **Relationship of F1-Score with ROC Curve**: The relationship of F1-Score with the ROC curve is indirect, because the ROC curve plots:

- True Positive Rate (TPR) = Recall = Sensitivity on the Y-axis
- False Positive Rate (FPR) = $1 - \text{Specificity}$ on the X-axis

Whereas, the F1-Score depends on Recall and Precision, and ROC depends on Recall and FPR, the two are related but not identical.

Key point to be noted is that a model with a good ROC performance may not always have the best F1-Score, especially in imbalanced datasets. This happens because ROC considers the trade-off between Sensitivity and False Positive Rate, but does not directly account for Precision. Since Precision depends heavily on the number of False Positives and the prevalence of the positive class, F1-Score may give a different impression.

Practically this means that:

- ROC is useful for studying classifier performance across thresholds.
- F1-Score is useful when you want one threshold-specific measure that balances Precision and Recall.
- In imbalanced datasets, F1-Score is often more informative than ROC-AUC for the positive class.

Thus, F1-Score is not directly plotted on the ROC curve, but it is still influenced by threshold changes that affect Precision and Recall.

Relationship of F1-Score with PR Curve: The relationship between F1-Score and the Precision-Recall (PR) Curve is much stronger, because the PR curve plots:

- Recall on the X-axis
- Precision on the Y-axis

Since, F1-Score is calculated directly from Precision and Recall, every point on the PR curve corresponds to a possible F1-Score at a given threshold.

The important insight from PR curve is that the F1-Score can be seen as a summary of a point on the PR curve. A better point on the PR curve, where both Precision and Recall are high, usually gives a higher F1-Score. In fact, F1-Score is often used to choose the best

classification threshold from the PR curve by selecting the threshold where the balance between Precision and Recall is strongest.

Practically this means that:

- PR curves help visualize trade-offs between Precision and Recall across thresholds.
- F1-Score helps summarize this trade-off at one chosen threshold.
- For imbalanced datasets, the PR curve and F1-Score together are often more meaningful than ROC analysis.

So, among all curve-based performance tools, F1-Score is most closely tied to the PR curve.

Finally, we learned that apart from other metrics the F1-Score is also an important performance evaluation metrics, but we should know when to use it i.e. When to use F1-Score? So, it is to be noted that F1-Score is especially appropriate when:

- the dataset is imbalanced
- the positive class is important
- both False Positives and False Negatives matter
- we need a single measure combining Precision and Recall

The analysis of F1-score is widely required in the case like disease diagnosis, fraud detection, spam classification, document retrieval, defect detection in manufacturing, intrusion detection in cybersecurity

We learned that F1-Score is very useful but there are some **Limitations of F1-Score** and they are listed below:

- It ignores True Negatives, so it does not reflect Specificity.
- It focuses mainly on the positive class.
- It may not be enough when negative-class performance is equally important.
- In such cases, it should be used along with Accuracy, Specificity, and ROC analysis.

So, F1-Score should not replace all other metrics. It should be used as part of a broader evaluation framework.

This section concludes with the understanding that F1-Score is an important performance evaluation metric that naturally extends the earlier discussion on Accuracy, Precision, Recall, Sensitivity, and Specificity. While Accuracy gives overall correctness and Recall/Sensitivity and Specificity explain class-wise detection ability, F1-Score provides a balanced single measure of how well the classifier handles the positive class by combining Precision and Recall.

Its main strength lies in situations where class imbalance exists and where both false alarms and missed positive cases are important. Through real-life examples such as medical diagnosis, spam filtering, and fraud detection, it becomes clear that F1-Score often gives a more realistic view of performance than Accuracy alone.

Its connection with the ROC curve is indirect, because ROC focuses on Recall and False Positive Rate. Its connection with the PR curve is direct, because F1-Score is derived from Precision and Recall, the two axes of the PR plot itself. Therefore, F1-Score is one of the most practical and meaningful metrics for evaluating classification models, especially when the positive class is of central interest.

☛ Check Your Progress 4

Q1. What is F1-Score? Why is it used in model evaluation?

.....
.....

Q2. Write the formula of F1-Score and explain its components.

.....
.....

Q3. Why is F1-Score preferred over Accuracy in some cases?

.....
.....

Q4. What does a high F1-Score indicate about a model?

.....
.....

12.6 ROC CURVE

After understanding Accuracy, Precision, Recall (Sensitivity), Specificity, and F1-Score, the next important performance evaluation tool is the ROC Curve, that is, the Receiver Operating Characteristic Curve. The ROC curve is not just a single numerical metric like Accuracy or F1-Score; rather, it is a graphical performance evaluation tool that helps us study how a classification model behaves at different decision thresholds. This makes it a natural continuation of the previous discussion. Accuracy, Precision, Recall, Sensitivity, Specificity, and F1-Score evaluate model performance at a chosen threshold, whereas the ROC curve shows how the classifier performs across all possible thresholds.

This is especially important because many classification models do not directly output final class labels such as “positive” or “negative.” Instead, they often output a score or probability. A threshold is then applied to convert that score into a predicted class. For example, if a disease prediction model gives a patient a probability of 0.72 of having a disease, a threshold such as 0.50 may classify the patient as diseased. But if the threshold is changed to 0.80, the same patient may be classified as healthy. Thus, model performance changes with the threshold, and the ROC curve helps us visualize this behaviour.

Basically, ROC Curve is a graph that plots:

- **True Positive Rate (TPR)** on the Y-axis
- **False Positive Rate (FPR)** on the X-axis

We already learned that the **True Positive Rate** is the same as **Recall** or **Sensitivity**:

$$TPR = \text{Sensitivity} = \text{Recall} = \frac{TP}{TP + FN}$$

And, the **False Positive Rate** is defined as:

$$FPR = \frac{FP}{FP + TN}$$

Since Specificity is:

$$\text{Specificity} = \frac{TN}{TN + FP}$$

we can also write:

$$FPR = 1 - \text{Specificity}$$

Thus, the ROC curve directly connects with the previously discussed metrics i.e. Sensitivity and Specificity where:

- **Sensitivity / Recall** appears on the Y-axis
- **Specificity** appears indirectly through $(1 - \text{Specificity})$ on the X-axis

So, the ROC curve is fundamentally a graphical representation of the trade-off between **Sensitivity** and **Specificity** across different threshold values.

Now, there might be a question in your mind that when we already have so many metrics for performance evaluation, why do we need ROC Curve for performance evaluation?

The answer is as follows:

A classifier rarely performs equally well at all thresholds. If we choose a very low threshold, the classifier may label many cases as positive. This increases Sensitivity because fewer actual positives are missed, but it may also increase False Positives, thereby reducing Specificity. On the other hand, if we choose a very high threshold, the classifier becomes stricter, reducing False Positives and improving Specificity, but it may miss more actual positive cases, lowering Sensitivity.

Therefore, instead of checking only one threshold, the ROC curve helps us evaluate the model over the entire range of threshold values. This gives a broader understanding of how well the model can separate positive and negative classes.

Now, let's understand the **Construction of ROC Curve**: To construct an ROC curve, we take the predicted probabilities or scores from the classifier and apply different thresholds, such as 0.9, 0.8, 0.7, 0.6, and so on. For each threshold, we calculate:

- True Positive Rate (Sensitivity / Recall)

- False Positive Rate (1–Specificity)

These pairs of values are then plotted as points on a graph. Joining these points gives the ROC curve.

It is Important to note for ROC points that:

- **(0, 0)** means the classifier predicts everything as negative
- **(1, 1)** means the classifier predicts everything as positive
- The ideal classifier is near **(0, 1)**, where False Positive Rate is 0 and True Positive Rate is 1
- A random classifier tends to lie near the diagonal line from **(0,0)** to **(1,1)**

Thus, the closer the ROC curve is to the **top-left corner**, the better the classifier is at distinguishing between the two classes.

Example 8: Let us consider a **disease detection model** for 10 patients. Suppose the actual class and predicted probability of disease are:

<u>Patient</u>	<u>Actual Class</u>	<u>Predicted Probability</u>
P1	1	0.95
P2	1	0.90
P3	1	0.80
P4	1	0.60
P5	0	0.70
P6	0	0.55
P7	0	0.40
P8	0	0.30
P9	0	0.20
P10	0	0.10

Solution: Here Actual Class can be coded as, **1 = diseased, 0 = healthy**. Referring to the table above it can be understood that there are 4 actual positive cases and 6 actual negative cases. Now, let us compute TPR and FPR at different thresholds.

Threshold > 0.90: No instance predicted positive. TP = 0, FP = 0, FN = 3, TN = 6;

Therefore, $TPR = \frac{0}{3} = 0$ and $FPR = \frac{0}{6} = 0$.

Hence, ROC Point = (0, 0)

Threshold = 0.90: The Predicted positive are patient: P1, P2 with TP=2; FN=2; FP=0; TN=6

$$\text{Therefore } TPR = \frac{2}{4} = 0.5 \text{ and } FPR = \frac{0}{6} = 0$$

Hence, Point on ROC curve: **(0, 0.5)**

Threshold = 0.70: The Predicted positive patients are: P1, P2, P3, P5 with
TP=3; FN=1; FP=1 & TN = 5.

$$\text{Therefore } TPR = \frac{3}{4} = 0.75 \text{ and } FPR = \frac{1}{6} \approx 0.167.$$

Hence, Point on ROC curve: **(0.167, 0.75)**

Threshold = 0.50: The Predicted positive patients are: P1, P2, P3, P4, P5, P6 with
TP = 4; FN = 0; FP = 2; TN = 4

$$\text{Therefore, } TPR = \frac{4}{4} = 1 \text{ and } FPR = \frac{2}{6} \approx 0.333$$

Hence, Point on ROC curve: **(0.333, 1)**

Threshold = 0.30: The Predicted positive patients are: P1, P2, P3, P4, P5, P6, P7, P8 with
TP = 4; FN = 0; FP = 4; TN = 2

$$\text{Therefore, } TPR = \frac{4}{4} = 1 \text{ and } FPR = \frac{4}{6} \approx 0.667$$

Hence, Point on ROC curve: **(0.667, 1)**

Threshold = 0.10: The Predicted positive patients are: all patients, with
TP = 4; FN = 0; FP = 6; TN = 0.

$$\text{Therefore, } TPR = \frac{4}{4} = 1 \text{ and } FPR = \frac{6}{6} = 1.$$

Hence, Point on ROC curve: **(1, 1)**

It can be interpreted from the above results, that as the threshold decreases the **Sensitivity / Recall increases, the False Positive Rate also increases but the Specificity decreases**. This shows the core idea of the ROC curve, that there is a trade-off between detecting more actual positives and avoiding false alarms.

Suppose this model is used for **cancer screening then the ROC curve can be interpreted as:**

- At a **high threshold** like 0.90, the test is very strict. It identifies only the most obvious positive cases. This leads to **no false alarms**, but it misses some actual patients. In real life, this means fewer healthy people are disturbed, but some cancer cases remain undetected.
- At a **moderate threshold** like 0.50, the model detects all cancer patients, which is excellent in terms of Sensitivity. However, some healthy people are wrongly identified as diseased, leading to additional medical tests.
- At a **very low threshold** like 0.10, the model classifies everyone as diseased. This ensures no patient is missed, but it creates too many false positives, which is impractical.

Thus, the ROC curve helps doctors or analysts choose a threshold depending on the seriousness of false negatives and false positives.

After understanding the Construction of ROC Curve and interpretation of the results at various thresholds, it is required the utility and concept of **Area Under the ROC Curve (AUC-ROC)**.

The ROC curve is often summarized by a single value called the **Area Under the Curve (AUC)**. The AUC gives the probability that the classifier will rank a randomly chosen positive instance higher than a randomly chosen negative instance. So, the higher the AUC, the better the model is at separating the two classes.

The **Area Under the ROC Curve** is known as **(AUC-ROC)**, where the meaning of AUC values can be understood as follows:

- AUC = 1.0 means perfect classification
- AUC = 0.5 means the classifier performs like random guessing
- AUC < 0.5 means worse than random, suggesting very poor discrimination

General interpretation scale for the values of AUC is as follows:

- 0.90 to 1.00 = excellent
- 0.80 to 0.90 = very good
- 0.70 to 0.80 = acceptable
- 0.60 to 0.70 = weak
- 0.50 to 0.60 = poor

These ranges are not absolute, but they provide a practical understanding.

Example 9: Calculate the **Area Under the ROC Curve** is known as **(AUC-ROC)**, for the Example 8 given above. The ROC points are determined for respective thresholds, now List ROC points in order. So, the calculated ROC points are **(0, 0) ; (0, 0.5) ; (0.167, 0.75) ; (0.333, 1) ; (0.667, 1) ; (1, 1)**

Solution: Now, Compute AUC using trapezoidal rule: The Area Under the ROC Curve is computed as the sum of trapezoid areas:

$$\text{Area} = \frac{(y_1 + y_2)}{2} (x_2 - x_1)$$

For the given ROC points: **(0, 0) ; (0, 0.5) ; (0.167, 0.75) ; (0.333, 1) ; (0.667, 1) ; (1, 1)**

the **Area Under the ROC Curve (AUC)** is computed using the trapezoidal formula:

$$\text{Area} = \frac{(y_1 + y_2)}{2} (x_2 - x_1)$$

We now compute the area **segment wise**.

Segment 1: From (0, 0) to (0, 0.5) : Here, $x_1 = 0$; $y_1 = 0$ and $x_2 = 0$, $y_2 = 0.5$. Therefore,

$$\text{Area}_1 = \frac{(0+0.5)}{2} (0 - 0) = \text{Area}_1 = \frac{0.5}{2} \times 0 = 0$$

Segment 2: From (0, 0.5) to (0.167, 0.75): Here, $x_1 = 0$; $y_1 = 0.5$ and $x_2 = 0.167$, $y_2 = 0.75$. Therefore, $\text{Area}_2 = \frac{(0.5+0.75)}{2} (0.167 - 0) = \frac{1.25}{2} \times 0.167 = 0.625 \times 0.167 = 0.104375$

Segment 3: From (0.167, 0.75) to (0.333, 1): Here, $x_1 = 0.167$; $y_1 = 0.75$ and $x_2 = 0.333$, $y_2 = 1$.
Therefore, $\text{Area}_3 = \frac{(0.75+1)}{2} (0.333 - 0.167) = \frac{1.75}{2} \times 0.166 = 0.875 \times 0.166 = 0.14525$

Segment 4: From (0.333, 1) to (0.667, 1): Here, $x_1 = 0.333$; $y_1 = 1$ and $x_2 = 0.667$, $y_2 = 1$. Therefore, $\text{Area}_4 = \frac{(1+1)}{2} (0.667 - 0.333) = \frac{2}{2} \times 0.334 = 1 \times 0.334 = 0.334$

Segment 5: From (0.667, 1) to (1, 1): Here, $x_1 = 0.667$; $y_1 = 1$ and $x_2 = 1$, $y_2 = 1$.
Therefore, $\text{Area}_5 = \frac{(1+1)}{2} (1 - 0.667) = \frac{2}{2} \times 0.333 = 1 \times 0.333 = 0.333$

Thus, the Total Area Under the Curve (AUC) can be obtained by adding all segment-wise areas, Giving: $\text{AUC} = 0 + 0.104375 + 0.14525 + 0.334 + 0.333 = 0.916625$

So, the final result is: $\boxed{\text{AUC} \approx 0.917}$

Based on the General interpretation scale for the values of AUC given above it can be interpreted that model performs excellent classification because $\text{AUC} \approx 0.917$ which lies between the range 0.90 to 1.00 that means excellent classification.

So, we learned how to determine AUC. Now, It is to be noted that *the ROC curve should not be seen as separate from Accuracy, Precision, Recall, Sensitivity, Specificity, or F1-Score. Rather, it extends them.* Let's understand the *Link of ROC curve with Accuracy, Precision, Recall / Sensitivity, Specificity and F1-Score*

- **ROC Link with Accuracy:** Accuracy gives a single summary of correct predictions at one threshold. However, if the threshold changes, Accuracy may also change. ROC analysis goes beyond a single threshold and evaluates the classifier more comprehensively.
- **ROC Link with Recall / Sensitivity:** Recall or Sensitivity is directly the Y-axis of the ROC curve. So, every ROC point tells us how well the model detects the positive class at that threshold.
- **ROC Link with Specificity:** Specificity is directly connected to the X-axis, because:

$$FPR = 1 - \text{Specificity}$$

Hence, moving right on the ROC graph means losing Specificity.

- **ROC Link with Precision:** Precision is not directly shown on the ROC curve. This is an important limitation. A classifier may have a good ROC curve and still have poor Precision if the positive class is rare.
- **ROC Link with F1-Score:** F1-Score combines Precision and Recall at a particular threshold. ROC, on the other hand, studies Recall and False Positive Rate across multiple thresholds. So, while F1-Score is a threshold-specific summary, ROC is a threshold-varying performance curve.

To better *understand the relevance of ROC curve let's understand a Real-life Example: Fraud Detection Case* - Consider a bank fraud detection system. The model assigns a fraud probability to each transaction.

- If the threshold is set very low, almost every suspicious transaction is flagged. This increases Sensitivity, so fraud cases are rarely missed. But it also increases False Positives, meaning many genuine customer transactions are blocked.
- If the threshold is set very high, fewer genuine customers are disturbed, so Specificity improves. However, some fraud cases may go undetected.

The ROC curve allows the bank to choose a threshold depending on business priorities. If preventing fraud losses is the top priority, a threshold with high Sensitivity may be selected. If customer convenience is equally important, a more balanced point on the ROC curve may be preferred.

This example illustrates the practical importance of the ROC curves in Real-life.

Now we are in the position to understand the ***Relationship between ROC Curve and PR Curve***: The **ROC curve** and the **PR curve** are both graphical tools for evaluating classification models, but they focus on different aspects.

- The **ROC Curve plots** X-axis as False Positive Rate and Y-axis as True Positive Rate / Recall / Sensitivity. Whereas,
- The **PR Curve plots** X-axis as Recall / Sensitivity and Y-axis as Precision

Thus, the ROC curve emphasizes the trade-off between **Sensitivity and Specificity**, while the PR curve emphasizes the trade-off between **Precision and Recall**.

Further we need to understand that, **When ROC is more useful?** the ROC curves are useful under following use cases, i.e. when:

- both positive and negative classes are important
- the dataset is reasonably balanced

- we want to understand overall class separation ability

AND

The PR curve is more useful under following use cases, i.e. when:

- the dataset is **imbalanced**
- the positive class is rare
- we care strongly about the quality of positive predictions.

In highly imbalanced problems, ROC curves can sometimes look overly optimistic because even a moderate number of false positives may correspond to a low False Positive Rate when negatives are very large. In such cases, the PR curve gives a clearer picture because Precision directly penalizes false positives.

Relationship of ROC Curve with PR Curve through previous metrics becomes easier to understand in continuity with the earlier sections, where we observed that:

- Recall / Sensitivity appears in both ROC and PR curves
- Specificity is represented only in ROC through $(1 - \text{Specificity})$
- Precision is represented only in the PR curve
- F1-Score is closely related to the PR curve because it is the harmonic mean of Precision and Recall
- ROC is therefore more connected with Sensitivity and Specificity
- PR is more connected with Precision, Recall, and F1-Score

So, if the discussion of Sensitivity and Specificity leads to ROC, then the discussion of Precision, Recall, and F1-Score leads to the PR curve.

The Advantages of ROC Curve are as follows:

- it evaluates performance across all thresholds
- it shows the trade-off between Sensitivity and Specificity
- it helps compare multiple classifiers visually
- AUC provides a single summary of discrimination ability
- it is easy to interpret in diagnostic and binary classification problems

Limitations of ROC Curve are as follows:

- it does not directly include Precision
- it may appear optimistic in highly imbalanced datasets
- it does not directly tell the best threshold unless application needs are considered
- two models with similar AUC may behave differently at a threshold important for the problem

Therefore, ROC should be interpreted along with other metrics such as Precision, Recall, Specificity, F1-Score, and sometimes the PR curve.

This section on the ROC curve concludes with the remark that ROC curve is an important performance evaluation tool that extends the earlier ideas of Accuracy, Precision, Recall,

Sensitivity, Specificity, and F1-Score. While those metrics evaluate a model at a fixed threshold, the ROC curve studies model behaviour across all thresholds. It plots Sensitivity (i.e. Recall) against False Positive Rate *i.e.* (1–Specificity), thereby visually capturing the balance between identifying actual positives and avoiding false alarms.

Through threshold-based numerical examples and real-life interpretation in disease screening and fraud detection, it becomes clear that the ROC curve is highly useful when we need to choose a suitable threshold and compare classifiers. Its summary measure, AUC-ROC, indicates the model's overall ability to separate positive and negative classes.

Its relationship with the PR curve is also important. ROC is closely tied to Sensitivity and Specificity, while PR is closely tied to Precision, Recall, and F1-Score. Therefore, ROC and PR curves should be seen as complementary tools rather than competing ones. Together with the previously discussed metrics, the ROC curve provides a complete and meaningful framework for evaluating classification models.

☛ Check Your Progress 5

Q1. What is an ROC Curve? Explain its purpose in model evaluation.

.....

.....

Q2. What are the axes of the ROC Curve?

.....

.....

Q3. What does the Area Under the ROC Curve (AUC) indicate?

.....

.....

Q4. How is ROC Curve useful in model comparison?

.....

.....

12.7 PR CURVE

After understanding Accuracy, Precision, Recall, Sensitivity, Specificity, F1-Score, and the ROC Curve, the next important graphical performance evaluation tool is the PR Curve, that is, the Precision-Recall Curve. The PR curve is especially useful in classification problems where the positive class is more important than the negative class, or where the dataset is imbalanced, meaning one class occurs much less frequently than the other. Thus, the PR curve naturally continues the earlier discussion. While the ROC curve emphasizes the relationship between Sensitivity and Specificity, the PR curve shifts the focus toward Precision and Recall, and therefore connects more directly with F1-Score and the practical usefulness of positive predictions.

In many real-life applications, it is not enough for a model to simply separate positive and negative classes overall. We often want to know two things very clearly: first, how many actual positive cases the model is able to find, and second, how many of the predicted positive cases are truly correct. These two concerns are captured by Recall and Precision, respectively. The PR curve studies the trade-off between these two measures across different threshold values.

We learned from our earlier discussions that the PR Curve is a graph in which, Recall is plotted on the X-axis and Precision is plotted on the Y-axis. Where, Recall is given by:

$$\text{Recall} = \frac{TP}{TP + FN}$$

and Precision is given by:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Since Recall is the same as Sensitivity, the PR curve also indirectly continues the earlier section on Sensitivity and Specificity. However, unlike the ROC curve, the PR curve does not use Specificity or False Positive Rate directly. Instead, it focuses on how Recall changes along with Precision when the classification threshold is varied.

This means the PR curve tells us that as the model tries to identify more positive cases, does it maintain the reliability of its positive predictions, or does Precision start falling because too many false positives are introduced?

Even if so many performance evaluation parameters are studied, we need to understand the relevance/need of the PR curves i.e. **Why do we need PR curves?**

The answer is as follows:

A single value of Precision, Recall, or F1-Score tells us model performance only at one chosen threshold. But many classification models produce probabilities or confidence scores. If we change the threshold, both Precision and Recall will also change.

- At a high threshold, the model predicts positive only when it is very confident. This usually leads to high Precision but lower Recall, because some actual positives are missed.
- At a low threshold, the model predicts more cases as positive. This usually increases Recall, but Precision may decrease because more false positives are included.

Therefore, instead of looking at only one threshold, the PR curve helps us understand this trade-off across many thresholds. This makes it especially useful when the aim is to select a threshold that balances the ability to find positives with the reliability of positive predictions.

In Continuum with previous performance measures, the PR curve should be understood in continuity with the previously discussed metrics, and we need to understand the Link of PR curves with Accuracy, Precision, Recall, F1-Score, Specificity etc.

- **Link of PR curve with Accuracy:** Accuracy measures overall correctness, but it may be misleading in imbalanced datasets. For example, if 95 out of 100 cases are negative, a model that predicts all cases as negative will still achieve 95% Accuracy, even though it completely fails to identify the positive class. The PR curve avoids this problem by focusing only on the positive class performance through Precision and Recall.
- **Link of PR curve with Precision and Recall:** The PR curve is directly built from these two measures. Every point on the PR curve corresponds to a pair of Precision and Recall values at a certain threshold.

- **Link of PR curve with Sensitivity:** Recall and Sensitivity are mathematically the same:

$$\text{Recall} = \text{Sensitivity} = \frac{TP}{TP + FN}$$

Thus, the Recall axis of the PR curve is also the Sensitivity axis.

- **Link of PR curve with Specificity:** Specificity is not directly represented in the PR curve. This is an important difference from the ROC curve. While ROC emphasizes the balance between Sensitivity and Specificity, the PR curve emphasizes the balance between Precision and Recall.
- **Link of PR curve with F1-Score:** The connection between the PR curve and F1-Score is very strong. Since F1-Score is the harmonic mean of Precision and Recall,

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

every point on the PR curve can be associated with an F1-Score. A point with both high Precision and high Recall generally gives a high F1-Score. Therefore, the PR curve is often used to identify the threshold where the classifier achieves the best balance between Precision and Recall.

- **Link of PR curve with ROC Curve:** The ROC curve plots Recall/Sensitivity against False Positive Rate, whereas the PR curve plots Recall against Precision.

Thus:

1. ROC is more connected with Sensitivity and Specificity
2. PR is more connected with Precision, Recall, and F1-Score

In balanced datasets, ROC and PR curves may both provide useful insight. But in highly imbalanced datasets, the PR curve is often more informative because Precision directly reflects the impact of false positives.

In the above section we learned about the Construction of ROC curve and interpreted the results, here we are going to extend our discussion by learning about the Construction of PR Curve. To draw a PR curve, we take the predicted probabilities or scores produced by the classifier and apply different thresholds. For each threshold, we calculate Recall and Precision. Finally, the The resulting pairs are plotted on a graph, and the joined points form the PR curve.

A good PR curve stays near the top-right region, because:

- high Recall means most actual positives are being found
- high Precision means most predicted positives are actually correct

Important Note: If the curve drops sharply downward as Recall increases, it means the model is finding more positives only by sacrificing Precision, that is, by generating many false positives.

Let us consider the same disease-detection style example to maintain continuity.

Example 10: Suppose a classifier gives the following predicted probabilities:

Patient	Actual Class	Predicted Probability
P1	1	0.95
P2	1	0.90
P3	1	0.80
P4	1	0.60
P5	0	0.70
P6	0	0.55
P7	0	0.40
P8	0	0.30
P9	0	0.20
P10	0	0.10

From the above tabular data, it can be observed that:

- Total actual positives = 4
- Total actual negatives = 6

Compute Precision and Recall at different thresholds.

Solution:

Threshold = 0.90: Predicted positive Patients are: P1, P2. With TP = 2; FP = 0; FN = 2

Therefore, Recall = $\frac{2}{4} = 0.5$ and Precision = $\frac{2}{2} = 1.0$. Thus, PR point = (0.5, 1.0)

Threshold = 0.70: Predicted positive Patients are: P1, P2, P3, P5. With TP = 3; FP = 1; FN = 1

Therefore, Recall = $\frac{3}{4} = 0.75$ and Precision = $\frac{3}{4} = 0.75$. Thus, PR point = (0.75, 0.75)

Threshold = 0.50: Predicted positive Patients are: P1, P2, P3, P4, P5, P6 with TP=4, FP=2, FN=0

Therefore, Recall = $\frac{4}{4} = 1$ and Precision = $\frac{4}{6} \approx 0.667$. Thus, PR point = (1.0, 0.667)

Threshold = 0.30: Predicted positive Patients are: P1, P2, P3, P4, P5, P6, P7, P8 with

TP = 4; FP = 4; FN = 0. Therefore, Recall = $\frac{4}{4} = 1$ and Precision = $\frac{4}{8} = 0.5$. Thus,

PR point = (1.0, 0.5)

Threshold = 0.10: Predicted positive Patients are: all patients. With TP = 4; FP = 6; FN = 0.

Therefore, Recall = $\frac{4}{4} = 1$ and Precision = $\frac{4}{10} = 0.4$. Thus, PR point = (1.0, 0.4)

From these results, we can see the central behaviour of the PR curve, and it is observed that:

- At a high threshold, Precision is very high because only the most confident positive cases are selected. But Recall is lower because some actual positives are missed.
- As the threshold decreases, Recall improves because more actual positives are captured.
- However, Precision starts to fall because the model also begins to include more false positives.

This is the exact trade-off shown by the PR curve.

Real-life Example: Disease Detection Case - Suppose this model is used to detect a serious disease, then it can be observed that:

- At a high threshold, the model is very strict. If it predicts disease, it is usually correct, so Precision is high. But it may miss some patients, so Recall is lower.
- At a lower threshold, the model identifies more patients who truly have the disease, so Recall becomes high. But some healthy individuals are also wrongly labeled as diseased, lowering Precision.

In practical terms, the doctor or analyst must decide what is more important:

- avoiding missed disease cases, which means prioritizing Recall

OR

- avoiding unnecessary panic and extra tests for healthy people, which means prioritizing Precision

The PR curve helps in choosing this balance.

Now, let's understand the **Mapping of PR Curve with F1-Score** i.e. *How to connect the above PR points with F1-Score?*

At threshold = 0.90: Precision = 1.0, Recall = 0.5. So, $F1 = 2 \times \frac{1.0 \times 0.5}{1.0 + 0.5} = \frac{1}{1.5} \approx 0.667$

At threshold = 0.70: Precision = 0.75, Recall = 0.75. So, $F1 = 2 \times \frac{0.75 \times 0.75}{0.75 + 0.75} = \frac{1.125}{1.5} = 0.75$

At threshold = 0.50: Precision = 0.667, Recall = 1.0. So, $F1 = 2 \times \frac{0.667 \times 1.0}{0.667 + 1.0} \approx 0.80$

It can be interpreted from the results that the PR point with a stronger balance of Precision and Recall gives a better F1-Score. So, the PR curve visually supports threshold selection, while the F1-Score gives a numerical summary at that chosen threshold.

Real-life Example: Spam Detection Case - Consider an email spam filter.

- If the spam filter uses a high threshold, then only emails that look very strongly suspicious are marked as spam. This gives high Precision, meaning most emails classified as spam are truly spam. But some spam mails may still remain in the inbox, causing lower Recall.
- If the threshold is lowered, more spam emails are captured, increasing Recall. However, some genuine emails may also get moved to spam, lowering Precision.

In such applications, the PR curve is highly useful because users care deeply about the quality of positive predictions, that is, whether emails marked as spam are really spam. Thus, PR analysis often becomes more meaningful than ROC analysis in such contexts.

Now, we need to address the Question, *Why PR Curve is more informative in imbalanced datasets?*

The answer is as follows:

The PR curve is particularly important in imbalanced datasets. Consider a case of fraud detection problem with 10,000 transactions, of which only 100 are fraudulent. A classifier might have a good ROC curve because even several hundred false positives may still produce a relatively low False Positive Rate when divided by the huge number of negatives. But from a business perspective, those false positives can be highly problematic, because many genuine transactions may be blocked.

The PR curve reveals this more clearly because Precision directly decreases when false positives rise. Thus, in rare-event detection problems such as fraud detection, disease detection, cyberattack detection, and fault detection, the PR curve often gives a more realistic view of classifier usefulness.

Shape and interpretation of PR Curve, reveals that A good PR curve remains high across a large range of Recall values, with following observations:

- A curve near the top-right region indicates strong performance
- A curve that falls sharply indicates that Recall is being increased at the cost of much lower Precision
- A higher curve is generally better than a lower one
- Unlike ROC, there is no universal diagonal baseline with exactly the same visual meaning.
- The baseline for a PR curve depends on the proportion of positive cases in the dataset. If the positive class is rare, the baseline Precision is low. This again shows why PR curves are sensitive to class imbalance.

In context of the Average Precision and area under PR curve it is important to understand that just as the ROC curve can be summarized using AUC-ROC, the PR curve can be summarized by the area under the PR curve, often called Average Precision (AP) in practical machine learning. A higher area means the classifier maintains strong Precision across a wide range of Recall values.

However, in academic explanation, it is often enough to state that the PR curve provides a threshold-wise graphical view of Precision and Recall, and that higher area indicates better positive-class performance.

Advantages of PR Curve are as follows:

- it directly focuses on the positive class
- it is more informative for imbalanced datasets
- it connects naturally with Precision, Recall, and F1-Score
- it helps choose thresholds when false positives matter significantly
- it gives a realistic picture in rare-event detection problems

Limitations of PR Curve are as follows:

- it does not directly show Specificity
- it focuses mainly on positive-class performance
- it may be less intuitive than ROC for balanced datasets
- its baseline changes with class distribution, so comparisons across datasets must be done carefully

Therefore, like ROC, it should not be used in isolation. It should be interpreted along with Accuracy, Precision, Recall, Specificity, F1-Score, and application context.

This section concludes with the understanding that the PR curve is an important performance evaluation tool that naturally extends the earlier discussion of Accuracy, Precision, Recall, Sensitivity, Specificity, F1-Score, and the ROC curve. While the ROC curve emphasizes the

trade-off between Sensitivity and Specificity, the PR curve emphasizes the trade-off between Precision and Recall. This makes it especially valuable when the positive class is rare or when the correctness of positive predictions is of primary importance.

Through numerical examples and real-life interpretation in disease screening, spam filtering, and fraud detection, it becomes clear that the PR curve helps us understand how well a model identifies true positives without generating too many false alarms. It is also closely connected with F1-Score, because every point on the PR curve represents a particular balance between Precision and Recall. Therefore, the PR curve is one of the most meaningful graphical tools for evaluating classification models, especially in imbalanced and high-stakes real-life applications.

☛ Check Your Progress 6

Q1. What is a Precision–Recall (PR) Curve? Explain its purpose.

.....
.....

Q2. What are the axes of the PR Curve?

.....
.....

Q3. Why is the PR Curve preferred over ROC Curve for imbalanced datasets?

.....
.....

Q4. Explain the trade-off between Precision and Recall in PR Curve.

.....
.....

12.8 SUMMARY

It is learned that the Unit 12 on Performance Evaluation Measures is focused on assessing how well a predictive or classification model performs. It begins with the confusion matrix, which is the foundation of evaluation and summarizes predictions into four categories: true positives, true negatives, false positives, and false negatives. Based on this matrix, key metrics such as accuracy, precision, and recall are derived. Accuracy measures overall correctness, precision focuses on how many predicted positives are actually correct, and recall evaluates how well the model identifies all actual positives. These metrics help in understanding different aspects of model performance rather than relying on a single measure.

The unit further introduces sensitivity and specificity, which are particularly important in domains like healthcare and fraud detection. Sensitivity (same as recall) measures the ability to correctly identify positive cases, while specificity measures how well the model identifies negative cases. To balance precision and recall, the F1-score is used, which provides a

harmonic mean of both and is especially useful when dealing with imbalanced datasets. These measures ensure that the evaluation captures both correctness and reliability of predictions.

Finally, the unit covers graphical evaluation techniques such as the ROC (Receiver Operating Characteristic) curve and the Precision-Recall (PR) curve. The ROC curve illustrates the trade-off between true positive rate and false positive rate, helping in selecting optimal thresholds, while the PR curve focuses on precision versus recall, making it more informative for imbalanced datasets. Together, these evaluation measures provide a comprehensive framework for analyzing and comparing model performance, enabling data scientists to select the most suitable model for real-world applications.

12.9 ANSWERS

☛ Check Your Progress 1

Q1. What is a Confusion Matrix? Why is it important in classification models?

Solution: Refer to Section 12.2

Q2. Explain the four components of a Confusion Matrix.

Solution: Refer to Section 12.2

Q3. What is the difference between Type-I and Type-II errors in the confusion matrix?

Solution: Refer to Section 12.2

Q4. How does a confusion matrix provide better insight than accuracy alone?.

Solution: Refer to Section 12.2

☛ Check Your Progress 2

Q1. What is Accuracy? When is it most useful in model evaluation?

Solution: Refer to Section 12.3

Q2. Define Precision and explain its importance.

Solution: Refer to Section 12.3

Q3. What is Recall? Why is it important in critical applications?

Solution: Refer to Section 12.3

Q4. Why should Accuracy, Precision, and Recall be interpreted together?

Solution: Refer to Section 12.3

☛ Check Your Progress 3

Q1. What is Sensitivity? Why is it important in classification problems?

Solution: Refer to Section 12.4

Q2. Define Specificity and explain its significance.

Solution: Refer to Section 12.4

Q3. Explain the relationship between Sensitivity and Recall.

Solution: Refer to Section 12.4

Q4. Why is there a trade-off between Sensitivity and Specificity?

Solution: Refer to Section 12.4

☛ Check Your Progress 4

Q1. What is F1-Score? Why is it used in model evaluation?

Solution: Refer to Section 12.5

Q2. Write the formula of F1-Score and explain its components.

Solution: Refer to Section 12.5

Q3. Why is F1-Score preferred over Accuracy in some cases?

Solution: Refer to Section 12.5

Q4. What does a high F1-Score indicate about a model?

Solution: Refer to Section 12.5

☛ Check Your Progress 5

Q1. What is an ROC Curve? Explain its purpose in model evaluation.

Solution: Refer to Section 12.6

Q2. What are the axes of the ROC Curve?

Solution: Refer to Section 12.6

Q3. What does the Area Under the ROC Curve (AUC) indicate?

Solution: Refer to Section 12.6

Q4. How is ROC Curve useful in model comparison?

Solution: Refer to Section 12.6

☛ Check Your Progress 6

Q1. What is a Precision–Recall (PR) Curve? Explain its purpose.

Solution: Refer to Section 12.7

Q2. What are the axes of the PR Curve?

Solution: Refer to Section 12.7

Q3. Why is the PR Curve preferred over ROC Curve for imbalanced datasets?

Solution: Refer to Section 12.7

Q4. Explain the trade-off between Precision and Recall in PR Curve.

Solution: Refer to Section 12.7

12.10 REFERENCES/FURTHER READINGS

1. *Introduction to Information Retrieval* by Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze, published by Cambridge University Press (1st Edition, 2008, ISBN: 978-0521865715).
2. *Pattern Recognition and Machine Learning* by Christopher M. Bishop, published by Springer (1st Edition, 2006, ISBN: 978-0387310732).

3. *Machine Learning: A Probabilistic Perspective* by Kevin P. Murphy, published by MIT Press (1st Edition, 2012, ISBN: 978-0262018029).



ignou
THE PEOPLE'S
UNIVERSITY